

Forensic Analysis of HEIF Image Container Files

Paul Haines

Submitted in partial fulfilment of
the requirements of Edinburgh Napier University
for the Degree of

MSc Advanced Security and Digital Forensics

School of Computing

November 2022

MSc dissertation check list

Milestones	Date of completion	Target deadline
Proposal		Week 3
Initial report		Week 7
Full draft of the dissertation		2 weeks before final deadline

Learning outcome	The markers will assess	Pages ¹	Hours spent
Learning outcome 1 Conduct a literature search using an appropriate range of information sources and produce a critical review of the findings.	* Range of materials; list of references * The literature review/exposition/background information chapter	11 - 33	180 hours
Learning outcome 2 Demonstrate professional competence by sound project management and (a) by applying appropriate theoretical and practical computing concepts and techniques to a non-trivial problem, <u>or</u> (b) by undertaking an approved project of equivalent standard.	* Evidence of project management (Gantt chart, diary, etc.) * Depending on the topic: chapters on design, implementation, methods, experiments, results, etc.	34-68	360 hours
Learning outcome 3 Show a capacity for self-appraisal by analysing the strengths and weakness of the project outcomes with reference to the initial objectives, and to the work of others.	* Chapter on evaluation (assessing your outcomes against the project aims and objectives) * Discussion of your project's output compared to the work of others.	69-82	120
Learning outcome 4 Provide evidence of the meeting learning outcomes 1-3 in the form of a dissertation which complies with the requirements of the School of Computing both in style and content.	* Is the dissertation well-written (academic writing style, grammatical), spell-checked, free of typos, neatly formatted. * Does the dissertation contain all relevant chapters, appendices, title and contents pages, etc. * Style and content of the dissertation.	120 (for example, 80 hours)	
Learning outcome 5 Defend the work orally at a viva voce examination.	* Performance * Confirm authorship	1 hour	

Have you previously uploaded your dissertation to Turnitin? No

Has your supervisor seen a full draft of the dissertation before submission? Yes

Has your supervisor said that you are ready to submit the dissertation? Yes

¹ Please note the page numbers where evidence of meeting the learning outcome can be found in your dissertation.

Authorship Declaration

I, Paul Haines, confirm that this dissertation and the work presented in it are my own achievement.

Where I have consulted the published work of others this is always clearly attributed;

Where I have quoted from the work of others the source is always given. With the exception of such quotations this dissertation is entirely my own work;

I have acknowledged all main sources of help;

If my research follows on from previous work or is part of a larger collaborative research project I have made clear exactly what was done by others and what I have contributed myself;

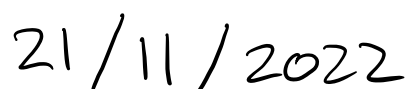
I have read and understand the penalties associated with Academic Misconduct.

I also confirm that I have obtained **informed consent** from all people I have involved in the work in this dissertation following the School's ethical guidelines

Signed:

A handwritten signature in black ink that reads "P. Haines". The signature is written in a cursive style with a large, looped initial 'P'.

Date:

A handwritten date in black ink that reads "21/11/2022". The date is written in a simple, clear style.

Matriculation no: 40448091

General Data Protection Regulation Declaration

Under the General Data Protection Regulation (GDPR) (EU) 2016/679, the University cannot disclose your grade to an unauthorised person. However, other students benefit from studying dissertations that have their grades attached.

Please sign your name below *one* of the options below to state your preference.

The University may make this dissertation, with indicative grade, available to others.

The University may make this dissertation available to others, but the grade may not be disclosed.

P. Haines.

The University may not make this dissertation available to others.

Abstract

In the late 1980's / early 1990s domestic (dial-up) internet connections were significantly slower than they are today. Due to their size, colour images such as photographs were rarely shared and web-sites avoided their use as they introduced significant delays into web-page loading times and adversely affected user-experience. That changed in the mid 1990's when image compression algorithms suitable for photographs (eg. JPEG) were developed. JPEG became the de-facto standard, and to a large-extent is still the default for imaging devices such as digital cameras and scanners today. Although alternative and in some cases superior formats came and went, JPEG remained the most commonly used format for over 20 years until 2017 when Apple selected the High Efficiency Image File (HEIF) format as the default for images captured and processed using its iPhone and other devices.

Apple devices hold a large market-share, consequently any forensic investigation has a reasonably high (and increasing) probability of encountering images saved in the HEIF format. The HEIF format implements advanced features including a modern compression algorithm offering significant improvements in single-image compression over JPEG. However, the new format is far more than a single-image format like JPEG. It is designed to be extensible and flexible, and can contain collections of images, video and more.

This thesis discusses the forensic implications that the HEIF format introduces, most notably, that it may be possible to distribute illegal content or even malware.

The complex structure and lack of adequate forensic tooling means that detailed analysis of HEIF files is currently a relatively complex, difficult, and time consuming task. A prototype analysis tool is proposed and developed that could be used by forensic examiners to quickly process multiple HEIF files, extracting key elements of forensic interest such as all images, metadata and other salient information ready for cross-referencing and further analysis.

Contents

1	INTRODUCTION	11
2	LITERATURE REVIEW.....	12
2.1	Digital Image Acquisition and Common Formats	12
2.2	Digital Image / Digital Media Forensics.....	16
2.3	The High Efficiency Image File Format HEIF.....	21
2.4	Application & Tool support for HEIF – current state-of-the-art.....	26
2.5	HEIF File Forensics.....	29
2.5.1	Forensic Considerations.....	29
2.5.2	HEIF File Forensics – Tools and Techniques	33
2.6	Conclusion	33
3	PROTOTYPE FORENSIC TOOL IMPLEMENTATION.....	34
3.1	Methodology.....	35
3.1.1	Analysis Tool Recommendations and Requirements.....	36
3.1.2	Scope.....	38
3.1.3	Software Development language and tool selection	38
3.1.4	Identification of useful software components and libraries.....	39
3.1.5	Solution design and development	40
3.1.6	Testing.....	43
4	RESULTS.....	48
4.1	Test Case A – Deconstruction of a HEIF file created on an iPhone	49
4.2	Test Case B - Batch processing test on a range of different HEIF files	56
4.3	Test Case C – Control test of the Editor Tool	58
4.4	Test Case D - Internal HEIF file change.....	58
4.5	Test Case E - Additional Hidden Image content.....	60
4.6	Test Case F - Additional Exif data	62
4.7	Test Case G - Additional unrelated content	64
4.8	Test Case H - External Content Reference.....	66
4.9	Test Case I – Nokia Conformance Tests.....	68
4.10	Test Case J – Auxiliary file / Steganography experiment.....	70
5	EVALUATION AND DISCUSSION	74
5.1	Hidden Image Content.....	74

5.2	Additional Hidden / Unrelated Content.....	78
5.3	Exif Metadata.....	79
5.4	External References.....	81
5.5	File, Box, Item Hashes	82
5.6	HEIF File Structure & Content.....	83
5.7	Auxiliary File + Steganography.....	84
5.8	Extensibility / Extend box definitions	85
5.9	Batch Processing.....	85
5.10	Facilitating further analysis using other tools.....	86
6	CONCLUSION AND RECOMMENDATIONS	87
7	REFERENCES	89
8	APPENDICES	96
8.1	Appendix A – Research Proposal.....	96
8.2	Appendix B – Project Management	102
8.3	Appendix C – Extract from process_heif.py	103
8.4	Appendix D – Map file extract.....	104
8.5	Appendix E – Contents file extract	105
8.6	Appendix F – HEIF File Output Directory (root).....	108
8.7	Appendix G – HEIF File Directory Structure.....	109
8.8	Appendix G – Exif Data Extract	110
8.9	Appendix H – Batch Process.....	111
8.10	Appendix I – ItemInfoEntry Box Hidden Flag.....	112
8.11	Appendix J – HeifEditor Script Exceprts	113
8.12	Appendix K – ExifTool output comparison.....	115

List of Tables

Table 1 – Proof-of-concept / Prototype Requirements & Features	37
Table 2 – Summary of test results (Test cases A-H)	48
Table 3 – Summary of Nokia conformance test results (Test case I)	69

List of Figures

Figure 1 - Hierarchical structure of image manipulation, forgery and tampering (Ferreira et al., 2020).....	17
Figure 2 - Image Forgery Detection Methods (Ferreira et al., 2020).....	19
Figure 3 - ISO Base Media File Format, Adapted from (Concion & Apple Inc, 2017a, 2017b; ISO Base Media File Format MPEG, n.d.; ISO/IEC 23008-12:2017 Information Technology — High Efficiency Coding and Media Delivery in Heterogeneous Environments, 2015).	21
Figure 4 - HEIF File Requirements (Concion & Apple Inc, 2017a, 2017b)	22
Figure 5 - High Level view of HEIF File Structure (Heikkilä, 2016).	23
Figure 6 - High level structure of a "moov" box in an HEIF file(Heikkilä, 2016)... ..	25
Figure 7 - General application support for preview of HEIF and HEVC files (McKeown & Russell, 2020)	27
Figure 8 - Beginning of an HEIF file (Apple iPhone 13)	29
Figure 9 – Schematic Structure of a sample HEIF file taken from an Apple iPhone 8+ adapted from (Cheeky4n6monkey, 2017).....	31
Figure 10 - Processing Pipeline	40
Figure 11 - ftyp and meta box details in map file	49
Figure 12 - Hex editor with ftyp box highlighted	49
Figure 13 - ftyp box details from contents file	50
Figure 14 - map file showing primary item, iinf and first infe box information.....	50
Figure 15 - Hex editor with pitm, iinf and infe boxes highlighted	50
Figure 16 - Contents file showing infe entry details (maps to pitm)	51
Figure 17 - Contents file showing details for the first infe entry.	51
Figure 18 - map file showing infe entries 48 and 49	51
Figure 19 - Hex editor highlighting the hidden flag for infe entry 49 (not hidden) ..	51
Figure 20 - Hex editor highlighting the hidden flag for infe entry 1 (hidden).....	51
Figure 21 - manual hash calculation for meta.bin	52
Figure 22 - Tile image - infe id = 1	52
Figure 23 - Composite image - pitm = infe id = 49	52
Figure 24 – map file extract - iref box entries.....	52
Figure 25 - Root of content extraction folder showing extracted images, exif, map and contents files.....	53
Figure 26 - iPhone HEIF bottom left tile	53
Figure 27 - Directory structure (abridged) generated using "tree -f" Linux command.....	54
Figure 28 - Auxiliary Image – depth map	55
Figure 29 - Sample Nokia Images	56
Figure 30 - Output from Batch Processed image files.	57
Figure 31 - Sample Nokia file "bothie_1440x960.heic"	57
Figure 32 – Order of infe entries before edit (from map file).....	59
Figure 33 - Order of Infe entries after edit (from map file).....	59
Figure 34 - "IMG_3802.HEIC" and "IMG_3802_test1.HEIC " loaded into GIMP appear identical	59
Figure 35 - "IMG_3802_test1.HEIC" without additional hidden image tile	60
Figure 36 - "IMG_3802_test2.HEIC" with additional hidden image tile	60
Figure 37 - Additional Tile Exported.....	61
Figure 38 - GIMP opening a HEIF image containing multiple unhidden images. ..	61

Figure 39 - Sections of map file for "IMG_3802_test2.HEIC" without additional Exif data.....	62
Figure 40 - sections of the map file for "IMG_3802_test3.HEIC", with additional Exif data.....	62
Figure 41 - Additional Exif file created	63
Figure 42 - Comparison of Exif files.....	63
Figure 43 - ExifTool output comparison	64
Figure 44 – Sections of map file for "IMG_3802_test3.HEIC" before jpeg is added	64
Figure 45 – Sections of map file for "IMG_3802_test4.HEIC" after the jpeg image was embedded	64
Figure 46 - Extracted JPEG image data	65
Figure 47 - Screen-shot of GIMP showing that the embedded JPEG content is not extracted.....	65
Figure 48 - map file extract for "IMG_3802_test4.HEIC" before a URL was embedded.....	66
Figure 49 - map file extract for "" after URL was embedded.....	66
Figure 50 - Comparison of "dref" sections of contents files	67
Figure 51 - Hex editor comparison of files with and without an embedded URL.....	67
Figure 52 - Original image along with its Auxiliary image	70
Figure 53 - Python script to hide data in an Auxiliary image	71
Figure 54 - Segments from the Map files for the original Auxiliary image	71
Figure 55 - Segments from the Map files for the modified Auxiliary image	71
Figure 56 - Source Auxiliary image and modified Auxiliary image	72
Figure 57 - Modified image viewed on an iPhone	73
Figure 58 - Exported Auxiliary image data showing hidden data	73
Figure 59 - Hex Editor showing a HEIF file contents.	74
Figure 60 - Hex Editor highlighted "infe" boxes.....	75
Figure 61 - Hex Editor highlighting the ID field for an "infe" box	75
Figure 62 - "iref" entries	75
Figure 63 - Map file extract for a simple HEIF file.....	76
Figure 64 – Map file showing hidden images.....	77
Figure 65 - Hidden images exported from an iPhone HEIF file.....	77
Figure 66 - Hex editor showing a segment of a HEIF file with additional hidden content.....	78
Figure 67 - Map file showing additional hidden content.....	78
Figure 68 - Exported JPEG image	78
Figure 69 - "infe" boxes associated with Exif data	79
Figure 70 - "iloc" box Item and Extent entries for IDs 53 and 55	79
Figure 71 - Segment of Exif data in "mdat" box	80
Figure 72 - "infe" and "iloc" entries related to Exif data.....	80
Figure 73 – Excerpt of Exif data data file created for ID 55, extracted using ExifRead Python library	80
Figure 74 - Hex Editor showing the "url" entry in the "dref" box.....	81
Figure 75 - Contents file showing "dref" box and External URL.....	81
Figure 76 - Heif file as viewed using a simple text editor	83

Acknowledgements

I'd like to thank my Sean McKeown for suggesting such an interesting topic, and my friends and family (particularly my wife Ursula) for their support.

1 Introduction

Digital cameras are common and are found in various devices from Smartphones to digital surveillance equipment. As a result digital images are abundant and can be used by forensics experts to help solve crime and prosecute offenders (Ferreira et al., 2020). However, powerful, easy to use editing software along with software tools capable of hiding data within image files are also freely available (Piva, 2013). It is therefore important for digital forensics experts to have appropriate tools and techniques to allow them to analyse digital image files and extract information relevant to a criminal investigation.

This project focuses on the relatively new HEIF media file format, and its implications for forensic investigation methods and tools.

2 Literature Review

2.1 Digital Image Acquisition and Common Formats

Digital image processing began in the late 1950's when the US National Bureau of Standards (NBS, now the National Institute of Standards and Technology, NIST) ran experiments to determine whether automatic processing techniques could be applied to pictorial information to reduce the amount of manual intervention required. The experiments lead to the creation of the first digital image scanning device and the scanning of a photograph into a first-generation electronic computer called a SEAC (Kirsch, 1998; Kirsch et al., 1958). The scanned image was represented by 176x176 binary pixels which required 3872 bytes of memory. Considering its small size and limited bit-depth, it consumed almost 65% of the SEAC's 6000 bytes of memory. In 1957 both memory and storage were limited in capacity and very expensive so at the time Kirsch it considered unrealistic to process and store large quantities of images (Kirsch, 1998).

Kirsch et al. (1958) didn't consider colour images at all, most likely due to limitations in early scanning and display technology, but they did suggest a way in which images could be scanned with different levels of "grayness". However, the additional storage required for grayscale images was not addressed. In fact, a 176x176 pixel image with just four levels of "grayness" (eg. white, light-gray, dark-gray & black) would require double the amount of storage (ie. 2 bits per pixel), far exceeding the total memory capacity of the SEAC. In his later journal article Kirsch (1998) suggests that in 1958 with the benefit of hind-sight, investigation of image compression algorithms could have been a logical next step to help mitigate limitations in memory and storage, but also acknowledged their implementation wouldn't really become feasible until the 1980s when computer processing speeds significantly increased and costs significantly decreased.

Modern digital imaging devices such as cameras and scanners natively capture images in a RAW format. This is essentially unprocessed image data taken directly from the imaging hardware. The first image scanned into the SEAC was a very small (176x176 pixels), very low resolution, very low bit-depth image stored in an easily decodable RAW format. However, modern imaging devices produce far larger, high resolution, full-colour (high bit-depth) images resulting in a very large and complex data

set. Consequently, different device manufacturers represent RAW image data in different ways, and non-conformance to any standard would result in inconsistencies and incompatibilities between manufacturers and devices (Petersilge, 2019). The lack of any standard would also mean that image files could not be exchanged and interpreted across software applications and computer systems. Even something as simple as differences in byte ordering (Big-Endian vs Little-Endian) would lead to incompatibilities and inconsistencies in the interpretation of RAW image data. Standard image formats are therefore essential and most digital cameras will by default automatically post-process RAW image data, converting it to a standard format (typically JPEG) before it is written to storage media.

In the late 1980s and early 1990s though memory and storage capacities were far larger (ie. tens of kilobytes to a few megabytes) than they were in 1958, they were still very limited by today's standards. Additionally, early users of the internet accessed services via dial-up modem with connection speeds of between 14.4Kbps to 28Kbps which is extremely slow when compared to modern consumer internet connection speeds. Memory limitations and slow internet connection speeds meant downloading web-pages with lots of image content was a problem for the typical internet user, with some users choosing to disable images downloads in their browsers in order to access the textual content of web-pages faster (Gellman, 1996). However, in the 1980s and 1990s, microprocessors were also many times faster and more powerful than the SEAC of 1958. Increasing use of the internet meant there was an need to share image content quickly efficiently via the internet so limitations in data storage and internet connection speeds along with increases in microprocessor speed helped to drive the development of image compression algorithms alongside standard image formats.

Nowadays, several standard image file formats are widely supported, with most offering some level of data compression, but the most commonly used formats were developed in the late 1980s and 1990s and lack modern compression algorithms and other modern features (McKeown & Russell, 2020). Some of the most common formats are summarised below:

TIFF (1986) – Tag Image File Format:

Now controlled by Adobe, the TIFF file format specification was originally developed in 1986 by the Aldus Corporation (*TIFF Revision 6.0 Specification*, 1992) and was designed to be as flexible as possible. Flexibility is achieved through the use of Tags in the file header that specify image metadata such as image dimensions, compression etc. TIFF was selected as a common format for scanner manufacturers due in part to its support for multiple images (ie. document pages) per TIFF file, and for its use of the LZW lossless compression algorithm (*TIFF Revision 6.0 Specification*, 1992). Since no data is lost during compression the original image can be fully reconstructed making it ideal for archiving and preservation purposes (Wiggins et al., 2001). Despite being supported by major image editing and Desktop Publishing applications, Wiggins et al suggest that the relatively large file sizes and slow processing speeds limits the use of TIFF file formats (Wiggins et al., 2001). Singh further suggests that it is not a popular choice for digital cameras for the same reasons (N. M. Singh, 2018).

GIF (1987) – Graphics Interchange Format:

GIF was released by Compuserve in 1987 and is still widely supported by web-browsers. It supports up to 8 bits per pixel, allowing the colour of each pixel to be selected from a pallet of up to 256 different colours. Each colour in the pallet is specified by 24 bit RGB value. GIF images are compressed using the Lempel-Ziv-Welch (LZW) lossless compression algorithm (*Graphics Interchange Format*, 1987). GIF images can also contain multiple frames allowing them to be displayed as animations (*GIF89a Specification*, 1990).

JPEG (1992) – Joint Photographic Exports Group:

The JPEG standard was published in 1992 and defines both the file format and the compression-decompression algorithm (CODEC) for digital photographs. JPEG is still the de facto standard image file format for the web and for digital cameras as it employs a 24 bit RGB colour model and along with a lossy compression algorithm which can achieve compression ratios of up to 20:1. However, this comes at the expense of lost data meaning it is not possible to reproduce an exact copy of the original image (Kabachinski, 2007).

PNG (1995) – Portable Network Graphic:

The PNG file format was created in 1995 as a replacement for the GIF format. It was developed as a flexible file format supporting lossless data compression, variable transparency, and other features. It has been generally supported by web browsers since 1996 and is commonly used. An extension to PNG is the Animated PNG (or APNG) format, which is backward compatible with PNG (Ansari et al., 2020).

There have been several unsuccessful attempts to bring legacy image file formats up to date. For example, though the original JPEG format is still the most used for digital cameras and digital images on the internet, there have been attempts to replace it with ones supporting more efficient compression algorithms and modern features:

JPEG 2000 (2000)

The JPEG 2000 standard was released in December 2000 with the intention of replacing the original JPEG standard due to its support for state-of-the-art compression algorithms. However, it is not backward compatible with JPEG, and is not generally supported by web browsers and graphics applications (Heikkilä, 2016).

JPEG XR (2009)

JPEG XR (ie. eXtended Range) was developed by Microsoft in 2009 with the same intention as JPEG 2000, but again it is not backward compatible with JPEG and is only currently supported by Microsoft web browsers (Heikkilä, 2016).

Despite being far superior to the original JPEG format, the newer formats were not widely adopted by device and software manufacturers, due in part to the lack of backward compatibility. In the intervening 30 years since JPEG was developed, other more advanced formats have come and gone, but none seriously challenged the dominance of the original JPEG format. However, in 2017, Apple adopted the HEIF standard as the photographic image format of choice for its devices. Since Apple has a large market share of mobile devices, HEIF image files are becoming more common and the dominance of JPEG may be under threat (see section 2.4).

Nowadays, most people carry high resolution digital cameras around in their pockets embedded in their Smartphones, and immediate uploads to social media and the internet is facilitating widespread sharing. Since those images are then immediately

available to other people via their mobile devices and desktop computers, digital forensic investigations are likely to involve an element of digital image forensics.

2.2 Digital Image / Digital Media Forensics

Forensic Science can be defined as the “application of scientific principles and techniques to matters of criminal justice especially as relating to the collection, examination, and analysis of physical evidence” (Ristenbatt et al., 2022).

Digital Forensics is a specialist branch of forensic science which can be defined as a legal procedure for the collection, analysis, storage and reporting of digital evidence (Li et al., 2021) . Digital Forensics is split into several different areas. Digital Image and Digital Media Forensics is one such area that can be defined as “the investigation of clues and evidence residing in digital media to identify the origin and integrity of the media” (ie. what is its source and has it been tampered with?) (Roy et al., 2020). To support such investigations, investigators must have efficient tools and procedures to effectively locate, extract and preserve digital evidence (Sindhu & Meshram, 2012).

Pollitt (2010) gives a brief history of Digital Forensics in terms of Epochs:

Pre-History – The Pre-History Epoch refers to the early years of computing (1960s to 1980s), when Mainframe Computers were the preserve of education and research establishments, governments, and big business. The earliest Digital Forensics investigations focussed on the analysis of digital audit logs for the investigation of financial and corporate fraud.

Infancy – The Infancy Epoch refers to the mid 1980s to the mid 1990s when home microcomputers and the IBM PC brought computing resources (and the potential for digital crime) to the masses and small business. Hobbyists developed code to access operating system internals, among them were law enforcement personnel, some of whom formed the International Association of Computer Investigative Specialists (IACIS) in 1989 (‘IACIS - History’, n.d.; Pollitt, 2010). Remote access to online services allowed early hackers to develop their skills. Famously, the unauthorised access of Prince Philips Prestel Mailbox in 1985 lead to the creation of the UKs Computer Misuse Act (1990) (Garrad, n.d.).

Childhood – The Childhood Epoch spans the mid 1990s to the mid 2000s when Microsoft Windows and Apple Mac based computers were to be found in every home, and internet access and mobile phones became essential. Digital crime extended beyond financial and corporate fraud, and low level hacking with the first case involving the creation and sharing of illegal images involving minors (Pollitt, 2010).

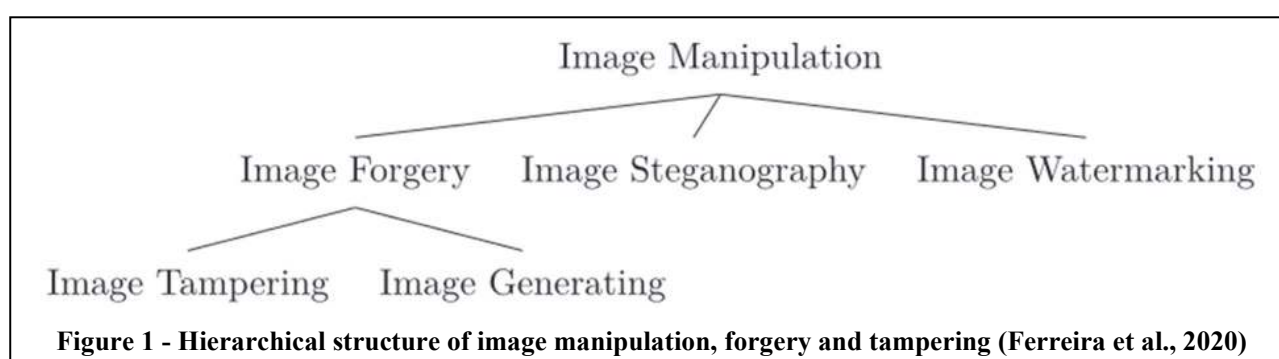
Adolescence – The Adolescence Epoch spans the mid 2000s to 2010, with the development of host-based forensics techniques and software tools such as EnCase and FTK imager (Pollitt, 2010).

Digital Image Forensics began in the Childhood Epoch and has grown to become one of the many specialist domains and disciplines of Digital Forensics (Mani et al., 2022). It is concerned with the collection, analysis, reporting and storage of digital image and video evidence. Its primary aims are to verify the provenance and integrity of digital media evidence (Arshad et al., 2018), and for the identification and extraction of illegal content, or hidden content that may be relevant to a criminal investigation.

Image Authenticity

Identifying the provenance or origin of an image is concerned with the identification of particular aspects related to the device that captured the image. For example, analysis of image metadata or other artefacts in a file may help to verify the make and model of a digital camera (Ferreira et al., 2020). Identification and analysis of hidden watermarks, digital signatures, or aberrations (eg. dead pixels) in a digital image may help to verify the specific device that captured an image (Piva, 2013).

Verifying the integrity / authenticity of a digital image is concerned with analysis of the contents of an image to determine if any changes have been made to the original, resulting in a manipulated image. Ferreira et al., (2020) splits “Image manipulation” into several categories (Figure 1).



Kee et al. (2011) note that it is also possible to use information such as camera manufacturer and model, and GPS location extracted from Exif metadata to verify image authenticity.

Image Tampering

Image Tampering is concerned with deliberate alteration of an image such as copying or moving areas of an image, or resampling (ie. resizing, rotating, up or down scaling etc) an image (Ali et al., 2005). Images are often manipulated using these methods to create “fake-news” content in the media or on social media. For example, Canada’s Green Party altered a photograph of their leader using a sing-use coffee cup (Cecco, 2019).

Image Generating is technique whereby computer (eg. deep-learning) algorithms are used to artificially create parts of an image (Ali et al., 2005). A recent example is the Magic Eraser feature on Google Pixel Smartphones which allows the user to select areas of an image to remove (*How to Use Magic Eraser on the Google Pixel*, n.d.).

Image editing software may make changes to the Exif metadata when saving so this can be used to indicate whether an image has been edited or tampered with (Kee et al., 2011)

Steganography is a method of hiding information within the metadata of an image or hiding data within the image itself. For the latter, the data to be hidden is encoded and applied to an original “cover image” which is altered in an imperceptible way (Ansari et al., 2020). The various methods for this are outside the scope of this project.

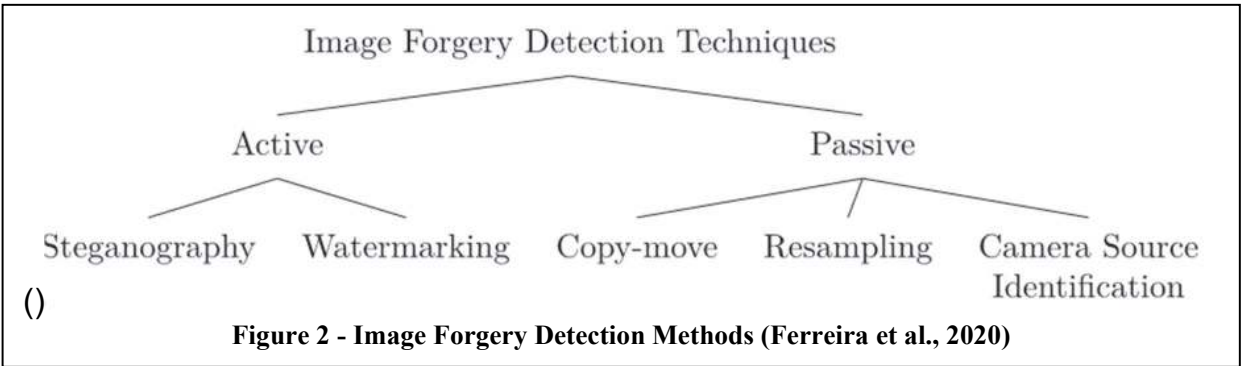
Watermarking typically employs Steganographic techniques to hide a digital signature within an image which can be used to determine whether an image has been altered.

Not all image alterations are signs of surreptitious behaviour however. Watermarking is an example of deliberate image manipulation intended to make it possible to verify the provenance of an image. Similarly, lossy image compression algorithms are used by the most common image file formats, but their intention is only to significantly reduce the number of bytes required to store or transfer the image. Though the image cannot be fully reconstructed in its original form, this is not generally regarded as Image Forgery. However “JPEG Rot” (Kabachinski, 2007) affects image integrity and is the result of repeated minor alterations to an image along with its subsequent

recompression and this could in theory be used as an anti-forensics technique to obfuscate the provenance of an image.

Some image content is classified as prohibited or illegal (*Indecent and Prohibited Images of Children* | *The Crown Prosecution Service*, n.d.). The UK government has tabled an amendment to the Online Safety Bill that may force service providers and developers of applications implementing end-to-end encryption to perform automated scanning of images exchanged using their services looking for illegal content (*Online Safety Bill, As Amended*, 2022; ‘UK Wants to Force Encrypted Platforms to Do CSAM-Scanning’, n.d.). The EU is planning something similar (‘Europe’s CSAM Scanning Plan Unpicked’, n.d.), however Apple quietly back-away from its plans to automatically scan images held on Apple Devices (Francisco, n.d.). Identification of prohibited image content is usually done by comparing file hash signatures against a database of known prohibited images (A. Singh, 2020). A. Singh (2020) notes that there are a number of different hashing techniques that can be used, however even subtle changes in a image file (either in its metadata or the image content itself) may make little or no difference to the image rendered, but would change the file hash completely. However, perceptual hashing techniques can be employed which would generate the same hashes for images with similar rendered content (Samanta & Jain, 2021).

Regarding Digital Image Forensics, Ferreira et al notes the difference between Active and Passive detection methods. Active methods require “knowledge of elements related to the original image” (eg. Steganography and Watermarking), whilst passive methods (such as Copy-Move, Resampling and Camera Source Detection) do not require prior knowledge of the original image (Figure 2).



Mani makes the same distinctions between active and passive techniques but with subtly different definitions. Mani suggests active techniques such as digital signatures or watermarks “may or may not be encrypted” and are embedded into an image such

that they can't be separated and retain their form no matter what processing is done. However Mani appears to contradict himself also suggesting that watermarks consist of private data embedded into the digital image via an unknown encryption key, so must by definition be encrypted (Mani et al., 2022). Mani et al acknowledge the use of Steganography but suggest this is an anti-forensics technique that can be used to hide data within an image, however, they don't acknowledge its use as a potential method of identifying provenance or integrity of an image.

Many Digital Image Forensic techniques can also be applied to Digital Video since they will involve analysis of metadata and analysis of individual images (or frames) within a video stream. However the task proves to be more difficult due the sheer number of individual images in the video stream, and also the complex structure of media container files such as Apple QuickTime (QuickTime File Format, 2002) format, or those based on the ISO Base Image File Format (*ISO/IEC JTC 1SC 29 - ISO/IEC FDIS 14496-12.Pdf*, n.d.). Additionally, the application of advanced lossy compression algorithms (eg. MPEG-x or H.26x) further complicate the forensic analysis process (Milani et al., 2012).

Forensic investigation tasks

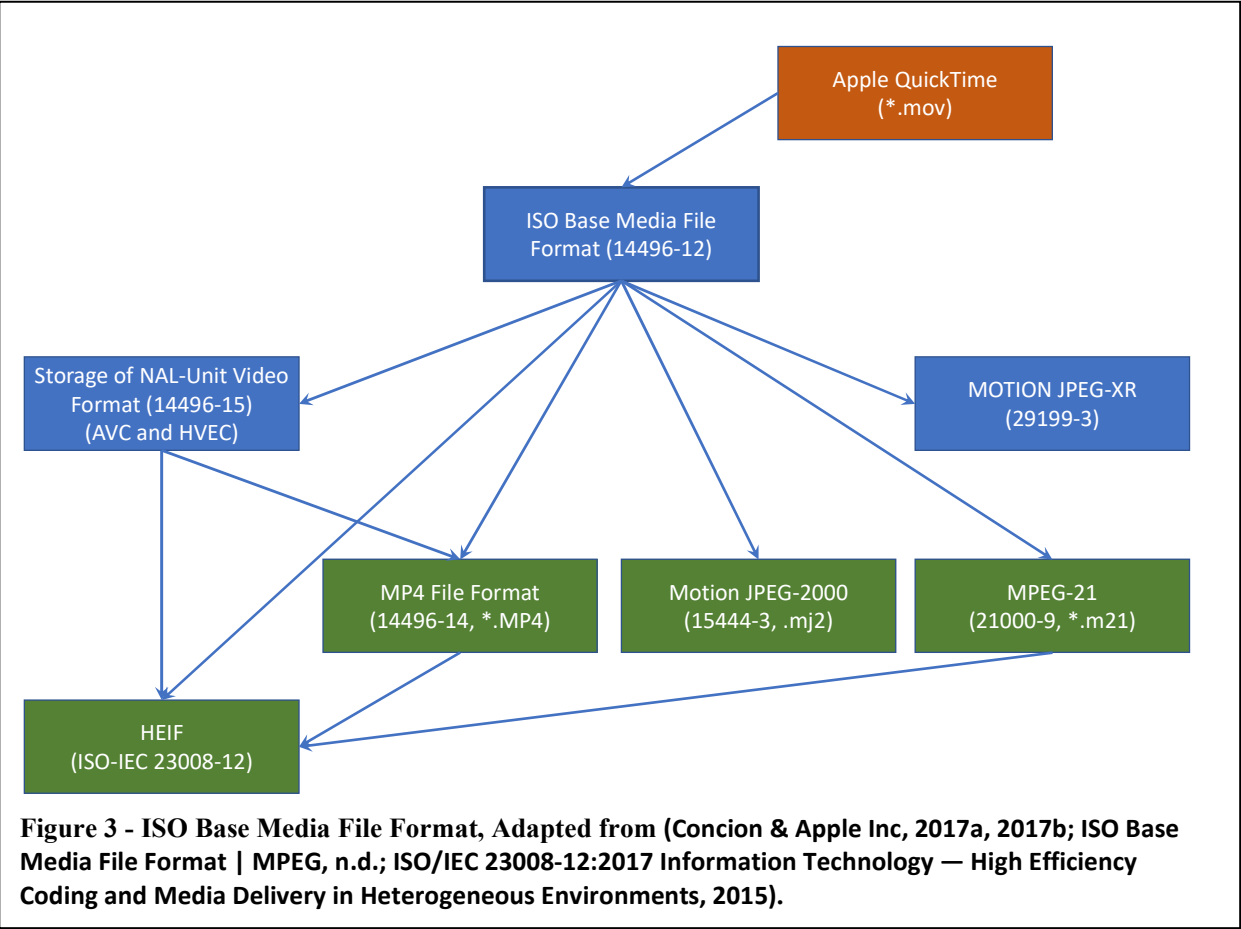
During an investigation, when presented with digital images, a forensic investigator will typically generate file signature hashes which can be compared against databases of known illegal content. Information taken from the image metadata (eg. camera make and model, or GPS coordinates), device generated watermarks and/or image aberrations are used to establish the provenance of an image. Where the authenticity of an image is in question, analysis of the metadata, along with more detailed analysis of the image content itself can be done to determine whether the image has been tampered with.

2.3 The High Efficiency Image File Format HEIF

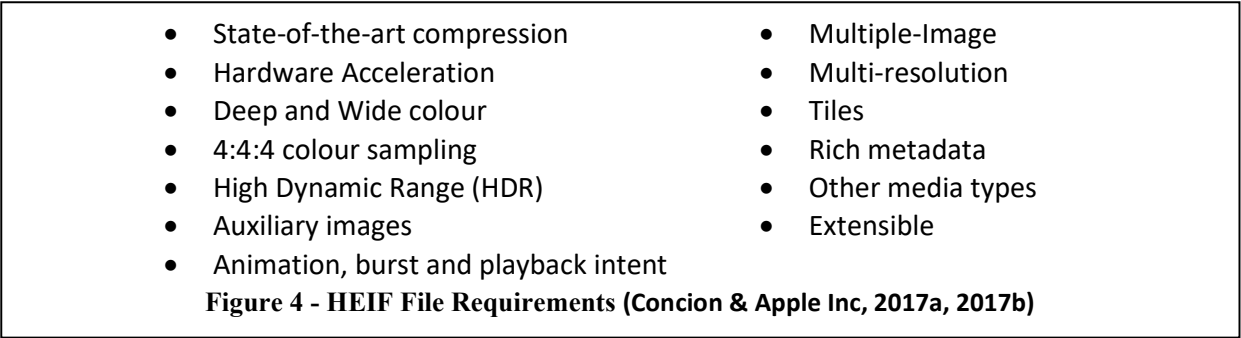
In 2017 Apple announced that it was adopting HEIF as the photographic image file format of choice for its devices due to its support for an advanced compression algorithm with very high compression rates, and other modern features (Concion & Apple Inc, 2017b, 2017a).

The High Efficiency Image File format (HEIF) standard was created in 2017 by the Moving Pictures Experts Group (MPEG) (*ISO/IEC 23008-12:2017 Information Technology — High Efficiency Coding and Media Delivery in Heterogeneous Environments*, 2017).

It is based on the ISO Base Media File Format (ISOBMFF) which itself was derived from Apples own QuickTime video file format (*ISO Base Media File Format | MPEG*, n.d.; *ISO/IEC JTC 1SC 29 - ISO/IEC FDIS 14496-12.Pdf*, n.d.). Figure 3 shows the relationship between, ISO Base Media File Format and other formats.



HEIF is more than a single image file format. It is a complex media container that can hold different types of media including single images, multiple-images, image sequences and video (along with audio). Figure 4 summarises the requirements specified by Apple for the new image format that replaced JPEG (ie. HEIF).



State-of-the-art compression appears top of Apples feature requirements list. The HEIF standard encapsulates the High Efficiency Video Coding (HEVC) or H.265 image compression standard (Hannuksela et al., 2015). HEVC is a modern lossy compression algorithm originally intended for video. Using the temporal inter-frame encoding method (which predicts differences between adjacent frames) for video, HEVC achieves around half of the bit-rate of the Advanced Video Coding (AVC) standard, whilst producing video frames of the same quality (Hannuksela et al., 2015).

HEVC can also be applied to still images using the intra-only encoding method, and studies show a bit-rate saving of around 44% when compared to JPEG images of the same quality (Tung Nguyen & Marpe, 2012).

Interestingly, though the HEIF format technically handles image collections and video, Apple has chosen not to apply the HEIF standard to Video or “Live Photos” (ie. a photographic image along with a short 3 second video of the scene) choosing instead to use the QuickTime file format compressed with HEVC (H.265) for video, and a single HEIF file containing a single compressed HEVC (H.265) image with an accompanying short (3 second) QuickTime video file compressed using HEVC (H.265) for “Live Photos”. This is covered in more detail in section 2.5.1.

Single Images and Image Collections

The HEIF file format (ISO/IEC 23008-12:2017 Information Technology — High Efficiency Coding and Media Delivery in Heterogeneous Environments, 2017) is based on the ISO Base Media File Format (ISOBMFF), and as such is highly structured (ISO/IEC JTC 1SC 29 - ISO/IEC FDIS 14496-12.Pdf, n.d.). It is organized into a hierarchical collection of data structures called “boxes” that contain specific types of information. A box may contain data fields or other boxes. Figure 5 shows a very high level view of the HEIF file structure (Heikkilä, 2016).

Figure 9 on page 31 shows an example file structure from an HEIF (.heic) file taken from an Apple iPhone 8+.

HEIF files must start with an “ftype” (or “FileTypeBox”) box which contains a list of brands that the file conforms to (Hannuksela et al., 2015). This allows viewer applications to determine whether they are able to support the file and the level to which the file will be supported.

It should be noted that neither the ISOBMFF nor the HEIF specifications require a Magic Number to identify the file type. The HEIF specification under general requirements states that all readers “shall parse the FileTypeBox” to confirm that one or more brands are supported (ISO/IEC 23008-12:2017 Information Technology — High Efficiency Coding and Media Delivery in Heterogeneous Environments, 2017).

For generic HEIF files supporting any encoding format the brands shall be “mif1” for a single image or “msf1” for video.

For HEIF files supporting HEVC encoding the brands shall be “heic” (or “heix”) for single images and “hevc” (or “hevx”) for image sequences (Hannuksela et al., 2015;

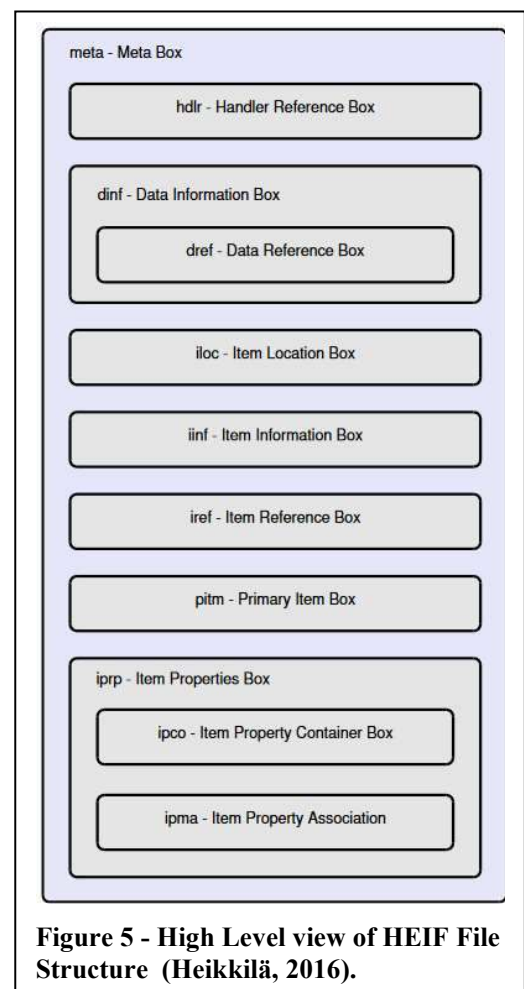


Figure 5 - High Level view of HEIF File Structure (Heikkilä, 2016).

ISO/IEC 23008-12:2017 Information Technology — High Efficiency Coding and Media Delivery in Heterogeneous Environments, 2017).

Apple Image files, use “heic” for the brand and “.heic” for the file name extension.

A single root-level “meta” box follows the “ftype” box and holds information about images (Heikkilä, 2016). The “meta” box begins with a “hdlr” field which is always set to “pict” for HEIF. Another key field is the “pitm” which identifies the primary image item (ie. the (first) image to be displayed by a viewer).

The meta box contains an “iinf” box which holds a separate “infe” entry record for each image item. Apples implementation divides the source image into 512x512 tiles. The sample file in Figure 9 shows 51 “infe” entries. The first 48 describe the individual 512x512 pixel tile images that combine to make the complete image. Item 49 describes a “grid” type derived image that binds the image tile images together to form the complete image. Item 50 describes the 320x420 pixel thumbnail image, and item 51 describes the image Exif data.

The meta box also contains an “iref” box with entries that describe the relationships between “infe” item entries. The sample file in Figure 9 shows a “dimg” (derived image) which is composite of the first 48 image items, a “thmb” which identifies the thumbnail image item and “cdsc” (content description) which identifies the Exif metadata item.

The “iprp” box contained within the “meta” box describes properties associated with the HEIF image. These are applied to the HEIF output creating a derived image which is computationally generated by applying operations to a one or more master images (McKeown & Russell, 2020). This is done by the viewer application without affecting the underlying image data and is therefore a non-destructive process. These properties can be descriptive such as specifying a colour profile, or transformative operations such as crop (“clap”), rotation (“irot”) or mirror (“imir”) and must be supported by the viewer application in order to be applied (McKeown & Russell, 2020). Derived images can be used to render multiple images together in a grid (as in Apples implementation), or as an overlay. Auxiliary images should not be displayed directly, and can be used to apply transparency and depth maps to an HEIF (McKeown & Russell, 2020).

The “iloc” box is another key entity contained within the “meta” box. It describes the location, offsets and sizes of item data within an “mdat” box (Heikkilä, 2016).

The compressed image data bytes are typically stored in “mdat” (“MediaData”) boxes located at the end of the file. The base HEIF standard allows for multiple “mdat” boxes to be included in the file, however for an apple image file, the data is stored in a single “mdat” box (Heikkilä, 2016).

Image Sequences and Video

The HEIF standard allows for image collections, image sequences and video. An image collection is regarded as a set of images with no ordered coding dependency between them. They use Intra-frame encoding only and cannot benefit from greater efficiencies of Inter-frame compression. They can be implemented by adding multiple image items to an HEIF file.

Image sequences and video on the other hand have an ordered coding dependency between images and can benefit from the Inter-frame encoding which identifies the differences between adjacent frames. For image sequences, timing data is advisory, but for video, timing data is mandatory (Hannuksela et al., 2015).

If an HEIF file includes image sequences or video, a “moov” box is present at the root level in the file and is not nested within the “meta” box. The “moov” box contains a “trak” box for each image sequence (or other media type such as audio).

The “trak” boxes contain location data for each of the image sequences, but also contain an “hdlr” box which must be set to “pict” as specified by the HEIF standard (Heikkilä, 2016).

Figure 6 illustrates the high level structure of the “moov” box within an HEIF file.

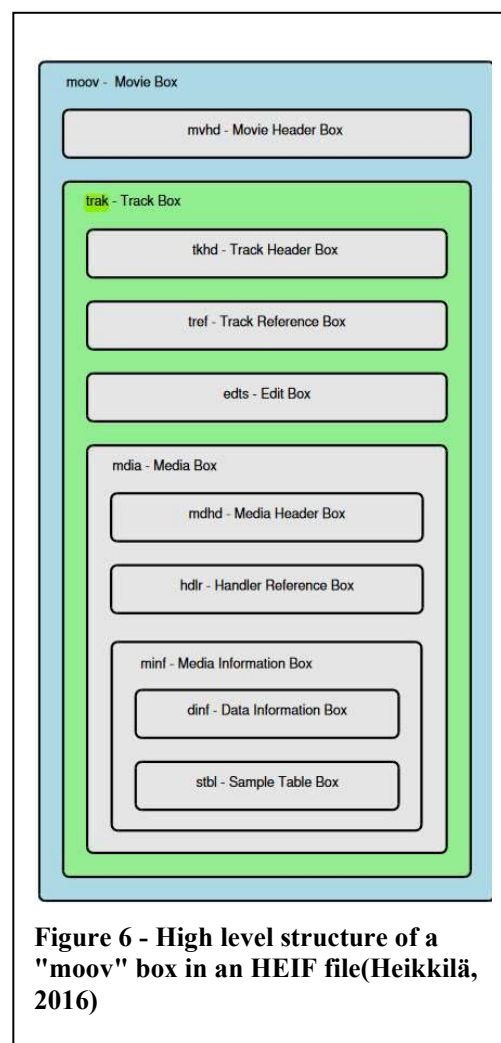


Figure 6 - High level structure of a "moov" box in an HEIF file(Heikkilä, 2016)

Apple does not make use of HEIF files to store image sequences or video, favouring its own QuickTime format instead. It does however support AVC and HEVC compression. Though QuickTime and HEIF files are related, their structure is somewhat different. For example, QuickTime files contain “trak” boxes, but the “trak” boxes do not contain an “hdlr” specifying “pict” as required by the HEIF specification. They do not therefore conform to the HEIF standard.

2.4 Application & Tool support for HEIF – current state-of-the-art

For the reasons highlighted in section 3 above, it is important that forensic investigators are armed with tools and techniques required to analyse the full range of image file formats, and this is certainly the case for the most common image file formats. However, in 2017, without warning, Apple adopted the HEIF file format for images (see section 2.3).

The sudden adoption of the new format came as a surprise to users and software manufacturers, but also to forensic investigators and forensic tool manufacturers. Though general software support for previewing HEIF files was initially extremely limited from a user’s perspective, it has been steadily improving.

General operating system and viewer application support

Leong (Cheeky4n6monkey, 2017, p. 4) noted in 2017 that the HEIF format was not widely supported by viewer or editor applications but there was limited support. The Dropbox Image Preview feature was found to display Apple HEIF image files, and VLC or Infraview could play HEVC encoded movie files. FFmpeg could be used to extract individual frames or transcode HEVC encoded movie files to other formats. Leong also found that metadata could be extracted using the ExifTool application. In 2020 McKeown & Russel (McKeown & Russell, 2020) found that both ExifTool and CopyTrans could access Exif metadata, but only metadata associated with the primary image could be accessed and extracted using these tools.

In 2020, McKeown & Russel (2020) found that Adobe Photoshop supported image viewing and editing but only on Apple Mac computers, and only for the primary image. There was no support for HEIF in the Windows version at all. Windows 10 (build 1809) added support for HEIF and HEVC files in its image and video viewer

applications with the installation of optional components available from the Windows Store, though only the primary image could be displayed in the Windows Photo viewer. GIMP was found to be the best tool for viewing and editing HEIF files on Windows as it includes support for multiple images within an HEIF file. Linux was found to have no native viewer support at all for HEIF files.

Figure 7 shows the level of support found for HEIF for a number of applications (McKeown & Russell, 2020).

Software	Still Cover Image							Additional Still Images				Burst Sequence		Animated Sequence	
	Display	Crop	Mirror	Rotate	Grid	Overlay	Alpha	Master	Aux	Thumb	Hidden	Cover	All	Cover	All
Win10 HEVC Free	✓	✓	✓	✓	✓	X	X	X	X	X	X	X	X	X	X
Win10 HEVC Paid	✓	✓	✓	✓	✓	✓	X 1st img	X	X	✓	X	X	X	X	X
CopyTrans HEIC Windows	✓	X	X	✓	✓	X	X	X	X	✓	X	X	X	X	X
Dropbox Preview	✓	X	X	✓	✓	X	X	X	X	X	X	✓	X	✓	X
macOS Preview 10.14.5	✓	✓	X	X	✓	X no derived	X no derived	✓	X	X	X	✓	one per infe	✓	X
macOS Preview 10.15.2	✓	✓	X	X	✓	✓	✓	✓	X	X	X	✓	✓	✓	✓ no animation
Photoshop Mac 2020 21.0.2	✓	✓	X	X	X	X 1st img	X 1st img	X	X	X	X	✓	1st frame	✓	X
Gimp 2.10.8 Windows	✓	✓	✓	✓	✓	✓	✓	✓	X	X	X	✓	one per infe	✓	X
FFmpeg	X	X	X	X	X	X	X	X	X	X	X	✓	✓	✓	✓

Figure 7 - General application support for preview of HEIF and HEVC files (McKeown & Russell,

Both Leong (Cheeky4n6monkey, 2017) and McKeown & Russel McKeown & Russell (2020) found a number of online tools available for viewing HEIF files, however it was stressed that they should not be used if the content of a sensitive nature.

Images within HEIF files have flags associated with them indicating the role and state of an image. An image can be flagged as Primary, Thumbnail, Auxiliary, Derived or Hidden. McKeown & Russel McKeown & Russell (2020) found that all image viewer applications tested respected the Hidden flag, but setting the auxiliary flag caused many applications to crash. Only the CopyTrans application was found to access embedded thumbnails. Fore HEVC video files, all applications tested ignored the hidden flag.

Several newer online and offline tools for viewing HEIF files are now available (Top 5 HEIC Viewers Available to Windows 11/10/8.1/8/7/XP/Vista, 2022), however Forensic tools support does not appear to have kept pace.

Forensic Tool Support

At the time of writing, Apple has approximately 28% of the global market share with iOS based mobile phones (over 50% share in the USA & UK) (*Mobile Operating System Market Share Worldwide*, n.d.), and the more recent adoption of the HEIF format by Android device manufactures, means more than 1 in 4 forensic investigations involving a mobile phone could require the analysis of HEIF files.

Though general application support appears to be improving, (McKeown & Russell (2020) found that popular digital forensics tools such as X-Ways Forensics (19.8), Encase (8.07), FTK Imager (4.2.1.4) and Autopsy 4.13 had no support for HEIF files whatsoever.

Faulkner (Faulkner, 2018) and Krawetz (Krawetz, 2019) suggested the best forensic approach was to export HEIF images to JPEG. However, viewer and editor applications have limitations in their ability to access and export certain features of HEIF files such as multiple images (and their associated metadata), embedded thumbnails, auxiliary, derived or hidden images, which means forensically relevant data could be missed using this method.

The distinct lack of forensic tools required to analyse the new HEIF file format makes it extremely difficult and time consuming for forensic investigators to do a thorough job.

Other image formats vs Market Share

Interestingly, though several image formats technically superior to JPEG have been developed over the years, and to some extent are used such as WebP and BGP (Mandeel et al., 2021), they failed to displace JPEG as the de-facto standard because they lacked the corporate backing of a consumer titan like Apple. The large mobile device market-share held by Apple, combined with its immediate over-night switch from JPEG to HEIF as a default image format has contributed significantly to the large number of HEIF image files now in circulation.

2.5 HEIF File Forensics

Due to its complex structure, forensic analysis of an HEIF file is has more in common with the analysis of video media container file formats than common single image file formats like JPEG.

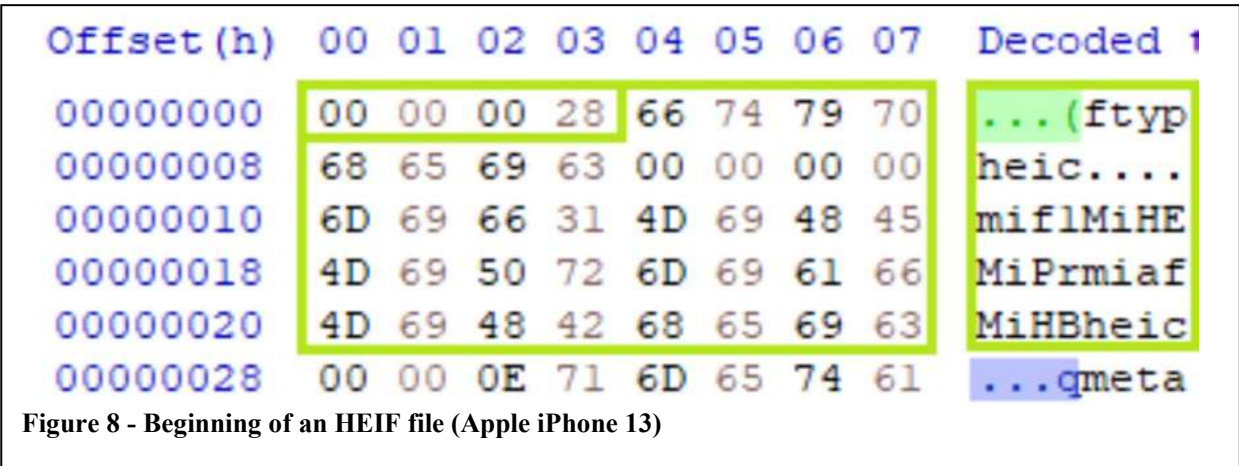
2.5.1 Forensic Considerations

Binary File Structure

Apple .heic and .mov files use Big-Endian byte ordering (meaning the most significant bytes precede the least significant bytes).

Each box begins with the 4-byte that contain the box-size followed by a 4-byte (human readable) box-type (Cheeky4n6monkey, 2017).

Figure 8 shows the beginning of a sample HEIF file taken from an Apple iPhone 13. The First 4 bytes (highlighted in green) show that the size of the “ftyp” box is 0x28 (ie. 40) bytes in total. The 4 bytes denoting the size of the next “meta” box is highlighted in blue.



Example implementation for Single Images (Apple)

As discussed previously, Apple breaks images down into 512x512 tiles, each of which has its own “infe” entry. This means that although an HEIF image originating from an Apple device may only contain a single image, it contains many “infe” entries:

Apple - “Live Photos” and Video

McKeown & Russell (2020) notes that for “Live Photos”, Apple uses “a sequence of JPEGs accompanied by a QuickTime movie file” instead of the HEIF format, but Jerian suggests that Apples implementation of an HEIF file is more like a “single frame encoded H.265 video”. To examine this further, an experiment was run using an iPhone 13 and default setting to transfer a “Live Photo” to a PC via a USB cable. I found a single JPEG file (rather than a series) accompanied by a QuickTime (AVC, H.264 encoded) movie file was created.

According to Jerian Cheeky (Jerian, 2017), and referring to Apple support (Apple Support, 2021), the device settings can be adjusted to ensure files are transferred in their original format. Running the same experiment again but with settings adjusted, I was able to transfer a “Live Photo” consisting of an HEIF (.heic) file with its accompanying QuickTime (HEVC H.265 encoded .mov) video file. A brief inspection of the HEIF file using GIMP suggests only a single image is present within the file.

To briefly verify this, I counted the number of “infe” boxes within the file. As mentioned previously, Apple breaks images down into 512x512 tiles so a 3024x4032 image should equate to roughly 6x8 (ie. 48) tiles with one “infe” box per tile. This would be consistent with the sample file in shown in Figure 9. However I counted 52 “infe” boxes within the file. As discussed previously, this would account for the additional “grid” derived image item, the thumbnail image item, the Exif metadata item, and a depth map image item (as supported by the iPhone 13). This clearly shows that a single image is contained within Apples implementation for an HEIF file, and is consistent with the example shown in Figure 9.

So, though a single HEIF file technically could support the functionality required for an a “Live” Photo, it appears Apple has chosen not to implement “Live” Photos in this way.

As discussed previously, Apple does not use the HEIF file format for video, so should an HEIF file contain a “moov” box, it is unlikely to originate from an apple device (so could be regarded as a form of fingerprint).

Label	Bytes	Description
ftype	0x18	major brand="heic", minor version=0, compatible brands= "mif1, heic"
meta	0xF74	
hdlr	0x22	always "pict" for HEIF
dinf	0x24	
pitm	0xE	Item_ID = 0x31 = Primary Item
iinf	0x43D	entry_count = 0x33 = number of items stored
infe	0x15	ItemInfoEntry, version = 2, item_id = 0x1, item_type = hvc1, item_name="", [Tile 1]
infe	0x15	ItemInfoEntry, version = 2, item_id = 0x2, item_type = hvc1, item_name="", [Tile 2]
....		
infe	0x15	ItemInfoEntry, version = 2, item_id = 0x30, item_type = hvc1, item_name="", [Tile 48]
infe	0x15	ItemInfoEntry, version = 2, item_id = 0x31, item_type = grid, item_name="", [derived image for all tiles]
infe	0x15	ItemInfoEntry, version = 2, item_id = 0x32, item_type = Exif, item_name="", [Thumbnail]
infe	0x15	ItemInfoEntry, version = 2, item_id = 0x33, item_type = Exif, item_name="", [Exif]
iref	0x94	version=0, [array of SingleItemTypeReferenceBox]
dimg	0x6C	from_item_ID = 0x31, ref_count=0x30, to_item_ID=0x1, 0x2..0x30 [derived image]
thmb	0xE	from_item_ID = 0x32, ref_count=0x1, to_item_ID=0x31 [thumbnail]
cdsc	0xE	from_item_ID = 0x33, ref_count=0x1, to_item_ID=0x31 [content description ref / exif]
iprp	0x6F3	
ipco	0x5AD	ItemPropertyContainerBox = property data
colr	0x230	colour information 1
hvcC	0x70	decoder config 1
ispe	0x14	spacial extent 1-1
ispe	0x14	spacial extent 1-2
irot	0x9	image rotation 1
pixi	0x10	pixel information 1
colr	0x230	colour information 2
hvcC	0x70	decoder config 2
ispe	0x14	spacial extent 2-1
pixi	0x10	pixel information 2
ipma	0x13E	Item Property Associatin = connects property data in ipco to item numbers
List of 0x32 items	0x3 + [size]	item number (2 bytes), size (1 byte), data (size bytes)
idat	0x10	
iloc	0x340	Version = 1, offset_size = 4, length_size = 4, base_offset_size=0, index_size=0, item_count=0x33
item_id=0x1, file offsets used, base_offset=0, extent_count=0x1, extent_offset=X1, extent_length=Y1		
item_id=0x2, file offsets used, base_offset=0, extent_count=0x1, extent_offset=X2, extent_length=Y2		
...		
item_id=0x33, file offsets used, base_offset=0, extent_count=0x1, extent_offset=X33, extent_length=Y33		
mdat	variable	contains data on EXIF / thumbnail / image data

Figure 9 – Schematic Structure of a sample HEIF file taken from an Apple iPhone 8+ adapted from (Cheeky4n6monkey, 2017)

Hidden and Auxiliary Images

In the HEIF technical specification the ItemInfoEntry (ie. "infe") box header specifies a "Hidden" flag. Referring to section 6.4.2 of the HEIF specification, this can be found in the first bit of Byte 12 of the ItemInfoEntry box header (*ISO/IEC 23008-12:2017 Information Technology — High Efficiency Coding and Media Delivery in Heterogeneous Environments*, 2017) (Refer to Appendix I – ItemInfoEntry Box Hidden Flag).

In theory, any image (or indeed any "infe") entry could be marked as hidden and assuming viewer applications respect this flag, the images would never be displayed. However, the specification states that the primary image item "pitm" should not be hidden. Auxiliary image items on the other hand, modify master image items in some way and should never be directly displayed by a viewer application.

In theory, it would be possible to include hidden image items within an HEIF file that contain steganographic or illegal image content that would never be displayed by a standard viewer.

Metadata

As with single Image File Formats that support Metadata (eg. JPEG), metadata items inside an HEIF file can be used to hide information. The HEIF standard supports the Exif, XMP and MPEG-7 metadata standards depending on their use. This should be considered when forensically analysing an HEIF file. Additionally, as with image “infe” entries, it may be possible to use the hidden flag to create metadata boxes that are not referenced by viewer applications

External References

HEIF files can contain links to external content via URLs. This complicates analysis of HEIF files from a forensics perspective since not all of the information associated with an HEIF file may be contained within it.

Forensic Hashes

Though there are restrictions on the way in which boxes are nested, HEIF is a flexible container format and as such image (or other “infe”) items could be reordered either deliberately or as an unanticipated consequence of editing. This would affect the binary content of the file and therefore any hash that would have been created for it (McKeown & Russell, 2020). Deconstruction of an HEIF file and creating hashes for individual “infe” and other box components may be an alternative to creating a single has for an HEIF file.

Additionally, the advanced inter-encoding compression method creates challenges when calculating hashes for image sequences and video. The insertion or deletion of frames would have a massive binary change between adjacent frames, so the use of perceptual hashes may be more appropriate (McKeown & Russell, 2020).

2.5.2 HEIF File Forensics – Tools and Techniques

Manual Forensic Analysis

HEIF files could be forensically analysed by manually carving them up using a Hex Editor with reference to the HEIF file specification, or it could be done by creating a software tool that uses existing APIs to identify and extract components.

2.6 Conclusion

The introduction and immediate wide-spread use of the new HEIF image file format, combined with its flexibility and complexity, mean it could (with a little effort) be used for nefarious purposes, perhaps linking to illegal content or even hiding illegal content in plain-sight. The lack of forensic analysis tooling makes the detection of illegal content extremely difficult and very time consuming for forensic analysts.

The remainder of this thesis discusses the development and evaluation of a prototype forensic analysis tool.

3 Prototype Forensic Tool Implementation

The research conducted by McKeown & Russell (2020) highlights the now widespread use of HEIF files, their complexity and flexibility, but also the lack of forensic tooling currently available. The flexibility of the file format means that with a little effort, it could be used for illegal purposes, but the lack of adequate tooling means current analysis methods are relatively complex and time consuming processes.

McKeown & Russell (2020) made several recommendations, one of which was the development of a forensic tool to assist forensic investigators in their task. The aim of this research is to add to the state-of-the-art by developing a prototype / proof-of-concept tool designed to address some of their concerns and implement some of their recommendations. Tool development was focussed on the following areas:

- Parsing of HEIF files to create a “Map” of the file structure
- Extraction of salient information from HEIF files in a human-readable form
- Extraction of All image content including the primary image, any hidden images, auxiliary images etc
- Extraction of All Exif metadata
- Deconstruction of HEIF files into their component parts including the extraction of binary content for further analysis using other tools
- Generation of Hashes for individual elements that constitute a HEIF file

3.1 Methodology

At a high level, an Agile development methodology was used for the design and development of the proof-of-concept / prototype tool. A set of outline requirements & features was established at the beginning of the development phase. Development then proceeded in an Agile fashion delivering clearly defined functionality in small increments to implement each feature. However, formal Agile processes (eg. sprint planning, sprint retrospectives etc) were not required since development was conducted by a single developer rather than a development team.

The steps below were followed to create the prototype tool:

- 1) Define a set of tool requirements and features (section 3.1.1)
- 2) Define the scope of the implementation (section 3.1.2)
- 3) Select Software Development Languages, tools (section 3.1.3)
- 4) Identify of useful software components and libraries (section 3.1.4)
- 5) Solution design and development (section 3.1.5)
- 6) Testing (section 3.1.6)

3.1.1 Analysis Tool Recommendations and Requirements

The first task in developing a new HEIF file forensic analysis tool was to identify a set of requirements and features. McKeown & Russell (2020) made several recommendations for HEIF file analysis, summarised below:

- HEIF files should be deconstructed into individual components to facilitate forensic analysis using a Hex Editor or others suitable tools.
- Deconstructed HEIF file components should be individually hashed as even minor changes that have no effect on image composition, would cause the HEIF file hash to change.
- The number and type of “infe” boxes should be examined. Hidden images are not rendered by compliant viewers, and other content can be embedded within a HEIF file and not rendered at all.
- The status of the hidden flag (bit 8 of the “infe” box) should be reported to the analyst, but cleared prior to the image extraction process to allow for ALL image objects (including Auxiliary images) to be extracted.
- The hidden flag (bit 8 of the “infe” box) should be cleared for ALL image objects (including Auxiliary images), and ALL image objects extracted.
- HEIF files are capable of referencing external content, so external references should be extracted for investigation.
- All Exif elements should be extracted for analysis. Readers typically extract Exif data associated with the primary image, or the first Exif item in the HEIF file.
- The moov box should be examined, and frames exported using a tool such as FFMpeg.

From the recommendations above, a set of requirements and features was extrapolated. Several additional requirements were identified and added to the list, then the list was prioritised using the MoSCoW method:

M = Must have, S = Should have, C = Could have, W = Won't have

Table 1 – Proof-of-concept / Prototype Requirements & Features

	Requirement / Feature	Priority
1	HEIF files should be deconstructed into their component parts. A directory structure reflecting the internal structure of the HEIF file should be created. Human readable and binary data should be extracted into the directory structure for further analysis.	Must
2	The prototype tool should be capable of processing a range of different HEIF files (eg. generated on an Apple iPhone, and Nokia sample files).	Must
3	Hashes should be created for each HEIF file component extracted along with a hash for the entire file.	Must
4	A human readable “map” file should be created showing the location of each component (ie. box or data item) within the HEIF file along with other basic details about the item such as its type, size, start and end positions, hash, and any other useful information appropriate for the component (eg. whether an image component is hidden or visible).	Must
5	All images should be extracted from the HEIF file, including auxiliary and thumbnail images. A copy of the original file should be taken, and the hidden flag cleared on all image components prior to extraction.	Must
6	Metadata should be extracted for all images components. This should include Exif metadata and any relevant flags. NB: As with image extraction, the hidden flag should be disregarded.	Must
7	References to external content (ie. URLs and URNs) should be extracted for further investigation.	Must
8	A log file should be output as part of the parsing and extraction process showing the status of each step and indicating any issues or errors encountered.	Must
9	The tool should be created with cross-platform with support in mind and should be capable of running on Windows, Linux and Mac OS.	Should
10	The tool should be capable of being run from a command line to allow for processing of batches of HEIF image files.	Should
11	The tool should be created to work with other tools, perhaps as a Plug-In to Autopsy or other Forensic Analysis tool set.	Could
12	Perceptual Hashing should be applied to single images and image collections.	Could
13	Perceptual Hashing should be applied to image sequences and video.	Won't
14	Image sequences should be reconstructed and hashed to void redundant frame attacks inserted into inter-coded sequences.	Won't

3.1.2 Scope

The primary focus of this thesis was the analysis of HEIF image files created by Apple iPhones. As such, Single Images and Image Collections were considered in-scope. At present, Apple's implementation of HEIF files does not make use of inter-encoded image sequences or video, so due to time-constraints, analysis of inter-encoded image sequences and video was considered out-of-scope for this project.

3.1.3 Software Development language and tool selection

Several factors were considered as part of Software Development language and tool selection.

Due to tight time constraints, familiarity with software development languages and availability of third-party HEIF Image extraction and Exif data extraction components / libraries for those languages were primary considerations. Implementing an HEVC decoder and Exif data decoder in addition to the other features would have been impractical in the time available. Cross-platform support was also an important consideration. C++, C# and Python are available on multiple operating systems and were all possible candidates.

Existing work into the development of code libraries related to the parsing of HEIF files was also considered. Hiki, (2017/2021) made a Github repository available with an outline implementation of an ISOBMFF parser written in Python.

Also considered was feature / requirement 11 which relates to the implementation of a plug-in for Autopsy.

Taking all these factors into consideration, Python 3.6 was selected as the most suitable language for the rapid development of the prototype HEIF analysis tool. PyCharm was selected as the Integrated Development Environment for Python due its support for active debugging.

3.1.4 Identification of useful software components and libraries

Export of image from HEIF files

To meet requirement 5 (Table 1 – Proof-of-concept / Prototype Requirements & Features), the PyHeif library (Paes & pypi.org, n.d.) was selected as the component to be used to export images from HEIF files. Brief prototyping and experimentation with the library indicated the tool could be used to export images reliably and consistently from HEIF files, but only image items with the “hidden” flag cleared (ie. unhidden image items) are exported. The library does not support the export of inter-encoded image sequences or video however.

Export of Exif metadata

To meet requirement 6 (Table 1 – Proof-of-concept / Prototype Requirements & Features), the ExifRead library (Sévi, n.d.) for Python was selected to export Exif data. Brief prototyping and experimentation indicated that the library could be used to export multiple Exif data items from a HEIF file, but to achieve this, each Exif data item had to be identified and isolated within HEIF file prior to Exif data extraction.

HEIF File parsing and desconstruction

To meet remaining must-have requirements (Table 1 – Proof-of-concept / Prototype Requirements & Features), an implementation of an ISOBMFF library (Hiki, 2017/2021) for Python was selected as a starting point.

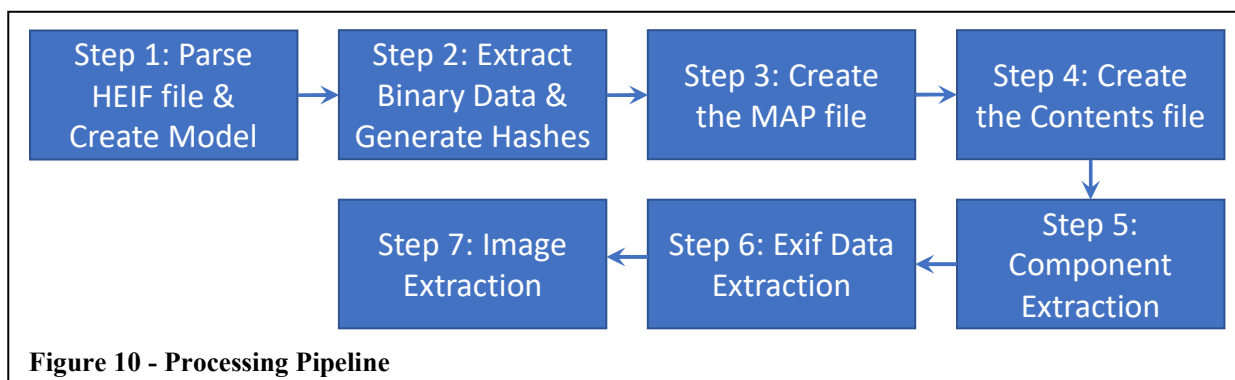
3.1.5 Solution design and development

Evaluation, testing and extension of the ISOBMFF library

The ISOBMFF library created by Hiki (Hiki, 2017/2021) provided a good foundation for the development of the HEIF file parser. However, though it worked perfectly with the sample HEIF file provided with the library, during testing and evaluation it was found to be an incomplete implementation and failed with errors when attempting to parse the Nokia HEIF sample files and HEIF files generated using an iPhone. Fortunately, Hiki designed his implementation to be extendable. The ISOBMFF library was first enhanced to create a processing log-file and handle errors appropriately. Information from the log file was used to extend the library, to implement the missing box structures (ie. grpl, sbgp, sgpd, tref, udes) required to parse each of the Nokia sample files, and HEIF files created using an Apple iPhone 13 Pro. Recording of the byte location of each Box and data item within the HEIF file was also added.

Processing Pipeline

Once the tool was capable of successfully parsing Nokia sample image files and image files captured using an Apple iPhone, and creating a log file, the processing pipeline was designed using the requirements list as a guide. Figure 10 shows the steps followed by the prototype tool to generate the output:



Appendix C – Extract from `process_heif.py` shows the high-level Python source file that executes each of the processing steps.

Step 1: HEIF File Parsing

This involves an initial pass of over the HEIF file to identify box structures (by their four character box-type identifier), building a hierarchical data-model in memory, extracting and storing salient information for each box and item such as type, size, start & end byte location, relevant flags and human readable data.

Step 2: Binary content extraction and hash value generation

This involves recursively traversing the in-memory Box structure created in step 1, and uses the byte location and box size stored in the model to locate and extract the binary data for each box, and (where appropriate) data items within a box. A hash value is generated during this step for each box and data-item, and the binary data + hash value are added to the in-memory data model.

Step 3: Map file creation

This involves recursively traversing the in-memory model, writing (as a minimum) the byte-location, type, size, end-location and hash value for each Box and data-item (one box / data-item per line). Where appropriate, additional information is also written, such as item IDs and the state of the “hidden” flag for an “infe” box.

This file is human-readable and is perhaps the most useful of the output files from an initial analysis perspective as it gives a clear overview of the structure of a HEIF file, and a quick indication of any images that may be marked as hidden in the file. Hash values can be compared with other map files to quickly identify differences between HEIF files that give an identical output or identify features that are common to multiple HEIF files. The contents of this file could be parsed, and details for items identified by their hash-value could be stored in a database as part of an investigation.

Step 4: Contents file Creation

This involves traversing the in-memory model and writing all human readable details for each box and data-item to a file. Any data recorded during the initial parse in step 1 but not written to the map file can be found in this file.

Step 5: Component extraction

This involves recursively traversing the in-memory model and creating a directory structure in the filesystem representing the hierarchical structure of the HEIF file (ie one directory per box). A text file containing human the readable box information along with the binary representation of the box and its contents are written to the directory created for the box. Additionally, where appropriate, the binary representation of any data-items that the box contains are written as individual files to the directory. The purpose of this step is to allow analysis of boxes and box contents using a Hex Editor or other tools as appropriate.

Step 6: Exif Data Extraction

This involves locating all Exif data items that were identified in step 1 and uses the ExifRead python library to extract the Exif data in a human readable form. A separate file is created for each Exif data item.

Step 7: Image Extraction

The final step is responsible for extracting all images contained in the HEIF file. Note that the “exportAllImages” subroutine for this step first creates a copy of the original file. Each “infe” box is located within the copy of the file and its “hidden” flag is cleared. The PyHeif python library is then used to extract all images within the HEIF file.

NB: In line with forensic analysis practices, a binary copy of the original file is taken prior to modification to un-hide any hidden images to preserve the integrity of the original file.

Command Argument interpreter

In addition to the processing steps discussed above, a section of code was added to interpret command line arguments so the tool could be called to batch-process a large number of files (requirement 10, Table 1 – Proof-of-concept / Prototype Requirements & Features on page 37).

HEIF Editor Tool

In addition to the prototype HEIF File analysis tool, a simple HEIF file editor (heifedit.py) was created to allow existing HEIF files to be amended to facilitate the creation of test scenarios (see sections 3.1.6 & 4 and Appendix J – HeifEditor Script Exceptrts).

Source Control - Git Repository

Source code for the prototype HEIF analysis tool and simple HEIF File editor can be found in the following Git repository (*PawQualityProducts/HeifER*, n.d.):

<https://github.com/PawQualityProducts/HeifER>

3.1.6 Testing

Testing using known good sample files

The prototype was tested with the sample with the Nokia Sample files and HEIF files created on an iPhone 13 pro. The purpose of the first battery of tests was to ensure that the tool is capable of processing a range of different HEIF files, and the expected output is created. The following criteria were tested:

- Each HEIF file is parsed and modelled without error, and the position of each component within the file is stored. This is verified by monitoring the process on-screen and checking the error log for Exceptions.
- Map file is created showing file structure and the locations, hash and other basic details of each component within the HEIF file. Since the Map file contains the start byte location and size of each box / data-item, the parsing of each HEIF file can be verified using a Hex Editor (ie. for each box / data-item shown in the Map file, navigate to the start-byte using a Hex Editor and verify it correctly maps to the box / data-item expected by looking for the 4 character ASCII “type” value).
- Details file is created showing all human readable details are extracted for each component. As with the Map file, output is verified using a Hex editor to view segments of the file for human readable content. Additionally, at a higher level, content can be verified against entries in the Map file itself.
- Log file is created showing the status of each processing stage, and logging any errors that occur during processing.
- A directory structure is created reflecting the structure of the HEIF file, and files are created in each directory containing human readable and binary data (as appropriate) corresponding to the contents of the HEIF file. Again, a Hex Editor can be used to verify the directory structure and exported binary content against the source HEIF file.

- All images are extracted from the HEIF file. Since data is located in the “mdat” box and encoded using HEVC (H.265) compression algorithm, verification of content is a little more difficult. Instead, the number of “infe” boxes should be compared with the number of images output. Known compliant HEIF viewers such as GIMP can be used to verify visible content
- All Exif data is extracted from the HEIF file. Output is verified using a Hex Editor to search test files for “Exif” box entries. ExifTool can also be used to extract primary Exif data which can be compared with the output (see Appendix G – Exif Data Extract and Appendix K – ExifTool output comparison).
- The Nokia sample files and Nokia Compliance Test file sets will be used to verify output in relation to the wider HEIF feature-set (eg. grid, overlay, crop, rotate).

Testing using amended sample files

The purpose of the second battery of tests was to demonstrate the ways in which the prototype tool could be used to detect and detail the differences between HEIF files that generate apparently identical output, and identify, locate and extract hidden and potentially illegal content.

To facilitate creation of test cases a basic editor tool was written in python (Appendix J – HeifEditor Script Exceptrs). This was essentially a simplified version of the ISOBMFF parser, but with object copy, amendment and file re-write capabilities. One complexity in modifying HEIF files is that item data location offsets (see “iloc” entries) must be modified as content is added or removed (this is done by the “rebase()” method).

Test-cases:

The following test cases were created.

A. Deconstruction of a HEIF file created on an iPhone:

The purpose of this test was to verify that the prototype tool was capable of deconstructing an HEIF file created on an iPhone, and generating the correct output (ie. map file, detailed contents file, component hashes, all images, all Exif data, a directory structure mirroring the structure of the HEIF file, and binary extracts from the file). This should meet requirements 1,2,3,4,5,6 & 8 (Table 1 – Proof-of-concept / Prototype Requirements & Features).

B. Batch processing test on a range of different HEIF files:

The purpose of this test is to check that the tool could be used to process a range of different HEIF file, and process files in a batch-processing scenario. This should meet requirement 10 (Table 1 – Proof-of-concept / Prototype Requirements & Features).

C. Control, test of the editor:

Load the sample image file into the editor and write an unmodified file back out (IMG_3802_test.HEIC): The purpose of this test was to validate the output of the HEIF Editor too to ensure it correctly parses and rewrites an unmodified HEIF file. The new file created by the tool should be binary-identical to the original HEIF file that was loaded.

D. Internal HEIF file change:

Alter the order of the first two “infe” records in the HEIF file (IMG_3802_test1.HEIC): The purpose of this test was to demonstrate that the internal order of components can be modified without corrupting or affecting the images output from a HEIF file, and that the prototype tool can be used to detect differences. Apart from the order of the first two “infe” records, the files should otherwise be identical.

E. Additional Hidden Image component:

A copy of one of the image tiles was added using the editor tool and a new file created (IMG_3802_test2.HEIC). The purpose of this test was to demonstrate that additional hidden HEVC images could be embedded within an HEIF file without affecting the output image, and to demonstrate that the tool could be used to detect differences between the new and the original file, and used to detect additional hidden content.

F. Additional Exif data:

A second Exif record attached to the new image content was added using the editor tool and a new file created (IMG_3802_test3.HEIC). The purpose of this test was to demonstrate that the prototype tool could be used to identify and extract additional Exif data not associated with the primary image object.

G. Additional unrelated content:

Added an embedded JPEG image (IMG_3802_test4.HEIC). The purpose of this test was to demonstrate that a JPEG image could be hidden within an HEIF file without affecting the output image, and demonstrate ways in which the prototype tool could be used to detect and extract the hidden image content.

H. External Content Reference:

A URL referencing external content was added using the editor tool and a new file created (IMG_3802_test5.HEIC). The purpose of this test was to demonstrate that external content can be referenced from within an HEIF file, and that the prototype tool can be used to extract references to external content for further analysis. Several complaint viewers were tested to see if URLs were retrieved.

I. Nokia Heif Conformance Test Images:

The prototype tool was run on the full batch of Nokia Conformance Test images (Technologies, 2016/2022). This was done to test resilience of the prototype tool using files that it had not been previously tested with. Some issues were expected, however the tool should be capable of handling them gracefully.

J. Auxiliary file / Steganography experiment:

The purpose of this test was to determine whether the Auxiliary image could be used in conjunction with Steganographic techniques to covertly distribute data without affecting the image rendered by compliant viewers.

4 Results

A summary of results for test cases A to H is shown in Table 2 below. Test Case I is covered separately:

Table 2 – Summary of test results (Test cases A-H)

Test Image	Test Case	Description	Purpose	Dir	Log	Map	contents	Images	Exif	Results
IMG_3802.HEIC	A	Original image from iPhone 13 Pro	Verify deconstruction of HEIF file and output from the prototype tool. Auxiliary file (depth map) extraction.	✓	✓	✓	✓	✓	✓	Pass: All hidden grid tiles extracted, primary image and Auxiliary depth map extracted, no errors.
Engineered Test Images										
IMG_3802_test.HEIC	C	Control, generated by editor. IMG_3802.HEIC was parsed and then the re-created.	Verify output from the editor tool. The file has been checked to ensure it was binary identical to IMG_3802.HEIC	✓	✓	✓	✓	✓	✓	Pass: The constructed HEIF file was identical to the source HEIF file (ie. hashes matched).
IMG_3802_test1.HEIC	D	Swapped order of info entries.	Detect and identify internal differences between two files that generate identical output but have different overall hash values.	✓	✓	✓	✓	✓	✓	Pass: Image loaded into gimp was identical to source, however overall file hashes were different. Internal box hashes helped to identify exactly where the differences were.
IMG_3802_test2.HEIC	E	Create a copy of one of the hidden image tiles and add it to the file.	Detect and identify additional content that wouldn't normally be displayed by a viewer application.	✓	✓	✓	✓	✓	✓	Pass: The additional hidden tile, not part of the composite image was identifiable in the map file and was extracted by the tool.
IMG_3802_test3.HEIC	F	Copy and subtly modify Exif data, and attach to another component within the file	Detect and extract additional Exif data not attached to the primary item	✓	✓	✓	✓	✓	✓	Pass: The additional Exif data was identified in the map file and extracted as a separate Exif extract file.
IMG_3802_test4.HEIC	G	Additional JPEG image (Anonymous.jpg) embedded into the file.	Detect hidden additional content not displayed by a viewer application	✓	✓	✓	✓	✓	✓	Pass: The additional content was identified in the map file, the binary content of the jpeg image was extracted and could be loaded into gimp.
IMG_3802_test5.HEIC	H	Added a URL entry referencing external content.	Detect and extract references to external content.	✓	✓	✓	✓	✓	✓	Pass: The URL referencing external content was identified in the map and contents files. The URL was extracted for further investigation.
Nokia Sample Images										
autumn_1440x960.heic	A & B	Nokia sample: Single Image & image derivations	Batch Processing, deconstruction and output from the prototype tool	✓	✓	✓	✓	✓	✓	Pass: Single image extracted. No errors logged
bird_burst.heic	A & B	Nokia sample: Image Bursts	Batch Processing, deconstruction and output from the prototype tool, multiple related image extraction.	✓	✓	✓	✓	✓	✓	Pass: Multiple burst-shot images extracted in addition to the primary image. No errors logged.
bothie_1440x960.heic	A & B	Nokia sample: Time Synchronised Capture	Batch Processing, deconstruction and output from the prototype tool, multiple related image extraction.	✓	✓	✓	✓	✓	✓	Pass: Additional hidden image extracted in addition to the primary image. No errors logged.
cheers_1440x960.heic	A & B	Nokia sample: Image Collection	Batch Processing, deconstruction and output from the prototype tool, multiple image extraction.	✓	✓	✓	✓	✓	✓	Pass: Additional hidden image extracted in addition to the primary image. No errors logged.
crowd_1440x960.heic	A & B	Nokia sample: Single Image	Batch Processing, deconstruction and output from the prototype tool	✓	✓	✓	✓	✓	✓	Inconclusive: Primary image (id=1002) was extracted, however although identified in the map and contents files, a second hidden image (id=1005) was not extracted by the PyHeif library but no errors were logged. Further investigation is required.
grid_960x640.heic	A & B	Nokia sample: Image Grid	Batch Processing, deconstruction and output from the prototype tool, multiple hidden image extraction	✓	✓	✓	✓	✓	✓	Pass: Grid image tiles extracted along with the composite primary image. No errors logged.
lights_1440x960.heic	A & B	Nokia sample: Single Image, User Description	Batch Processing, deconstruction and output from the prototype tool	✓	✓	✓	✓	✓	✓	Inconclusive: Primary image (id=1002) was extracted, however although identified in the map and contents files, a second hidden image (id=1005) was not extracted by the PyHeif library but no errors were logged. Further investigation is required. Unclear as to where User Description may be in the file.
old_bridge_1440x960.heic	A & B	Nokia sample: Single Image	Batch Processing, deconstruction and output from the prototype tool	✓	✓	✓	✓	✓	✓	Pass: Single image extracted. No errors logged
overlay_1000x680.heic	A & B	Nokia sample: Image Overlay	Batch Processing, deconstruction and output from the prototype tool	✓	✓	✓	✓	✓	✓	Pass: Composite (derived) overlay image (id=1012) extracted along with the 4 images that make up the derived image (ids=[1002, 1005, 1008, 1011]), no errors logged.
rally_burst.heic	A & B	Nokia sample: Image Bursts	Batch Processing, deconstruction and output from the prototype tool, multiple related image extraction.	✓	✓	✓	✓	✓	✓	Pass: 4 burst shot frames identified in the map file (ids=[1246, 1249, 1252, 1255]) were extracted. Primary image was set to 1246.
random_collection_1440x960.heic	A & B	Nokia sample: Image Collection	Batch Processing, deconstruction and output from the prototype tool, multiple image extraction.	✓	✓	✓	✓	✓	✓	Pass: 4 burst shot frames identified in the map file (ids=[1002, 1008, 1014, 1020]) were extracted. Primary image was set to 1002.
sea1_animation.heic	A & B	Nokia sample: Image Sequences	Batch Processing, deconstruction and output from the prototype tool, multiple related image extraction.	✓	✓	✓	✓	✓	✓	Pass: Moov box for the animation was identified in the map file and extracted. Primary image was extracted, no errors logged.
season_collection_1440x960.heic	A & B	Nokia sample: Image Collection	Batch Processing, deconstruction and output from the prototype tool, multiple image extraction.	✓	✓	✓	✓	✓	✓	Pass: 4 images identified in the map file (ids=[1006,1012,1018,1024]) were extracted. 4 thumbnails were also identified in the map file (ids=[1003,1009,1015,1021]), but mapped directly back to the other 4 images and were not extracted by PyHeif as distinct images. No errors logged.
ski_jump_1440x960.heic	A & B	Nokia sample: Single Image	Batch Processing, deconstruction and output from the prototype tool	✓	✓	✓	✓	✓	✓	Pass: Single image extracted. No errors logged
spring_1440x960.heic	A & B	Nokia sample: Single Image	Batch Processing, deconstruction and output from the prototype tool	✓	✓	✓	✓	✓	✓	Pass: Single image extracted. No errors logged
starfield_animation.heic	A & B	Nokia sample: Image Sequences	Batch Processing, deconstruction and output from the prototype tool, multiple related image extraction.	✓	✓	✓	✓	✓	✓	Pass: Moov box for the animation was identified in the map file and extracted. Primary image was extracted, no errors logged.
stereo_1200x800.heic	A & B	Nokia sample: Stereo Image	Batch Processing, deconstruction and output from the prototype tool, multiple related image extraction.	✓	✓	✓	✓	✓	✓	Pass: Both images left (id=1002, primary, visible) and right (1008, hidden) were extracted. No errors logged
summer_1440x960.heic	A & B	Nokia sample: Single Image	Batch Processing, deconstruction and output from the prototype tool	✓	✓	✓	✓	✓	✓	Pass: Single image extracted. No errors logged
surfer_1440x960.heic	A & B	Nokia sample: Single Image	Batch Processing, deconstruction and output from the prototype tool	✓	✓	✓	✓	✓	✓	Pass: Single image extracted. No errors logged
winter_1440x960.heic	A & B	Nokia sample: Image Collection & Image Derivations, Alpha Mask	Batch Processing, deconstruction and output from the prototype tool, multiple image extraction. Overlay identification and extraction	✓	✓	✓	✓	✓	✓	Pass: Single image extracted with the alpha mask applied. Alpha mask identified in the map file (ipma within iprp box). PyHeif did not extract the alpha mask as a separate image however. No errors logged

4.1 Test Case A – Deconstruction of a HEIF file created on an iPhone

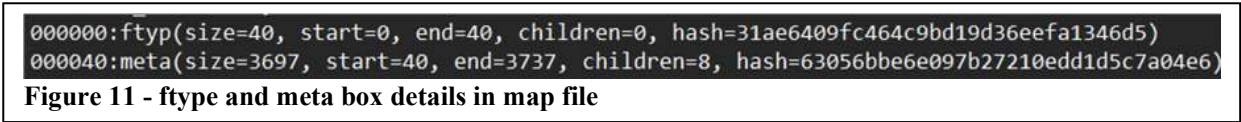
A HEIF image file “IMG_3802.HEIC” taken from an iPhone 13 pro was processed by the prototype forensic analysis tool using a command similar to the one below:

```
python3 process_heif.py IMG_3802.HEIC -outdir IMG_3802.HEIC.export
```

As expected, the directory “IMG_3802.HEIC.export” was created containing a directory structure mirroring the box structure of the input HEIF file (see Appendix G – HEIF File Directory Structure). The root of the “IMG_3802.HEIC.export” directory contained all of the images extracted (Appendix F – HEIF File Output Directory (root)) along with the map file (Appendix D – Map file extract), contents file (Appendix E – Contents file extract) and Exif metadata file (Appendix G – Exif Data Extract) for the primary image. Each sub-directory was examined to confirm that it contained the files and any sub-directories (if appropriate) as expected.

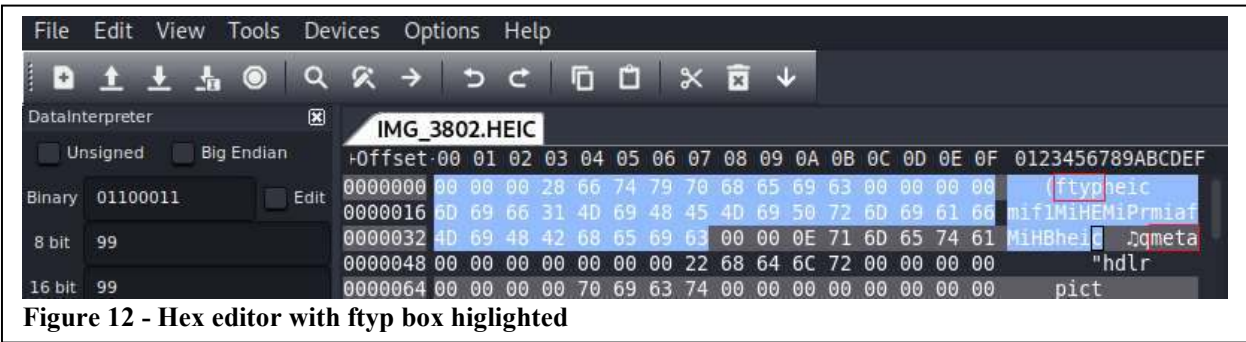
A Hex Editor along with the map and contents files output by the tool was also used to verify the output.

For example, the map file (Figure 11) shows the “ftyp” box starting at byte 0 in the file and has a length of 40 bytes, immediately followed by the “meta” box:



The Hex Editor is shown below (Figure 12) with bytes 0 to 39 highlighted:

Figure 12 shows the contents of the “ftyp” box highlighted in the Hex Editor at the beginning of the file as expected, immediately followed by the “meta” box at byte 40 as indicated by the map file.



The contents file shows basic details for the “ftyp” box (Figure 13), including the major brand (heic) and minor brands (mif1, MiHE etc...) extracted from the file. These can also be seen in the Hex Editor (Figure 12).

Similarly, the extract from the map file below (Figure 14) shows basic information for the primary (“pitm”) item, Item Information (“iinf”) box and the first of the item information entry (“infe”) boxes:

NB: Note that the first image item (denoted by the first “infe” box) is flagged as Hidden in the map file (Figure 14).

```
Box Type=ftyp
Size=40
StartByte=0
Children=0
Hash=31ae6409fc464c9bd19d36eefa1346d5
major_brand=heic
minor_version=0
Compatible Brands(6):
  compatible_brand=mif1
  compatible_brand=MiHE
  compatible_brand=MiPr
  compatible_brand=miaf
  compatible_brand=MiHB
  compatible_brand=heic
```

Figure 13 - ftyp box details from contents file

```
000122:-pitm(size=14, start=122, end=136, children=0, hash=daaac087c1539cc3d1f333b67549c085)
000136:-iinf(size=1147, start=136, end=1283, children=53, hash=8c67aac8ec693c775b3e4a045887968b)
000150:-infe(size=21, start=150, end=171, children=0, hash=62e16b72e69c095493e87e873534fb42, type=hvc1, id=1 ****HIDDEN****)
```

Figure 14 - map file showing primary item, iinf and first infe box information

Figure 15 shows a Hex Editor with the “pitm”, “iinf” and first of the “infe” boxes located at the expected start and end locations as indicated by the map file (Figure 14).

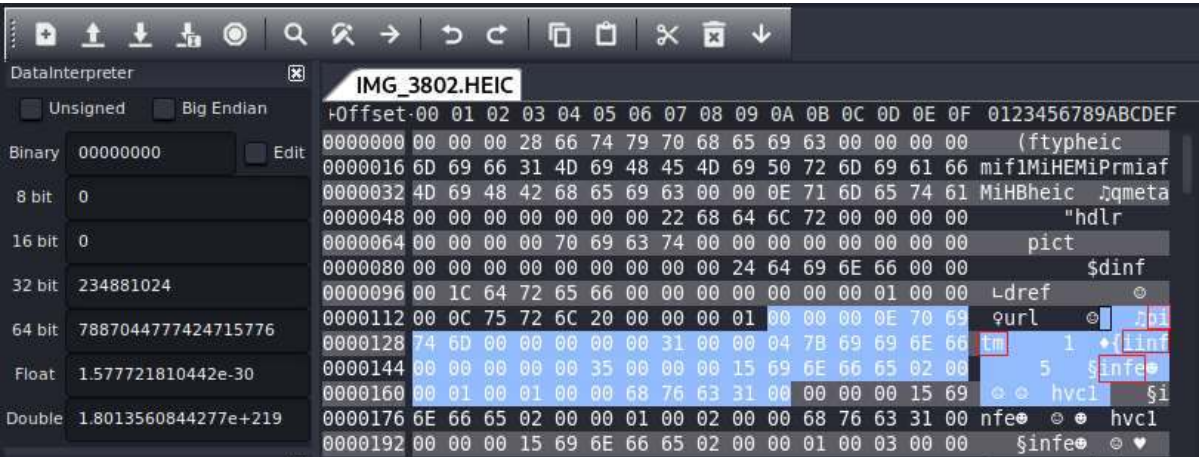


Figure 15 - Hex editor with pitm, iinf and infe boxes highlighted

Details for the “pitm” box taken from the contents file show that the primary item is item 49. This maps to “infe” entry 49 (Figure 16). Note that the infe entry for the primary item is not flagged as hidden, however, the first “infe” entry is flagged as hidden in the contents file (Figure 17). The reason for this is that “infe” entries 1 to 48 are the image “tiles” that make up the image grid and the individual tiles are hidden. However, “infe” entry 49 is the “grid” item that binds the image tiles together to produce the final image (ie. the primary item), and the primary item should never be hidden.

```

Box Type=infe
Size=21
StartByte=1158
Children=0
Hash=916e34770d6ab6dadb96fbb1ec0ccaed
Version=2
Flags=0
item_id=49
item_protection_index=0
item_name=
item_extension=None
item_type=grid
content_type=None
content_encoding=None
uri_type=None

```

Figure 16 - Contents file showing infe entry details (maps to pitm)

```

Box Type=infe
Size=21
StartByte=150
Children=0
Hash=62e16b72e69c095493e87e873534fb42
Version=2
Flags=1
*** HIDDEN ***
item_id=1
item_protection_index=0
item_name=
item_extension=None
item_type=hvc1
content_type=None
content_encoding=None
uri_type=None

```

Figure 17 - Contents file showing details for the first infe entry.

This is also reflected in the map file (Figure 18):

```

001137:--infe(size=21, start=1137, end=1158, children=0, hash=4b0ebf8538ce7e2cdb35cfffab637febe, type=hvc1, id=48 ***HIDDEN***)
001158:--infe(size=21, start=1158, end=1179, children=0, hash=916e34770d6ab6dadb96fbb1ec0ccaed, type=grid, id=49)

```

Figure 18 - map file showing infe entries 48 and 49

This state of the hidden flag can also be seen in the Hex Editor (Figure 19 & Figure 20):

```

0001136 00 00 00 00 15 69 6E 66 65 02 00 00 01 00 30 00  $infe 0 0
0001152 00 68 76 63 31 00 00 00 00 15 69 6E 66 65 02 00  hvcl $infe
0001168 00 00 00 31 00 00 67 72 69 64 00 00 00 00 15 69  1 grid $i
0001184 6E 66 65 02 00 00 00 00 32 00 00 68 76 63 31 00  nfe 2 hvcl

```

Figure 19 - Hex editor highlighting the hidden flag for infe entry 49 (not hidden)

```

0000112 00 00 00 00 00 00 00 00 01 00 00 00 02 00 00 00  hvcl 1 {iinf
0000128 74 6D 00 00 00 00 00 31 00 00 04 7B 69 69 6E 66  tm 1 {iinf
0000144 00 00 00 00 00 35 00 00 00 15 69 6E 66 65 02 00  5 $infe
0000160 00 01 00 01 00 00 68 76 63 31 00 00 00 00 15 69  hvcl $i
0000176 6E 66 65 02 00 00 01 00 02 00 00 68 76 63 31 00  nfe hvcl

```

Figure 20 - Hex editor highlighting the hidden flag for infe entry 1 (hidden)

Similar comparisons were done for each of the boxes and box items output by the tool. Additionally, hash values were compared for each item. For example, the map and contents files show the md5 hash for the binary representation of the “meta” box and its contents to be “63056bbe6e097b27210edd1d5c7a04e6”. Manually calculating the hash for the binary content extracted from the file for the meta box and its content (002_meta.bin) shows that they match (Figure 21).

The images output by the tool were also verified against the number and type of “infe” entries in the “iinf” box. Entries 1 to 48 corresponded

```
total 40
drwxr-xr-x 2 kali kali 4096 Oct 23 12:28 001_hdir
drwxr-xr-x 3 kali kali 4096 Oct 23 12:28 002_dinf
-rw-r--r-- 1 kali kali 3697 Oct 23 11:51 002_meta.bin
-rw-r--r-- 1 kali kali 110 Oct 23 11:51 002_meta.txt
drwxr-xr-x 2 kali kali 4096 Oct 23 12:28 003_pitm
drwxr-xr-x 55 kali kali 4096 Oct 23 12:28 004_iinf
drwxr-xr-x 7 kali kali 4096 Oct 23 12:28 005_iref
drwxr-xr-x 4 kali kali 4096 Oct 23 12:28 006_iprp
drwxr-xr-x 2 kali kali 4096 Oct 23 12:28 007_idat
drwxr-xr-x 55 kali kali 4096 Oct 23 12:28 008_iloc

(kali@kali)-[~/samples/processed/IMG_3802.HEIC.ex]
$ md5sum 002_meta.bin
63056bbe6e097b27210edd1d5c7a04e6 002_meta.bin
```

Figure 21 - manual hash calculation for meta.bin

to images “001_id_1_primary_False.jpg” to “001_id_48_primary_False.jpg”. These were verified to be the “tile” images that make up the primary image but were rotated 90 degrees to the left (see Figure 22). 001_id_49_primary_False.jpg was verified to be the final composite image (ie. the primary item) (see Figure 23).



Figure 22 - Tile image - infe id = 1



Figure 23 - Composite image - pitm = infe id = 49

NB: Image tiles (infe entries 1 to 48) are mapped to the primary image (infe entry 49) by the “dimg” entry in the “iref” box (Figure 24):

```
001283:--iref(size=176, start=1283, end=1459, children=5, hash=36bf6676272f559920f089c6b2cd5733)
001295:--dimg(size=108, start=1295, end=1403, hash=decf9d0730a4fe6fc7fc5c2f5c87c19a, from_id=49, to_ids=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 1
001403:--thmb(size=14, start=1403, end=1417, hash=8a467bf1894aced2e919e690063d764f, from_id=50, to_ids=[49])
001417:--auxl(size=14, start=1417, end=1431, hash=751c42311f1a8c027fd0296611472bcf, from_id=51, to_ids=[49])
001431:--cdsc(size=14, start=1431, end=1445, hash=589e15c969067399c7a478b9ee5930a3, from_id=52, to_ids=[51])
```

Figure 24 – map file extract - iref box entries

Figure 25 shows all images (including hidden image tiles, composite primary image and auxiliary image) extracted into the root of the HEIF file export directory structure, along with the map file, contents file, and Exif metadata file:

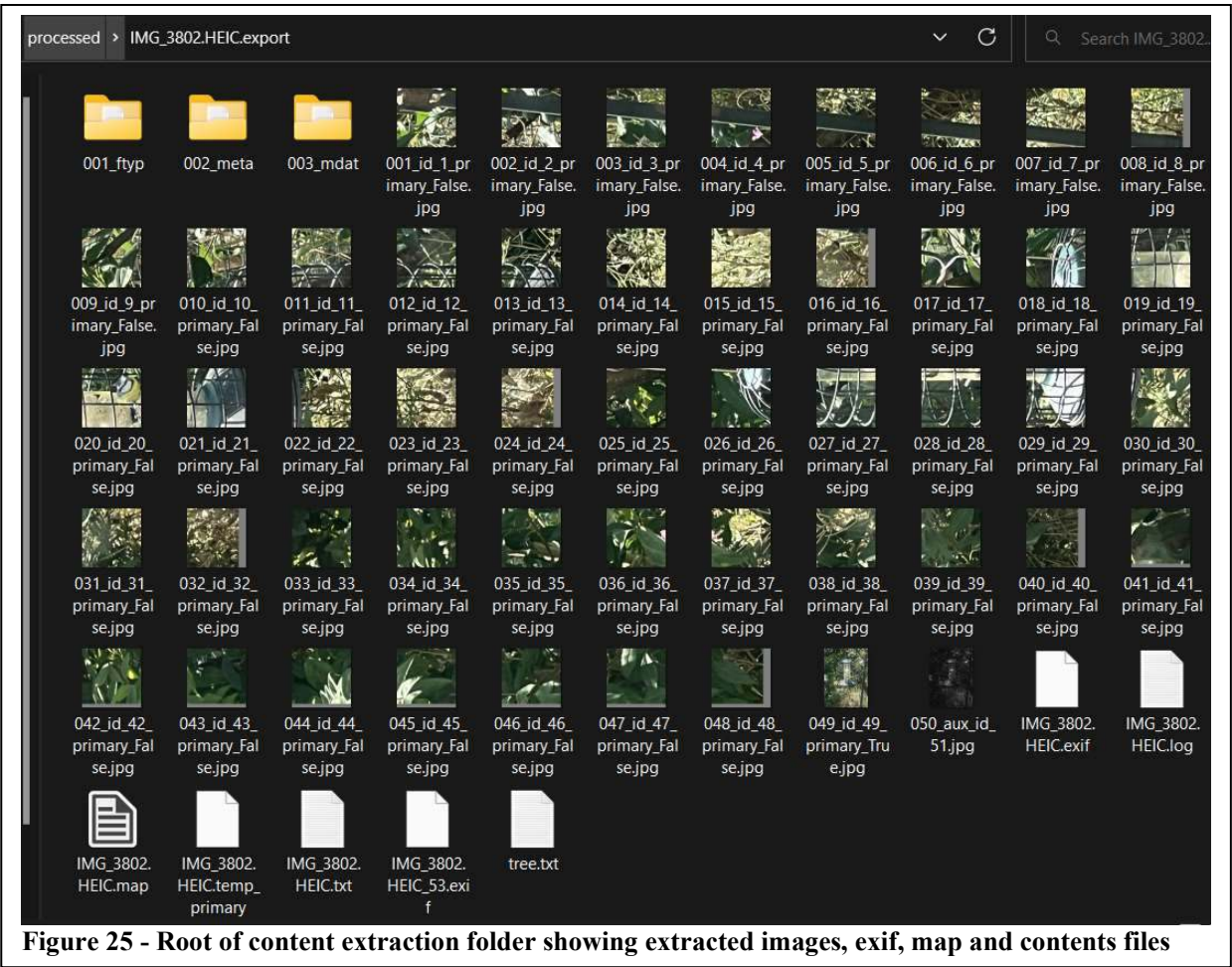
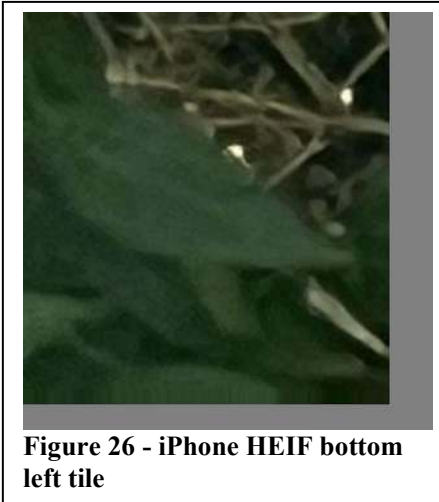


Figure 25 - Root of content extraction folder showing extracted images, exif, map and contents files

The image data itself is stored within the “mdat” section of the HEIF file and is encoded using the “HEVC” (Hs.265) compression algorithm. They are extracted using the PyHeif Python library and output to the root of the HEIF file content extraction directory structure as JPEG (“.jpg”) image files.

It’s worth noting that iPhone images help to test some of the HEIF feature set such as Grid, Crop and Rotate. For the sample image “IMG_3802.HEIC”, the composite image is made up of 48 512 x 512 pixel tiles. To construct the portrait image, the tiles are rotated 90 degrees to the right, but also, the “tiles” along the left and bottom edges are cropped (see Figure 26)



The tree -f command was used to list the directory structure created. Figure 27 shows the output (abridged, with x---x indicating multiple elements of the same type) generated using the “tree -f” Linux command. Directories are highlighted in red:

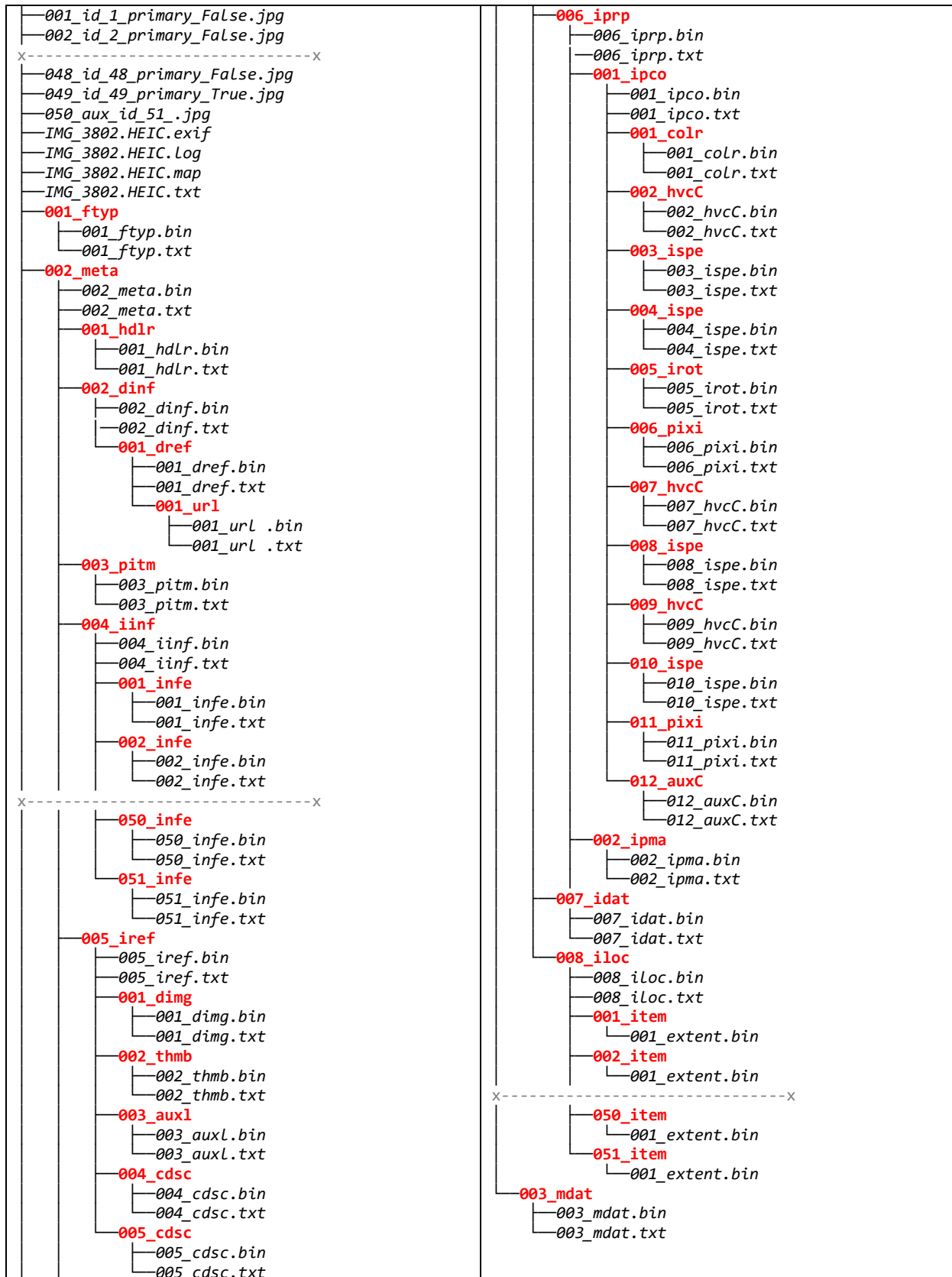


Figure 27 - Directory structure (abridged) generated using "tree -f" Linux command

The prototype forensic analysis tool also exported the Auxiliary image, which is used on an iPhone to represent a depth-map of the scene (Figure 28) This is also mapped to the primary image by the “auxl” entry in the iref box (Figure 24).



Figure 28 - Auxiliary Image – depth map

The thumbnail image item (infe 50) is mapped to the primary image via the “thmb” entry in the “iref” box (Figure 24), however this was not exported as a separate image by the PyHeif library.

4.2 Test Case B - Batch processing test on a range of different HEIF files

The bash script file shown in Appendix H – Batch Process was used to run the prototype tool on several different HEIF files. The set of sample image files from Nokia was used as source images (Nokia, 2021). Figure 29 shows how these files are displayed in Windows File Explorer:

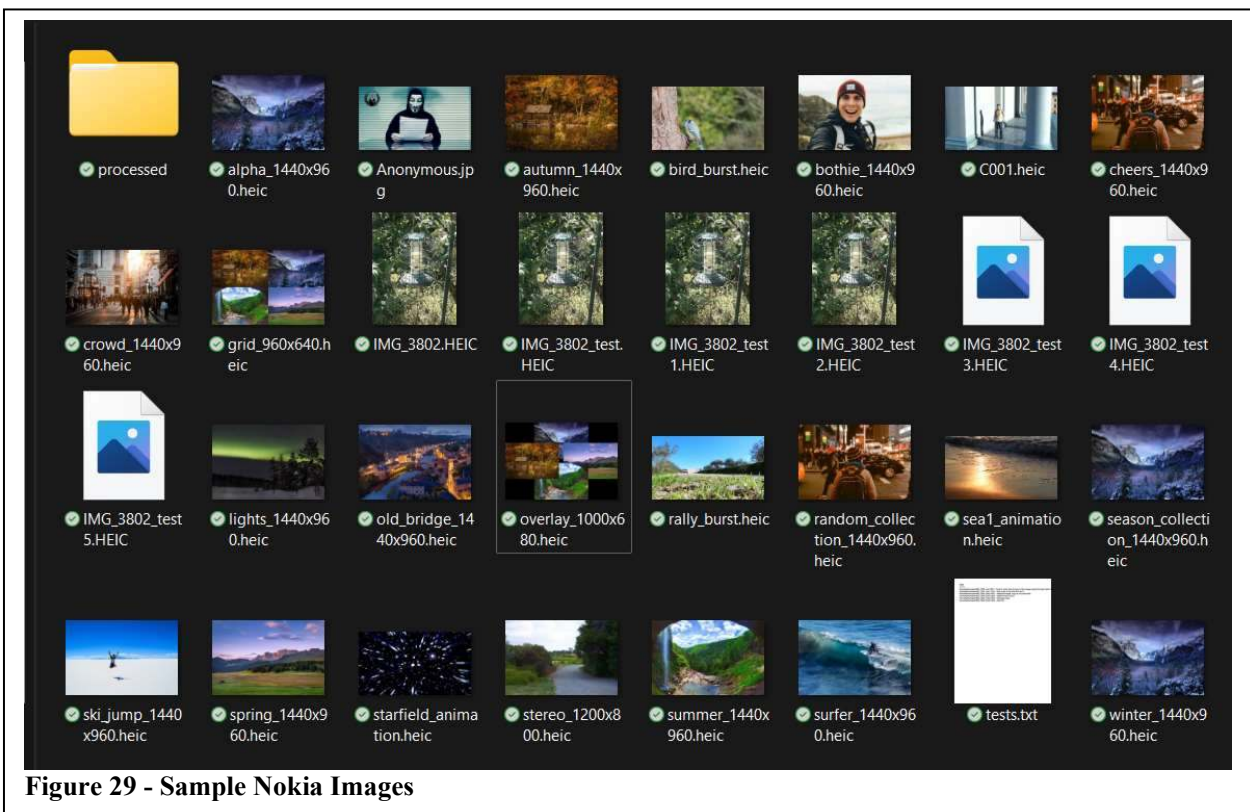


Figure 29 - Sample Nokia Images

The tool processed 29 HEIF files within twenty seconds and was executed on a VMWare virtual machine running Kali Linux with 8Mb ram. The virtual machine was running on an Acer Predator PT515-52 laptop with intel i7-10750H CPU, Nvidia RTX 2080 GPU and 64Gb system RAM. The output directories are shown in Figure 30.

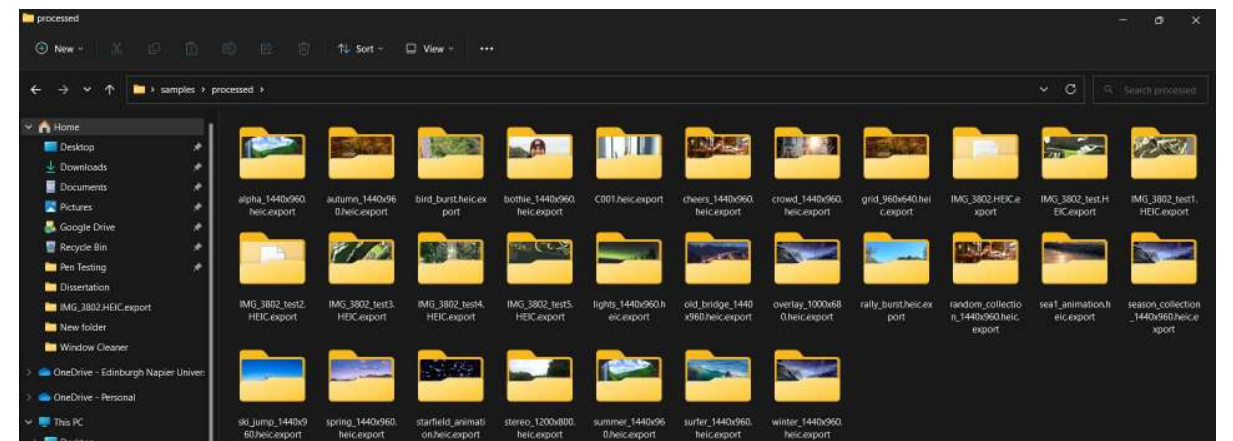


Figure 30 - Output from Batch Processed image files.

The root directory of one of the exported files (“bothie_1440x960.heic”) is shown in Figure 31.

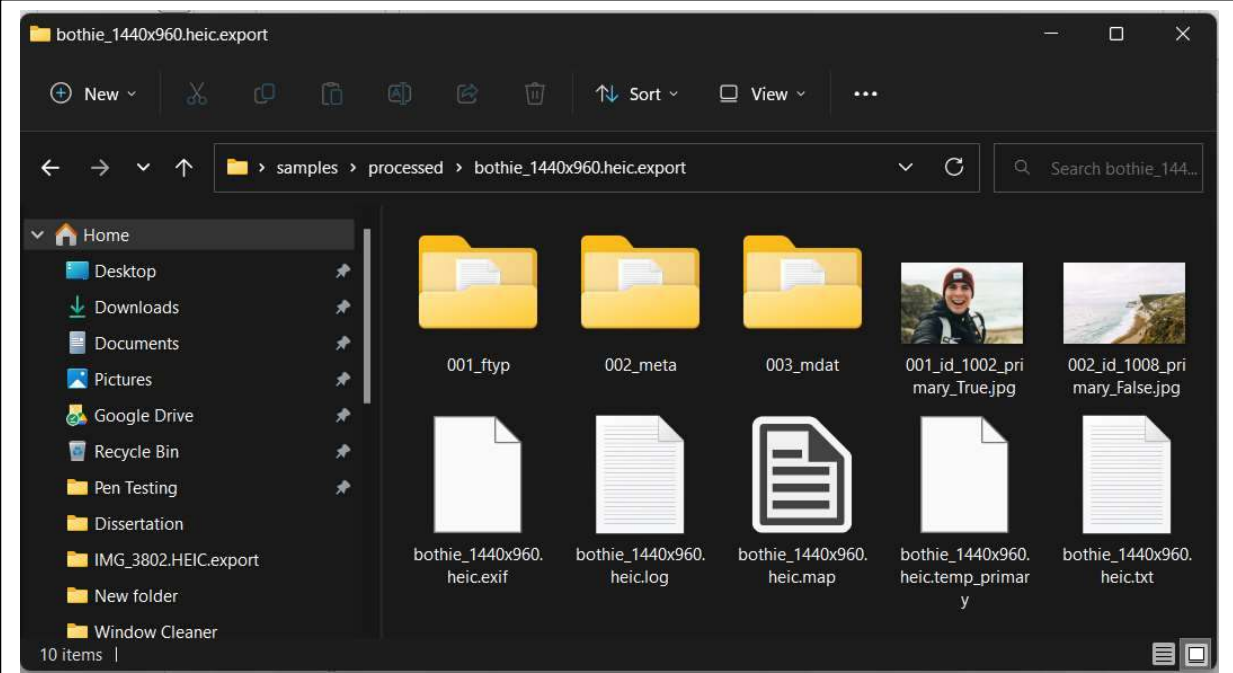


Figure 31 - Sample Nokia file "bothie_1440x960.heic"

As expected, a map file, contents file, log file, exif file, and a directory structure reflecting the structure of the HEIF file are created. The directories contain binary exports of box and item data. Figure 31 shows the HEIF file contains two different images (“001_id_1002_primary_True.jpg” & “002_id_1008_primary_False.jpg”).

Similar checks to those conducted in 4.1 (Test Case A – Deconstruction of a HEIF file created on an iPhone) were performed on a random sample to ensure the files were processed correctly.

Note: although no errors were logged, two of the tests in Test Case B are marked as “inconclusive” and require further investigation. For “crowd_1440x960.heic” though the Nokia sample is labelled as a “single image”, the map file contains an additional “infe” and “iloc” entry that could indicate the presence of a second image, however the PyHeif library does not extract a second image. For “lights_1440x960.heic” additional “infe” and “iloc” entries also suggest the potential presence of a second image which was not extracted by the PyHeif library, but Nokia’s label for the image also suggests additional user-defined data exists somewhere in the file. However, no evidence of this additional user-defined data was found in the HEIF file when examined using a Hex Editor.

4.3 Test Case C – Control test of the Editor Tool

Though not strictly in-scope as far as testing of the prototype HEIF file analysis tool is concerned, it was important to verify that the editor tool written to create test cases was capable of parsing an input file to create an in-memory model, and then writing the model out to produce an exact copy of the input file.

The editor tool was tested with a small set of input HEIF files taken from the Nokia set and a file created on an iPhone. Hashes were calculated for the source files, and for the files created by the tool and compared to ensure the tool created an identical binary copy. Once this was confirmed, the tool could be used to create samples for the remaining test cases with a reasonable level of confidence.

4.4 Test Case D - Internal HEIF file change

The purpose of this test was to see if the prototype tool could be used to detect subtle differences in files that produce identical output (but have different file hashes). The “IMG_3802.HEIC” was altered using the editor tool to swap the order of two “infe” entries (see Appendix J – HeifEditor Script Exceptrts). Figure 32 shows file and box hashes, along with the order of “infe” entries 1 and 2 in map file from the original file.

```

File=IMG_3802.HEIC, hash=af17a02ecac189e492d5b5d8263c2a18
000000:ftyp(size=40, start=0, end=40, children=0, hash=31ae6409fc464c9bd19d36eefa1346d5)
000040:meta(size=3697, start=40, end=3737, children=8, hash=63056bbe0e997b27210edd1d5c7a04e6)
000052:-hdlr(size=34, start=52, end=86, children=0, hash=40a701e65ee1aca1288a85aff8a02c6c)
000086:-dinf(size=36, start=86, end=122, children=1, hash=5ca8ac3580f6779b6362b7b1e7cfeb52)
000094:--dref(size=28, start=94, end=122, children=1, hash=ac0eab12278326c08f7cb3e861005e40)
000110:---url (size=12, start=110, end=122, children=0, hash=470b5e0ae161f2b8581d42f6056f0e59)
000122:-pitm(size=14, start=122, end=136, children=0, hash=daaac087c1539cc3d1f333b67549c085)
000136:-iinf(size=1147, start=136, end=1283, children=53, hash=8c67aac8ec693c775b3e4a045887968b)
000150:--infe(size=21, start=150, end=171, children=0, hash=62e16b72e69c095493e87e873534fb42, type=hvc1, id=1 ***HIDDEN***)
000171:--infe(size=21, start=171, end=192, children=0, hash=15b1939cdd5ed6ed5252c9d63a881aa5, type=hvc1, id=2 ***HIDDEN***)
000192:--infe(size=21, start=192, end=213, children=0, hash=30a09341a78d016235fc8c613152a134, type=hvc1, id=3 ***HIDDEN***)
000213:--infe(size=21, start=213, end=234, children=0, hash=d4ea2c3de22f14b8e28e2d9c3699225, type=hvc1, id=4 ***HIDDEN***)
000229:--extent=1 (size=8, start=229, end=237, data=(type=mdat, length=2720, start=51559, end=540
003737:mdat(size=1158355, start=3737, end=1162092, children=0, hash=d3c804e8a531f37d1840e915f9f8d3a0)

```

Figure 32 – Order of infe entries before edit (from map file)

Figure 33 shows that the order of infe entries are reversed in “IMG_3802_test1.HEIC”.

```

File=IMG_3802_test1.HEIC, hash=fbd6a2f705f4f6e2ee1b8fd77a3d56d2
000000:ftyp(size=40, start=0, end=40, children=0, hash=31ae6409fc464c9bd19d36eefa1346d5)
000040:meta(size=3697, start=40, end=3737, children=8, hash=6663d5ab7edf9006207bc3b6e1eb6c76)
000052:-hdlr(size=34, start=52, end=86, children=0, hash=40a701e65ee1aca1288a85aff8a02c6c)
000086:-dinf(size=36, start=86, end=122, children=1, hash=5ca8ac3580f6779b6362b7b1e7cfeb52)
000094:--dref(size=28, start=94, end=122, children=1, hash=ac0eab12278326c08f7cb3e861005e40)
000110:---url (size=12, start=110, end=122, children=0, hash=470b5e0ae161f2b8581d42f6056f0e59)
000122:-pitm(size=14, start=122, end=136, children=0, hash=daaac087c1539cc3d1f333b67549c085)
000136:-iinf(size=1147, start=136, end=1283, children=53, hash=7b9e605f4d12f86eb170243fb3828bc9)
000150:--infe(size=21, start=150, end=171, children=0, hash=15b1939cdd5ed6ed5252c9d63a881aa5, type=hvc1, id=2 ***HIDDEN***)
000171:--infe(size=21, start=171, end=192, children=0, hash=62e16b72e69c095493e87e873534fb42, type=hvc1, id=1 ***HIDDEN***)
000192:--infe(size=21, start=192, end=213, children=0, hash=30a09341a78d016235fc8c613152a134, type=hvc1, id=3 ***HIDDEN***)
000213:--infe(size=21, start=213, end=234, children=0, hash=d4ea2c3de22f14b8e28e2d9c3699225, type=hvc1, id=4 ***HIDDEN***)
000229:--extent=1 (size=8, start=229, end=237, data=(type=mdat, length=2720, start=51559, end=540
003737:mdat(size=1158355, start=3737, end=1162092, children=0, hash=d3c804e8a531f37d1840e915f9f8d3a0)

```

Figure 33 - Order of Infe entries after edit (from map file)

Both image files were loaded into GIMP (see Figure 34) and appeared visually identical. However, when comparing the highlighted areas Figure 32 and Figure 33 the overall file hashes (first line in each example) are different. From a forensic investigation perspective, it is important to be able to identify situations like this since illegal images are usually identified by their file hash. As shown, we can use differences in box and item hashes to zero-in on the sections of the file that are different. However, we can determine that the image content is identical between the two files because the hashes of the binary data in the “mdat” boxes are the same in both files.

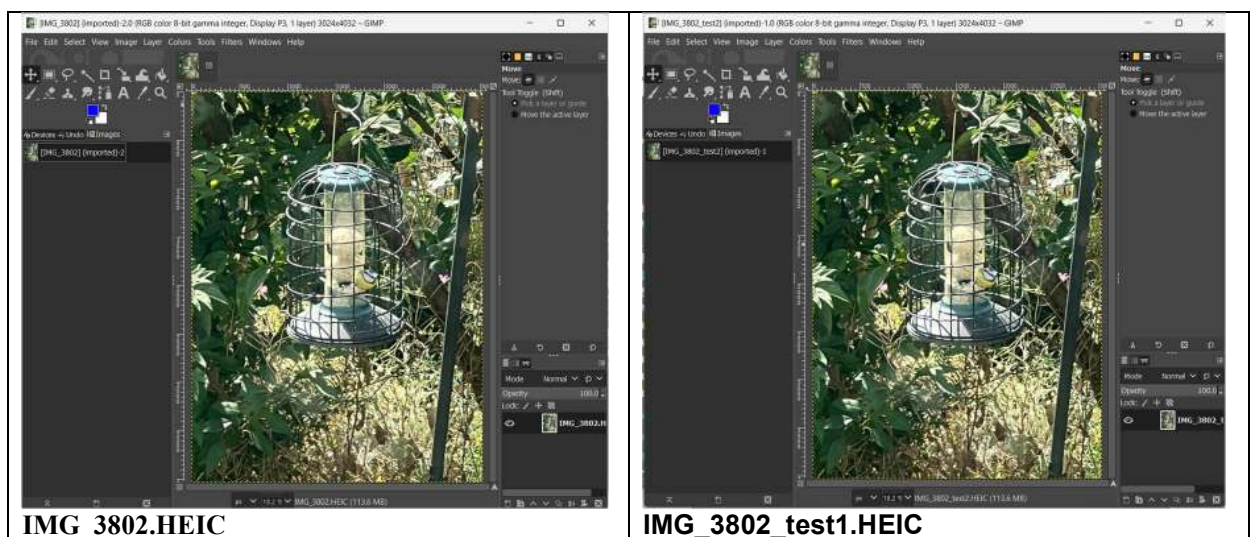


Figure 34 - "IMG_3802.HEIC" and "IMG_3802_test1.HEIC" loaded into GIMP appear identical

4.5 Test Case E - Additional Hidden Image content

The purpose of this test was to determine whether the prototype tool could be used to detect hidden image content within a HEIF file. The editor tool was used to create a copy of one of the existing image tiles and insert it into the file to create “IMG_3802_test2.HEIC” (see Appendix J – HeifEditor Script Exceptrts). Figure 35 shows sections of the map file for “IMG_3802_test1.HEIC” which does not have the additional hidden image tile.

```
000136:--iinf(size=1147, start=136, end=1283, children=53, hash=7b9e605f4d12f86eb170243fb3828bc9)
000150:--infe(size=21, start=150, end=171, children=0, hash=15b1939cdd5ed6ed5252c9d63a881aa5, type=hvc1, id=2 ***HIDDEN***)
000171:--infe(size=21, start=171, end=192, children=0, hash=62e16b72e69c095493e87e873534fb42, type=hvc1, id=1 ***HIDDEN***)
000192:--infe(size=21, start=192, end=213, children=0, hash=30a00241a78d016235f5c6c613152a134, type=hvc1, id=3 ***HIDDEN***)

001221:--infe(size=41, start=1221, end=1262, children=0, hash=3a334fec9b6e504cba4e955fb9169aba, type=mime, id=52 ***HIDDEN***)
001262:--infe(size=21, start=1262, end=1283, children=0, hash=1bbd875ef5e821c1a8bfe5b22f5bcc54, type=Exif, id=53 ***HIDDEN***)
001283:--iref(size=176, start=1283, end=1459, children=5, hash=36bf6676272f559920f089c6b2cd5733)
001295:--dimg(size=108, start=1295, end=1403, hash=decf9d0730a4fe6fc7fc5c2f5c87c19a, from_id=49, to_ids=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48])
001403:--thmb(size=14, start=1403, end=1417, hash=8a467bf1894aced2e919e690063d764f, from_id=50, to_ids=[49])
001417:--auxl(size=14, start=1417, end=1431, hash=751c42311f1a8c027fd0296611472bcf, from_id=51, to_ids=[49])
```

Figure 35 - “IMG_3802_test1.HEIC” without additional hidden image tile

Figure 36 shows the same sections but for “IMG_3802_test2.HEIC” which has an additional hidden tile.

```
000136:--iinf(size=1168, start=136, end=1304, children=54, hash=11d5e7c158d12e24a67ae508e1753ec9)
000150:--infe(size=21, start=150, end=171, children=0, hash=15b1939cdd5ed6ed5252c9d63a881aa5, type=hvc1, id=2 ***HIDDEN***)
000171:--infe(size=21, start=171, end=192, children=0, hash=62e16b72e69c095493e87e873534fb42, type=hvc1, id=1 ***HIDDEN***)

001262:--infe(size=21, start=1262, end=1283, children=0, hash=1bbd875ef5e821c1a8bfe5b22f5bcc54, type=Exif, id=53 ***HIDDEN***)
001283:--infe(size=21, start=1283, end=1304, children=0, hash=99fcda865a8871d7fd64664821fba0ec, type=hvc1, id=54 ***HIDDEN***)
001304:--iref(size=176, start=1304, end=1480, children=5, hash=36bf6676272f559920f089c6b2cd5733)
001316:--dimg(size=108, start=1316, end=1424, hash=decf9d0730a4fe6fc7fc5c2f5c87c19a, from_id=49, to_ids=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48])
001424:--thmb(size=14, start=1424, end=1438, hash=8a467bf1894aced2e919e690063d764f, from_id=50, to_ids=[49])
001438:--auxl(size=14, start=1438, end=1452, hash=751c42311f1a8c027fd0296611472bcf, from_id=51, to_ids=[49])
```

Figure 36 - "IMG_3802_test2.HEIC" with additional hidden image tile

Comparing the “iinf” boxes, note that they have different hashes, and the “iinf” box in the second file contains an additional child item (item id=54) which is a hidden “hvc1” image object. However, we can see that the tile is not part of the composite grid image because it is not included in the image id list of the “dimg” box (see Figure 36).

However, the additional image tile was exported as image id=54 (See Figure 37)

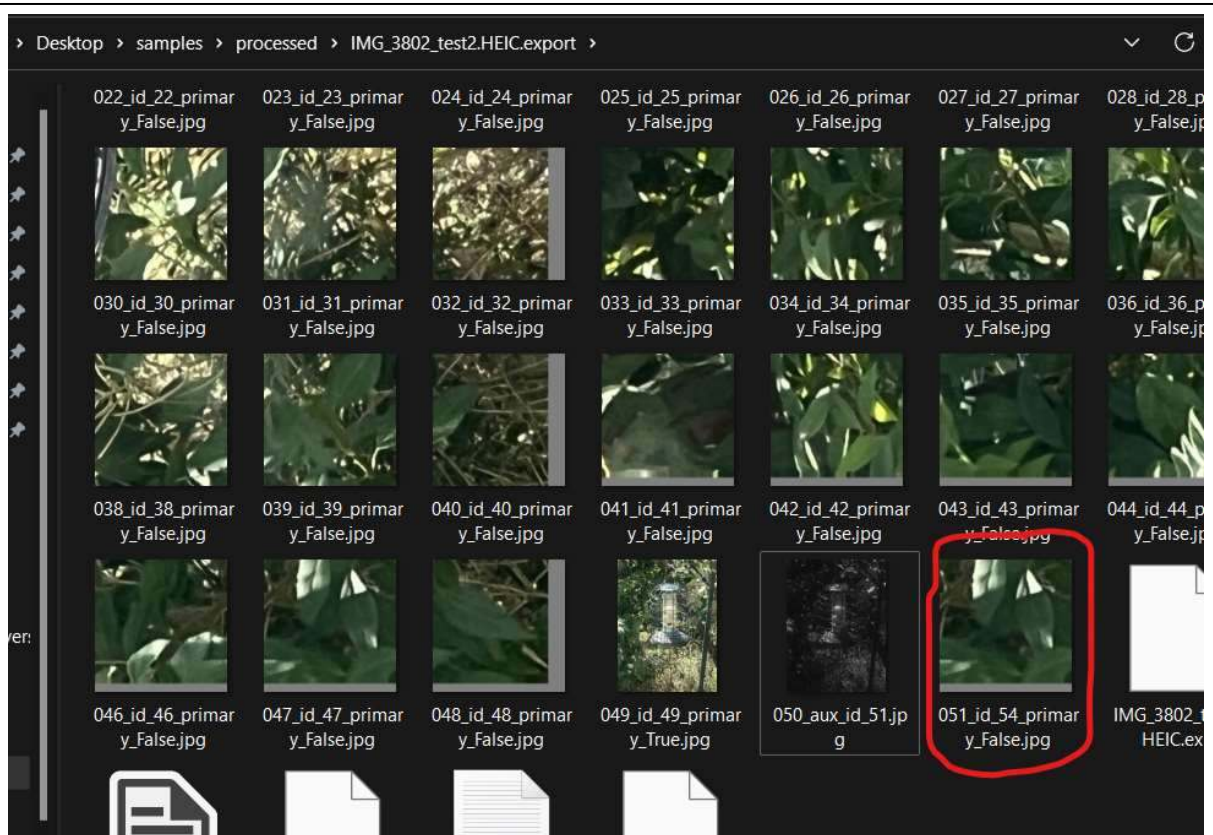


Figure 37 - Additional Tile Exported

When opening the file with GIMP, the file opened as if it contained a single image. In contrast, opening a HEIF image that contains multiple unhidden images, a second dialog is displayed allowing the user to select an image to edit as shown in Figure 38:

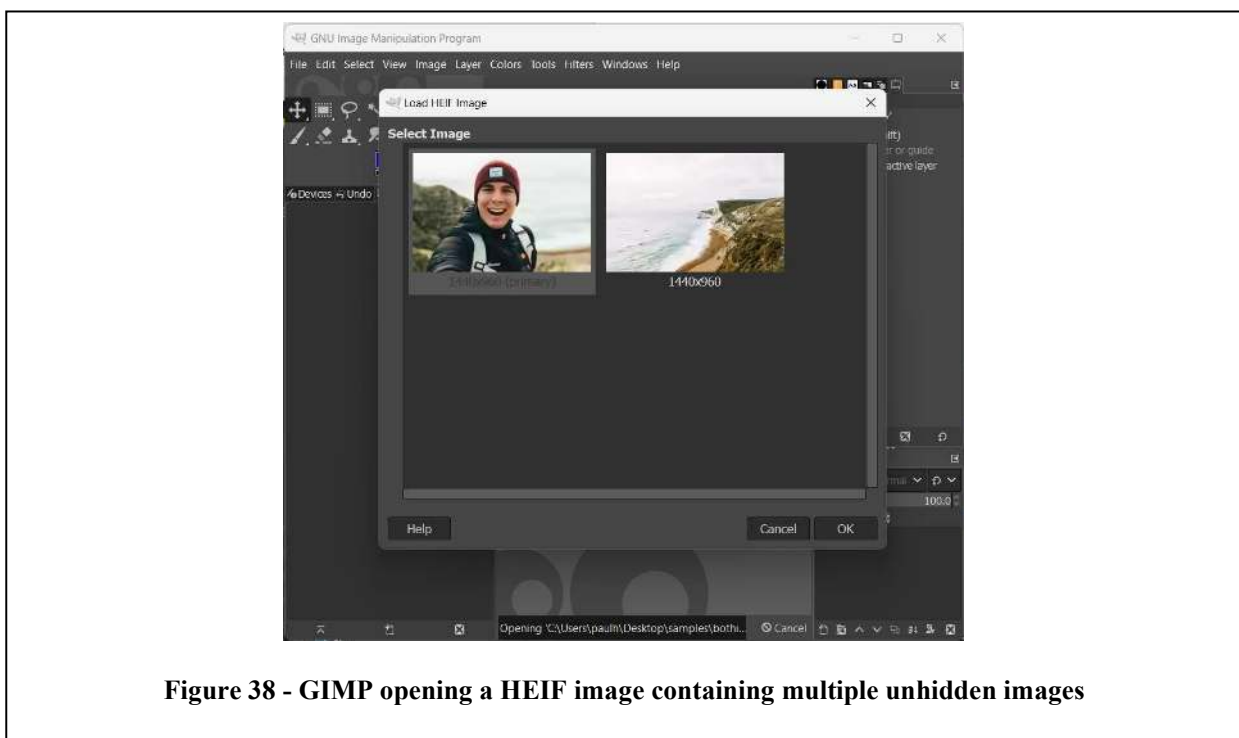


Figure 38 - GIMP opening a HEIF image containing multiple unhidden images

4.6 Test Case F - Additional Exif data

The purpose of this test was to determine whether the prototype tool could be used to detect and extract additional Exif data not associated with the primary image item.

The editor tool was used to copy and subtly alter the existing Exif data item and add it to the HEIF file to create “IMG_3802_test3.HEIC” (see Appendix J – HeifEditor Script Exceptrts). Figure 39 shows sections of the map file for

“IMG_3802_test2.HEIC” which does not have the additional Exif data item.

```
000122:-pitm(size=14, start=122, end=136, children=0, hash=daaac087c1539cc3d1f333067549c085)
000136:-iinf(size=1168, start=136, end=1304, children=54, hash=11d5e7c158d12e24a67ae508e1753ec9)
000150:-infe(size=21, start=150, end=171, children=0, hash=15b1939cdd5ed6ed5252c9d63a881aa5, type=hvc1, id=2 ***HIDDEN***)
000171:-infe(size=21, start=171, end=192, children=0, hash=62e16b72e69c095493e87e873534fb42, type=hvc1, id=1 ***HIDDEN***)
001262:-infe(size=21, start=1262, end=1283, children=0, hash=1bbd875ef5e821c1a8bfe5b22f5bcc54, type=Exif, id=53 ***HIDDEN***)
001283:-infe(size=21, start=1283, end=1304, children=0, hash=99fcda865a8871d7fd64664821fba0ec, type=hvc1, id=54 ***HIDDEN***)
001304:-iref(size=176, start=1304, end=1480, children=5, hash=36bf6676272f559920f089c6b2cd5733)
003764:- Item=54, (size=16, start=3764, end=3780, extents=1, id=54)
003772:- Extent=1 (size=8, start=3772, end=3780, data=(type=mdat, length=13711, start=1081806, end=1095517, hash=
64ccc9ed025316bde2219a9393429005)
003780:mdat(size=1158355, start=3780, end=1162135, children=0, hash=d3c804e8a531f37d1840e915f9f8d3a0)
```

Figure 39 - Sections of map file for "IMG_3802_test2.HEIC" without additional Exif data

Figure 40 shows the same sections of the map file for “IMG_3802_test3.HEIC”. The “iinf” box has a different hash and contains an additional Exif item (id=55). Figure 40 also shows an additional cdsc item which associates the new Exif item with the new image tile (id=54) created in Test Case E - Additional Hidden Image content. The hash values for the “mdat” boxes were also different because the additional Exif data is stored in the “mdat” box.

```
000122:-pitm(size=14, start=122, end=136, children=0, hash=daaac087c1539cc3d1f333067549c085)
000136:-iinf(size=1189, start=136, end=1325, children=55, hash=5734f90fc97c0b72b3239558e5d6539b)
000150:-infe(size=21, start=150, end=171, children=0, hash=15b1939cdd5ed6ed5252c9d63a881aa5, type=hvc1, id=2 ***HIDDEN***)
000171:-infe(size=21, start=171, end=192, children=0, hash=62e16b72e69c095493e87e873534fb42, type=hvc1, id=1 ***HIDDEN***)
001283:-infe(size=21, start=1283, end=1304, children=0, hash=99fcda865a8871d7fd64664821fba0ec, type=hvc1, id=54 ***HIDDEN***)
001304:-infe(size=21, start=1304, end=1325, children=0, hash=7b213c25b70aac418e3afbf11107de38, type=Exif, id=55 ***HIDDEN***)
001325:-infe(size=21, start=1325, end=1346, children=0, hash=1f6a3141416f329f5f4d7a9369ab1656)
001487:-cdsc(size=14, start=1487, end=1501, hash=bf0cab9149f4a7cbaa809b5a35b41362, from_id=53, to_ids=[49])
001501:-cdsc(size=14, start=1501, end=1515, hash=691def2de2cfa1ca57e91e99b683d52d, from_id=55, to_ids=[54])
001515:-infn(size=1404, start=1515, end=2919, children=2, hash=2218c790ha9c1d4a9ccf787c8a9533161)
003815:- Item=55, (size=16, start=3815, end=3831, extents=1, id=55)
003823:- Extent=1 (size=8, start=3823, end=3831, data=(type=mdat, length=2720, start=1162186, end=1164906, hash=
8c718d8050e7a0112ff9e2c86057e2ae)
003831:mdat(size=1161075, start=3831, end=1164906, children=0, hash=b11ed01813a5e9f8c5eff1875ab8afc4)
```

Figure 40 - sections of the map file for "IMG_3802_test3.HEIC", with additional Exif data

Figure 41 shows that an additional Exif file (id=55) was exported.

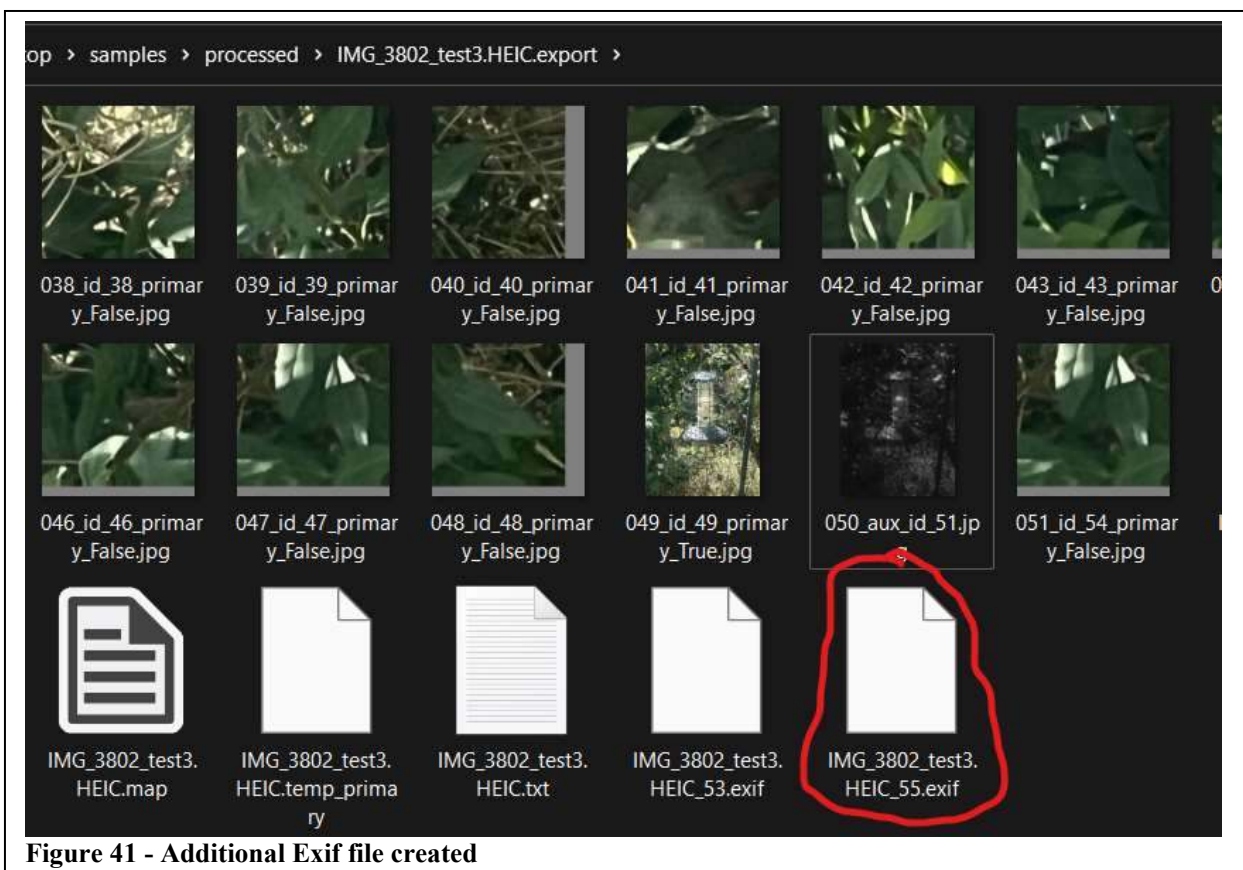


Figure 41 - Additional Exif file created

Figure 42 shows segments of the two exif files exported from “IMG_3802_test3.HEIC”.

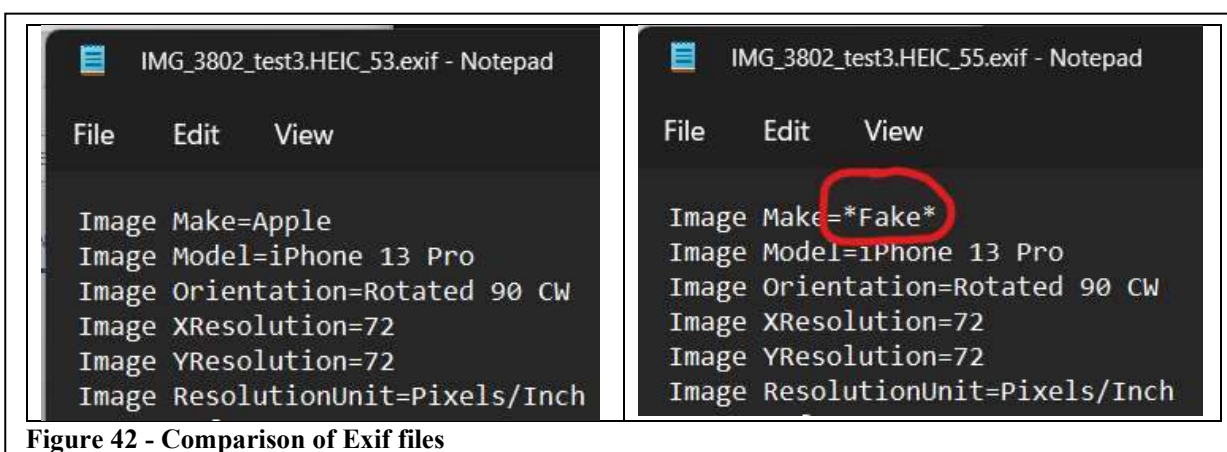


Figure 42 - Comparison of Exif files

ExifTool.exe (*ExifTool by Phil Harvey*, n.d.) was used to determine whether additional Exif data could be detected or extracted using a well-established digital forensics tool. Exif data was extracted from “IMG_3802.HEIC” (no additional Exif data) and “IMG_3802_test3.HEIC” (with additional Exif data). The output from both was almost identical. No additional Exif data was exported from the second file, however, there was an additional entry on line 133 for file “IMG_3802_test3.HEIC”. This was “Warning: Invalid atom size” though no clear definition could be found for this error message. Figure 43 shows extracts comparing the ExifTool outputs for both files. Appendix K – ExifTool output comparison shows complete output for both files.

ExifTool Version Number	: 12.50	ExifTool Version Number	: 12.50
File Name	: IMG_3802.HEIC	File Name	: IMG_3802_test3.HEIC
Directory	: .	Directory	: .
File Size	: 1162 kB	File Size	: 1165 kB
File Modification Date/Time	: 2022:08:06 22:36:09+01:00	File Modification Date/Time	: 2022:10:23 16:47:04+01:00
File Access Date/Time	: 2022:11:18 18:10:48+00:00	File Access Date/Time	: 2022:11:18 18:12:17+00:00
File Creation Date/Time	: 2022:08:06 15:34:22+01:00	File Creation Date/Time	: 2022:10:23 17:12:10+01:00
File Permissions	: -rw-rw-rw-	File Permissions	: -rw-rw-rw-
File Type	: HEIC	File Type	: HEIC
File Type Extension	: heic	File Type Extension	: heic
MIME Type	: image/heic	MIME Type	: image/heic
Major Brand	: High Efficiency Image Format	Major Brand	: High Efficiency Image Format
HEVC still image (.HEIC)		HEVC still image (.HEIC)	
Minor Version	: 0.0.0	Minor Version	: 0.0.0
Compatible Brands	: mif1, MiHE, MiPr, mif, MiHB, heic	Compatible Brands	: mif1, MiHE, MiPr, mif, MiHB, heic
Handler Type	: Picture	Handler Type	: Picture
Primary Item Reference	: 49	Primary Item Reference	: 49
X-----X		X-----X	
X-----X		Warning : Invalid atom size	
X-----X		X-----X	
Hyperfocal Distance	: 43.43 m	Hyperfocal Distance	: 43.43 m
Light Value	: 11.6	Light Value	: 11.6
Lens ID	: iPhone 13 Pro back triple	Lens ID	: iPhone 13 Pro back triple
camera 9mm f/2.8		camera 9mm f/2.8	

Figure 43 - ExifTool output comparison

4.7 Test Case G - Additional unrelated content

The purpose of this test was to determine whether the prototype tool could be used to detect and extract additional hidden but unrelated content. The editor tool was used to embed a jpeg image within the HEIF file (see Appendix J – HeifEditor Script Excerpt) to create “IMG_3802_test4.HEIC”. Figure 44 shows sections of the map file for “IMG_3802_test3.HEIC” which does not have the additional jpeg image embedded.

```
000122:-pitm(size=14, start=122, end=136, children=0, hash=daaac087c1539cc3d1f33306/549c085)
000136:-iinf(size=1189, start=136, end=1325, children=55, hash=5734f90fc97c0b72b3239558e5d6539b)
000150:-infe(size=21, start=150, end=171, children=0, hash=15b1939cdd5ed6ed5252c9d63a881aa5, type=hvc1, id=2 ***HIDDEN***)
000171:-infe(size=21, start=171, end=192, children=0, hash=62e16b72e69c095493e87e873534fb42, type=hvc1, id=1 ***HIDDEN***)
001304:-infe(size=21, start=1304, end=1325, children=0, hash=7b213c25b70aac418e3afbf11107de38, type=Exif, id=55 ***HIDDEN***)
001325:-iref(size=190, start=1325, end=1515, children=6, hash=1fc3141446f53acffdd72c83603b4fff)
001337:-dimg(size=108, start=1337, end=1445, hash=decf9d0730a4fe6fc7fc5c2f5c87c19a, from_id=49, to_ids=[1, 2, 3, 4, 5, 6, 7,
003815:- Item=55, (size=16, start=3815, end=3831, extents=1, id=55)
003823:- Extent=1 (size=8, start=3823, end=3831, data=(type=mdat, length=2720, start=1162186, end=1164906, hash=
8c718d8050e7a0112ff9e2c86057e2ae)
003831:mdat(size=1161075, start=3831, end=1164906, children=0, hash=b11ed01813a5e9f8c5eff1875ab8afc4)
```

Figure 44 – Sections of map file for “IMG_3802_test3.HEIC” before jpeg is added

Figure 45 shows the same sections of the map file for “IMG_3802_test4.HEIC” after the jpeg image was embedded. Note the additional “infe” entry and differences in the “iinf” and “mdat” box hashes. The binary data for the jpeg image file was added to the “mdat” box.

```
000136:-iinf(size=1210, start=136, end=1346, children=56, hash=50bda0ad283d223011fe54075e3a134f)
000150:-infe(size=21, start=150, end=171, children=0, hash=15b1939cdd5ed6ed5252c9d63a881aa5, type=hvc1, id=2 ***HIDDEN***)
001304:-infe(size=21, start=1304, end=1325, children=0, hash=7b213c25b70aac418e3afbf11107de38, type=Exif, id=55 ***HIDDEN***)
001325:-infe(size=21, start=1325, end=1346, children=0, hash=b9aaeb38bc78f01e80e2eebca0fb57db, type=jpeg, id=56 ***HIDDEN***)
001346:-iref(size=190, start=1346, end=1536, children=6, hash=1fc3141446f53acffdd72c83603b4fff)
001358:-dimg(size=108, start=1358, end=1466, hash=decf9d0730a4fe6fc7fc5c2f5c87c19a, from_id=49, to_ids=[1, 2, 3, 4, 5, 6, 7,
003858:- Item=56, (size=16, start=3858, end=3874, extents=1, id=56)
003866:- Extent=1 (size=8, start=3866, end=3874, data=(type=mdat, length=132870, start=1164949, end=1297819,
hash=b6f9eba19649e57e22863018813701b3)
003874:mdat(size=1293945, start=3874, end=1297819, children=0, hash=3ac7a79cfcaa1fca550354d078930b30e)
```

Figure 45 – Sections of map file for “IMG_3802_test4.HEIC” after the jpeg image was embedded

The jpeg image binary data was automatically extracted by the tool into to directory “IMG_3802_test4.HEIC.export\002_meta\008_iloc\056_item\ 001_extent.bin”. The file was displayed correctly in Linux but had to be renamed with the “.jpg” extension (ie. “001_extent.jpg”) to allow Windows to interpret it correctly. (see Figure 46 - Extracted JPEG image data).



Note: One improvement would be to automatically detect the binary content type either by using the type field of the “infe” box or by examining the binary content looking for “magic numbers” that identify the file-type. The file could then be created with an appropriate extension (in this case “.jpg”) to ensure it is viewed correctly across different operating systems.

Figure 47 shows the GIMP image editing application with “IMG_3802_test4.HEIC” loaded, showing that the embedded JPEG content is not extracted and would be invisible to a forensic investigator using a compliant viewer to inspect the contents of a HEIF file. The file was opened in GIMP first with the Hidden flag set, then the test was repeated with the Hidden flag cleared. In both cases, GIMP did not display or detect the additional JPEG content.



4.8 Test Case H - External Content Reference

The purpose of this test was to determine whether the prototype tool could be used identify and extract references to external content. The editor tool was used to embed a URL into to the image (see Appendix J – HeifEditor Script Exceptrts) creating test file "IMG_3802_test5.HEIC". Figure 48 shows sections of the map file created for test file "IMG_3802_test4.HEIC" which does not have a URL embedded within it.

```
File=IMG_3802_test4.HEIC, hash=014587b4f4f247894c4551a54c09767f
000000:ftyp(size=40, start=0, end=40, children=0, hash=31ae6409fc464c9bd19d36eefa1346d5)
000040:meta(size=3834, start=40, end=3874, children=8, hash=646c681be77a64989326bd53826401cb)
000052:-hdlr(size=34, start=52, end=86, children=0, hash=40a701e65ee1aca1288a85aff8a02c6c)
000086:-dinf(size=36, start=86, end=122, children=1, hash=5ca8ac3580f6779b6362b7b1e7cfeb52)
000094:--dref(size=28, start=94, end=122, children=1, hash=ac0eab12278326c08f7cb3e861005e40)
000110:--url (size=12, start=110, end=122, children=0, hash=470b5e0ae161f2b8581d42f6056f0e59)
000122:-pitm(size=14, start=122, end=136, children=0, hash=daaac087c1539cc3d1f333b67549c085)
```

Figure 48 - map file extract for "IMG_3802_test4.HEIC" before a URL was embedded.

Figure 49 shows the same sections of the map file but for test file "IMG_3802_test5.HEIC" which has a URL embedded within it.

```
File=IMG_3802_test5.HEIC, hash=24de32157f6d97c22eb284a9f806152a
000000:ftyp(size=40, start=0, end=40, children=0, hash=31ae6409fc464c9bd19d36eefa1346d5)
000040:meta(size=3872, start=40, end=3912, children=8, hash=6edab4996db46665783c8c22d6e8b233)
000052:-hdlr(size=34, start=52, end=86, children=0, hash=40a701e65ee1aca1288a85aff8a02c6c)
000086:-dinf(size=74, start=86, end=160, children=1, hash=f0691d28f69ac478177f16e07f2ce946)
000094:--dref(size=66, start=94, end=160, children=1, hash=c805cb101c0b47839689958ba07cd9d1)
000110:--url (size=50, start=110, end=160, children=0, hash=4e093760e32e98da2f9cbe8394ac99dc)
000160:-pitm(size=14, start=160, end=174, children=0, hash=daaac087c1539cc3d1f333b67549c085)
```

Figure 49 - map file extract for "" after URL was embedded.

Note that the overall file hashes, and hashes for the "meta", and "dref" boxes and "url" entry are all different as we'd expect, but it's the size property for the "url" entry which is larger in "IMG_3802_test5.HEIC" that suggests there could be something there. However, the map file does not show details of the "url" entry content, for this we must refer to the Contents files. Figure 50 shows a comparison of the "dref" sections of both contents files and shows the URL that was embedded into "IMG_3802_test5.HEIC" (ie. <http://www.pawQualityproducts.co.uk>):

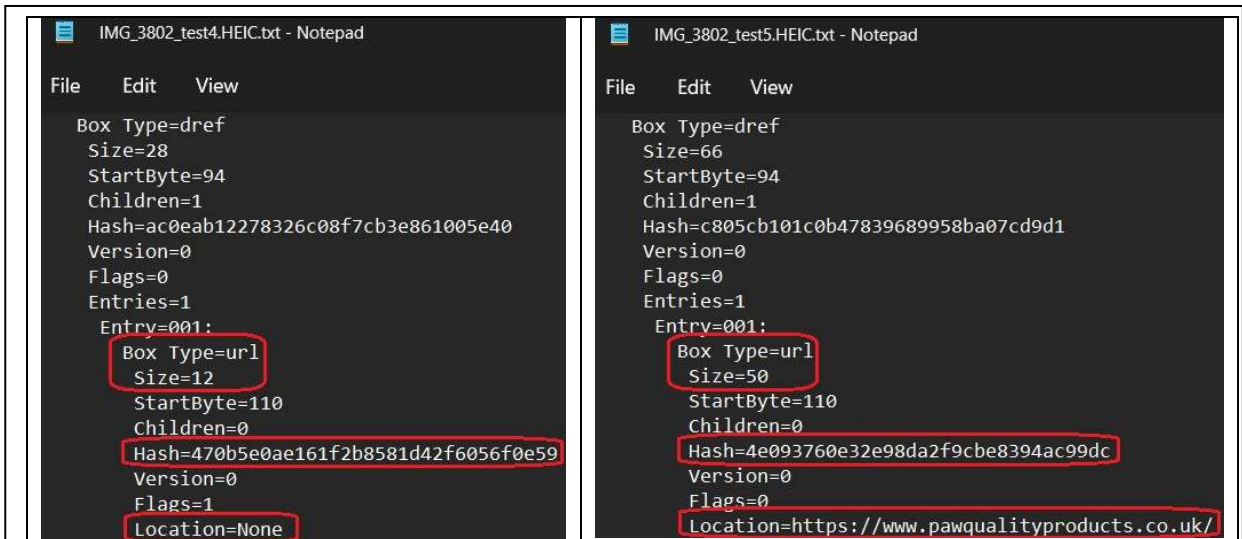


Figure 50 - Comparison of "dref" sections of contents files

One interesting observation made as part of this test was that images taken from iPhones always have a "url" entry in their "dref", box but the entry is empty. Since the "dref" box is typically located at the start of the file, this can be seen using a Hex Editor (see Figure 51).

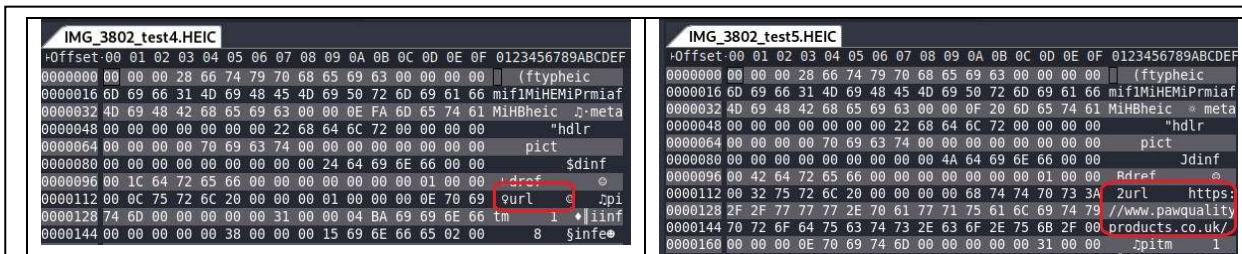


Figure 51 - Hex editor comparison of files with and without an embedded URL.

To check that the file with the update URL was still valid, and to see if a compliant viewer would display or open the URL, GIMP (*GIMP*, n.d.), "HEIC Viewer" from the Windows Store (*Get HEIC Image Viewer from the Microsoft Store*, n.d.), and an Online HEIC Image viewer (*HEIC File Viewer Online*, n.d.), were used to check that the file created with the updated URL was still valid, and to see if a compliant viewer referenced or displayed the URL. In all cases, the file was opened, but the URL was not displayed, nor was there any evidence that the URL was accessed by the viewer.

4.9 Test Case I – Nokia Conformance Tests

Conformance test images were sourced from Nokia (Technologies, 2016/2022), and a test script was created similar to the one in Appendix H – Batch Process. All 63 images were processed in under 30 seconds. See Table 3 – Summary of Nokia conformance test results for details of the output created and any errors logged. In summary, 41 files were processed without error, 5 had missing box definitions, 22 had errors logged by PyHeif/Exif_Read, and for those no Images/Exif data were extracted. In all cases a directory structure, Map file, Log file and Contents file were created. None caused the process to crash or abort abnormally.

Some of the errors appeared to be due to limitations of the PyHeif library. This was determined by comparing the resulting error messages from the PyHeif output to the log file with the “conformance_file_descriptions.xlsx” spreadsheet supplied with the conformance image files.

For example:

- 12 images (C026 to C032 and C034 to C038) generated the same error “Unsupported file-type: Unspecified: File does not include any supported brands”. These were all inter-encoded / intra-encoded image sequences rather than single images or image collections.
- 3 images (C005, C006, C052) generated the same error: “ERROR:cannot write mode RGBA as JPEG”. Of those, two were associated with an Alpha mask / Alpha Plane, and the other the primary item was a Thumbnail rather than an image or derived image.
- 4 images (C008, C013, C023, C039) generated the same error: “Invalid input: Item has no data: Item with ID **** has no compressed data”. Of those, two related to “identity derived” images, one was a grid of 4 derived images, and the other was two cropped images.
- 1 image (C044) generated error: “ERROR:Input is not a HEIF/AVIF file”. This was a “Predictively coded image item as the primary item without alternatives”.

For the others (C041, MIAF001, Multilayer004) it isn’t clear from the classifications whether this is likely to be an incompatibility or some other issue, so further investigation is required.

Table 3 – Summary of Nokia conformance test results (Test case I)

Test Image	Generated Files						Errors			Comments / Errors logged in Log file
	Dir	Log	Map	contents	Images	Exif	Box Defs	PyHeif Error	Exif Error	
C001	✓	✓	✓	✓	✓	✓				
C002	✓	✓	✓	✓	✓	✓				
C003	✓	✓	✓	✓	✓	✓				
C004	✓	✓	✓	✓	✓	✓				
C005	✓	✓	✓	✓	✓	✓		✗	✗	ERROR:cannot write mode RGBA as JPEG
C006	✓	✓	✓	✓	✓	✓		✗	✗	ERROR:cannot write mode RGBA as JPEG
C007	✓	✓	✓	✓	✓	✓				
C008	✓	✓	✓	✓	✓	✓		✗	✗	Invalid input: Item has no data: Item with ID 1006 has no compressed data
C009	✓	✓	✓	✓	✓	✓				
C010	✓	✓	✓	✓	✓	✓				
C011	✓	✓	✓	✓	✓	✓				
C012	✓	✓	✓	✓	✓	✓				
C013	✓	✓	✓	✓	✓	✓	✗	✗	✗	Invalid input: Item has no data: Item with ID 1003 has no compressed data 580:---No definition found for box type clap.
C014	✓	✓	✓	✓	✓	✓				
C015	✓	✓	✓	✓	✓	✓				
C016	✓	✓	✓	✓	✓	✓				
C017	✓	✓	✓	✓	✓	✓				
C018	✓	✓	✓	✓	✓	✓				
C019	✓	✓	✓	✓	✓	✓				
C020	✓	✓	✓	✓	✓	✓				
C021	✓	✓	✓	✓	✓	✓				
C022	✓	✓	✓	✓	✓	✓				
C023	✓	✓	✓	✓	✓	✓	✗	✗	✗	715:---No definition found for box type clap. Invalid input: Item has no data: Item with ID 1009 has no compressed data
C024	✓	✓	✓	✓	✓	✓				
C025	✓	✓	✓	✓	✓	✓				
C026	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C027	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C028	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C029	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C030	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C031	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C032	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C033	✓	✓	✓	✓	✓	✓				
C034	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C035	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C036	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C037	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C038	✓	✓	✓	✓	✓	✓		✗	✗	Unsupported file-type: Unspecified: File does not include any supported brands.
C039	✓	✓	✓	✓	✓	✓	✗	✗	✗	414:---No definition found for box type clap. 454:---No definition found for box type clap. Invalid input: Item has no data: Item with ID 1003 has no compressed data
C040	✓	✓	✓	✓	✓	✓				
C041	✓	✓	✓	✓	✓	✓	✗	✗	✗	244:---No definition found for box type edts. 842:---No definition found for box type ctt. 874:No definition found for box type csig. Unsupported file-type: Unspecified: File does not include any supported brands
C042	✓	✓	✓	✓	✓	✓				
C043	✓	✓	✓	✓	✓	✓				
C044	✓	✓	✓	✓	✓	✓		✗	✗	ERROR:Input is not a HEIF/AVIF file
C045	✓	✓	✓	✓	✓	✓				
C046	✓	✓	✓	✓	✓	✓				
C047	✓	✓	✓	✓	✓	✓				
C048	✓	✓	✓	✓	✓	✓				
C049	✓	✓	✓	✓	✓	✓				
C050	✓	✓	✓	✓	✓	✓				
C051	✓	✓	✓	✓	✓	✓				
C052	✓	✓	✓	✓	✓	✓		✗		ERROR:cannot write mode RGBA as JPEG
C053	✓	✓	✓	✓	✓	✓				
MIAF001	✓	✓	✓	✓	✓	✓		✗		No Images in the file
MIAF002	✓	✓	✓	✓	✓	✓				
MIAF003	✓	✓	✓	✓	✓	✓				
MIAF004	✓	✓	✓	✓	✓	✓				
MIAF005	✓	✓	✓	✓	✓	✓				
MIAF006	✓	✓	✓	✓	✓	✓				
MIAF007	✓	✓	✓	✓	✓	✓				
Multilayer001	✓	✓	✓	✓	✓	✓				
Multilayer002	✓	✓	✓	✓	✓	✓				
Multilayer003	✓	✓	✓	✓	✓	✓				
Multilayer004	✓	✓	✓	✓	✓	✓	✗	✗	✗	260:---No definition found for box type avcC. 332:---No definition found for box type oinf. 410:No definition found for box type tols. 424:No definition found for box type lhvC. ERROR: Invalid input: Non-existing item ID referenced: 'pitm' box references a
Multilayer005	✓	✓	✓	✓	✓	✓				

4.10 Test Case J – Auxiliary file / Steganography experiment

As a final test, an experiment was run to see if the Auxiliary image in a HEIF file (used on an iPhone to store Depth Map information for a scene) could be replaced and used along with very simple Steganographic techniques to hide data or potentially malicious code (Burt, 2022) within a HEIF file.

A python script was created that used the Editor tool (used in previous tests) to load a HEIF file (“IMG_3221_aux_edited.heic”), load an Auxiliary image from another HEIF file (in this case binary data exported from another HEIF file using the prototype tool), encode some data in the image (in this case, by simply replacing bytes rather than using an established Steganographic technique such as Least Significant Bit), adding the auxiliary image data to the end of the “mdat” box and altering the “iloc” entry for the Auxiliary Image to point to the new Auxiliary image data before writing out an edited version of the HEIF file (“IMG_3221_aux_edited.heic”).

Figure 52 shows the original image taken from an iPhone, along with the original Auxiliary (depth map) image.



Figure 52 - Original image along with its Auxiliary image

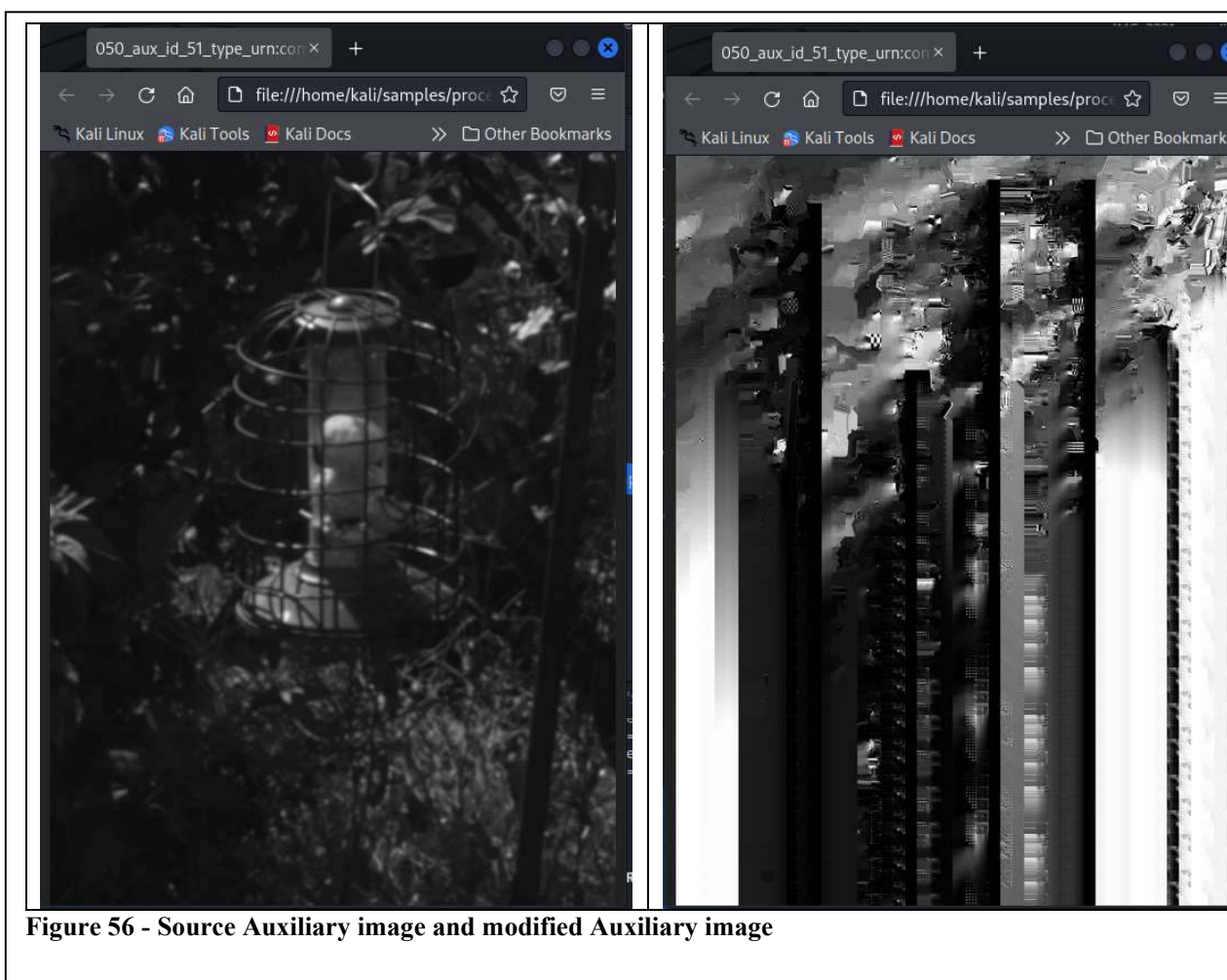


Figure 56 - Source Auxiliary image and modified Auxiliary image

The file was then opened using GIMP, and on an iPhone (Figure 57) to ensure it could still be viewed using compliant viewers and there was no obvious change to the rendering of the image. The binary data exported by the tool for the Auxiliary image was viewed in a Hex editor (Figure 58) to prove the data (in this case “PawQualityProducts.co.uk”) was successfully hidden in the HEIF file.

This test demonstrates that using only Python and a little effort, the Auxiliary Image in a HEIF file could be used as a method to distribute data using very simple steganographic techniques, and that it would be difficult to detect without deconstructing the file and exporting the images.

Since the original Auxiliary file data is still contained in the “mdat” box, but not referenced in the “iloc” entries that this test demonstrates that unreferenced data can be hidden in the “mdat” box.



Figure 57 - Modified image viewed on an iPhone

001_extent.bin																
Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
000096	1C	83	16	2A	50	4A	F3	B5	Lâ	P	≤	↓				
000104	36	C9	07	DC	61	69	0F	10	6f	•	a	◀				
000112	B3	CA	80	F6	77	99	48	EA		Ç	w	H	Ω			
000120	F8	F7	74	77	51	FA	87	49	°	≈	t	Q	Ç	I		
000128	AE	38	41	7E	75	99	58	4D	«	8A	u	X	M			
000136	40	E0	8A	C7	61	14	1D	20	@	α	è	a	+			
000144	D2	57	97	BD	6C	C0	A1	11	⌘	W	ù	l	í			
000152	FA	F9	6C	87	69	51	12	5D	·	·	l	i	+			
000160	DE	43	37	EE	74	4C	A0	06	⌘	C7	t	á	♠			
000168	CB	F2	F0	C4	79	42	BD	6F	⌘	≥	≡	y	ù			
000176	14	4A	D2	4E	50	8F	87	8D	⌘	J	T	P	ç	i		
000184	C0	8E	83	29	72	C1	44	15	L	Â	r	0	§			
000192	06	79	BA	01	6F	62	56	B8	♣	y		o	V	j		
000200	CD	0F	C7	AC	64	CD	D7	D4	=	*	†	d	†			
000208	DF	18	31	AC	75	9B	16	59	■	†	u	Y				
000216	89	BF	D0	E1	63	AF	41	8E	ë	†	c	AA				
000224	CF	64	B4	1F	74	94	CB	37	±	d	†	t	†	7		
000232	D1	C9	EA	8F	73	18	A0	DB	⌘	†	Ω	S	á			
000240	8A	DF	CD	D9	2E	4D	DA	8B	è	†	.	†	i			
000248	A6	20	72	47	63	52	9C	BD	â	†	r	c	†			
000256	A1	2C	EB	0D	6F	29	E3	B5	í	†	o	†	†			
000264	08	59	3B	4B	2E	69	E7	31	□	Y	;	.	†	1		
000272	83	17	09	A3	75	4F	A6	46	â	†	o	u	†	F		
000280	92	1F	06	16	6B	8E	51	17	Æ	†	♠	k	Q	±		
000288	1D	33	92	C5	42	E2	F0	BA	↵	3	Æ	†	B	†		

Figure 58 - Exported Auxiliary image data showing hidden data

NB: The tool could be enhanced to ensure that all “iloc” entries combined reference all of the content of the “mdat” box and that no unreferenced data exists.

5 Evaluation and Discussion

HEIF files are now in common use, but their complexity and lack of adequate tooling makes them difficult and time consuming for forensic examiners to analyse.

5.1 Hidden Image Content

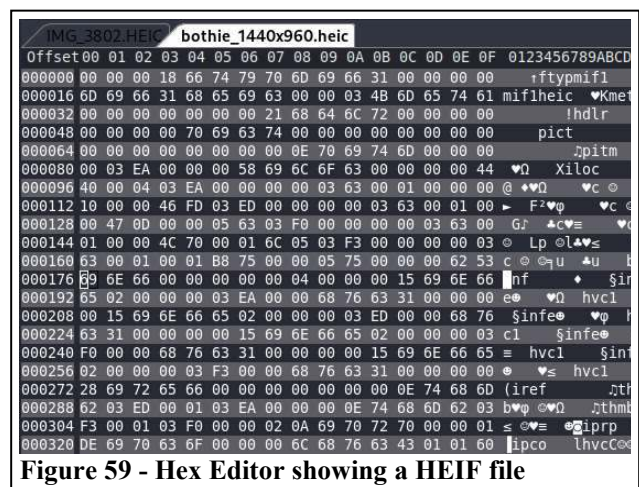
Until recently, even viewing images within a HEIF file would have been a challenge due to the lack of compliant viewer applications, but in recent years the number of compliant viewer applications across operating systems has improved somewhat and the images they contain can now be viewed quite easily. However, as per the HEIF specification, compliant viewer and editor applications such as GIMP will only display un-hidden images. As discussed and demonstrated previously, HEIF files can legitimately contain hidden images, but could also be used to distribute images or data of a prohibited or illegal nature. Therefore, one of the biggest issues for forensic investigators with regard to HEIF files is the identification, extraction and classification of hidden images.

Manual Analysis

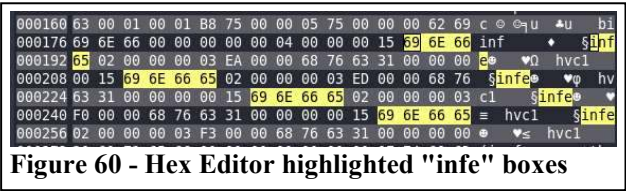
Without an appropriate tool, to analyse a HEIF file a forensic investigator would first extract un-hidden image content using a compliant viewer/editor such as GIMP. They would then have to manually parse and dissect the HEIF file using a Hex Editor or similar tool to determine whether a HEIF file contains any hidden images.

To manually identify hidden images, they would need to locate and count all “infe” boxes within the file, noting their “item_type” (“hvc1” for images), ID, and the state of their hidden flag, and comparing the results of this with the number of images displayed in the viewer application. However, this isn’t a simple task. Figure 59 shows a Hex Editor displaying a segment of a relatively simple HEIF file, that contains two images, neither of which is hidden.

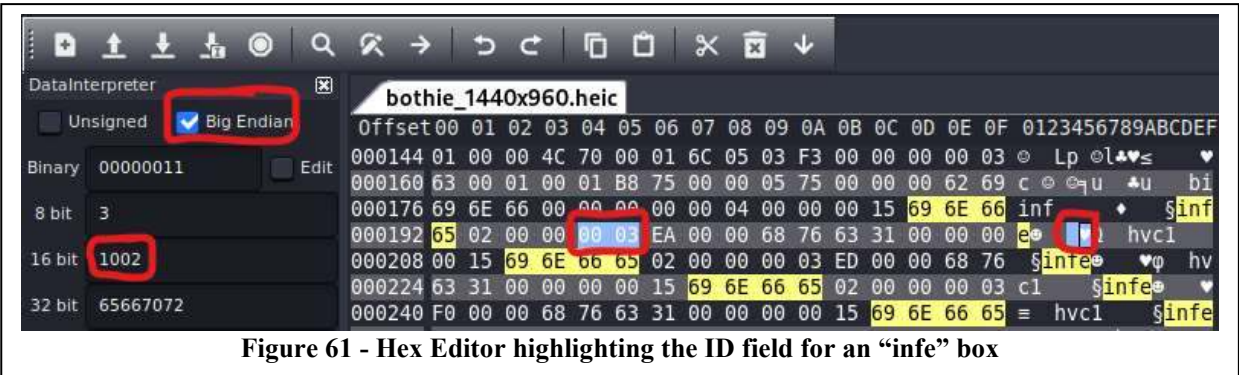
As can be seen, it is difficult to pick out the “infe” box entries and very difficult to interpret IDs and hidden flag values. Figure 60 highlights the “infe” box entries shown in Figure 59.



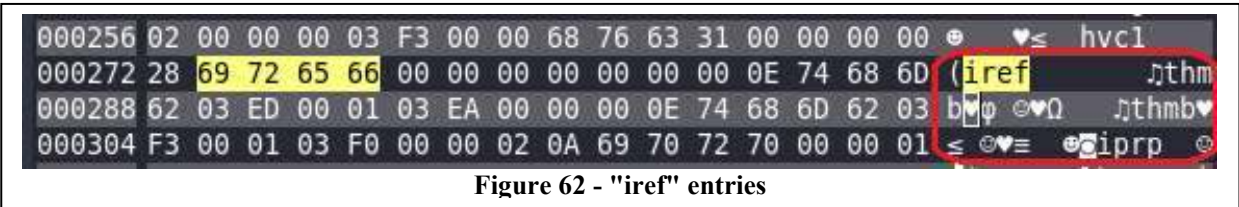
Once the “infe” box entries are located, determining their item-type is relatively straightforward as it is human readable (ie. in this case each is associated with



an image “hvc1”), however the ID and the state of the hidden flag aren’t quite so easy to see and determining their values can be difficult and prone to error. For example, Figure 18 and Figure 19 on page 51 show a comparison of a set and un-set hidden flag for an “infe” box. For the HEIF file shown in Figure 59 and Figure 60, though there are only two unhidden images, the file contains 4 “infe” boxes. To determine the reason for this, the “infe” entries must be cross-referenced with other sections of the HEIF file, and to do that, the ID of each “infe” box must be determined. Figure 61 shows the ID field highlighted for an “infe” box.



The ID is a 16 bit big-endian formatted binary integer value (in this case has a value of 1002), so not only would the forensic investigator have to know where to find the ID fields, they would also need to know how to correctly decode their values. The ID for each “infe” boxes (“1002”, “1008”, “1005” and “1011”), must be associated with entries in the “iref” box to determine their role. Figure 62 shows two thumbnail entries associating thumbnail id=“1005” with image id=“1002” and thumbnail id=“1011” with image id=“1008”, so from that the investigator would be able to determine that there are 2 images and two thumbnails with each thumbnail associated with (and generated from) each of the two images.



As can be seen, even determining basic image contents of a single HEIF file that does not contain any hidden images without a tool designed for the job is relatively difficult and time consuming and is likely to be an error prone task.

As stated previously, HEIF files can legitimately contain hidden images. An image generated with an iPhone for example has 48 individual hidden “tile” images that are combined to create the unhidden derived “grid” image. They also contain a thumbnail, Auxiliary image (ie. a depth map), Exif, and mime entries, meaning they have a total of 53 individual “infe” boxes, plus associated “iref” entries that describe their relationships. So, the forensic examiner must locate each of those items in the HEIF file using a Hex Editor, extract and correctly decode their contents, then cross reference and correctly interpret them before determining whether the image has even the potential to contain illegitimate hidden image content.

Since compliant HEIF file viewers only display unhidden images, to view all hidden images in a HEIF file a forensic investigator would also have to manually locate and clear the “hidden” flag for each “infe” box using a Hex Editor prior to opening the file in a compliant viewer.

Prototype Tool

The prototype tool developed as part of this thesis aims to address the issues discussed above. The Map file created during processing shows an outline of the complete file structure in a human readable form (see Figure 63). The location, size and end position along with any other useful information such as ID’s and state of the hidden flag are also shown. It is possible to validate the contents of the map file against the HEIF file using a Hex Editor to verify the start position for each box and item listed.

```
File-bothie_1440x960.heic, hash=1cf4152eaf81b921c4ced61146ac52e4
000000:ftyp(size=24, start=0, end=24, children=0, hash=9c6e6e64477a0084f856af3241ead362)
000024:meta(size=843, start=24, end=867, children=7, hash=2dd9a70619a11dfc78eda517ab9286ca)
000036:hdrl(size=33, start=36, end=69, children=0, hash=63266d7ddc9038364e39dc50d4e19dcb)
000069:pltx(size=14, start=69, end=83, children=0, hash=b9f8843bbac924588f76c8754c7e366)
000083:-ilcc(size=88, start=83, end=171, items=4, hash=27eb308c98a70885f164259/a4ae/c3)
000099:- Item=1, (size=18, start=99, end=117, extents=1, id=1002)
000109:- Extent=1 (size=8, start=109, end=117, data=(type=mdat, length=18173, start=883, end=19056, hash=30732129a56b65577fd7d557763f2647)
000117:- Item=2, (size=18, start=117, end=135, extents=1, id=1005)
000127:- Extent=1 (size=8, start=127, end=135, data=(type=mdat, length=1379, start=19056, end=20435, hash=7230c481bc463e417fd4e4e0711c30e8)
000135:- Item=3, (size=18, start=135, end=153, extents=1, id=1008)
000145:- Extent=1 (size=8, start=145, end=153, data=(type=mdat, length=93189, start=20435, end=113624, hash=c5b0ba8f12db15841e871004aa78aca5)
000153:- Item=4, (size=18, start=153, end=171, extents=1, id=1011)
000163:- Extent=1 (size=8, start=163, end=171, data=(type=mdat, length=1397, start=113624, end=115021, hash=72d1ed5e9b111b5f270a45833b86be94)
000171:-infe(size=98, start=171, end=269, children=4, hash=35896b6cd5fb455d57ddca77264dc250)
000185:- infe(size=21, start=185, end=206, children=0, hash=af4493f4fe494f6a61233363eed45d0, type=hvcl, id=1002)
000206:- infe(size=21, start=206, end=227, children=0, hash=ab74547a268845472d8312d1f2ba5568, type=hvcl, id=1005)
000227:- infe(size=21, start=227, end=248, children=0, hash=005bc16f03ef20d563fd320c40d2e1ea, type=hvcl, id=1008)
000248:- infe(size=21, start=248, end=269, children=0, hash=7f979df9900905e6e64e354c648593e5, type=hvcl, id=1011)
000269:-iref(size=40, start=269, end=309, children=2, hash=68c99ff82ee9fec2967e6f2d740e3fd6)
000281:-thumb(size=14, start=281, end=295, hash=16c79535e5ffid46329bc2e0a3fcb35, from_id=1005, to_ids=[1002])
000295:-thumb(size=14, start=295, end=309, hash=16c79535e5ffid46329bc2e0a3fcb35, from_id=1011, to_ids=[1008])
000309:-iprp(size=522, start=309, end=831, children=2, hash=63a1a370a01eb9df09c2de7b91c30293)
000317:-ipco(size=478, start=317, end=795, children=6, hash=ebc8d23f894b16b5daafc0ee056b4107)
000325:-hvcc(size=108, start=325, end=433, children=0, hash=24e00d0d796142702ee9ca4b19f2bceb)
000433:-ispe(size=20, start=433, end=453, children=0, hash=75d6b33fb81519ba5410667bce37b631)
000453:-hvcc(size=107, start=453, end=560, children=0, hash=b6d61ba5ca80d1adce35a9e14c8e2972)
000560:-ispe(size=20, start=560, end=580, children=0, hash=61f57fedcfff50d0186158f85dea)
000580:-hvcc(size=108, start=580, end=688, children=0, hash=24e00d0d796142702ee9ca4b19f2bceb)
000688:-hvcc(size=107, start=688, end=795, children=0, hash=b6d61ba5ca80d1adce35a9e14c8e2972)
000795:-ipma(size=36, start=795, end=831, children=0, hash=4c1d6af14bd7f2e1e183ab0b47f526e)
000831:-grpl(size=36, start=831, end=867, children=0, hash=86a83ff18945df5bb0e38cfb24f5171dh)
000867:mdat(size=114154, start=867, end=115021, children=0, hash=994681cb189d52906e6d89bf45551d2)
```

Figure 63 - Map file extract for a simple HEIF file.

If a file contained hidden images or other hidden items, the status of the hidden flag for each “infe” box would also be shown (see Figure 64).

```

001095:--infe(size=21, start=1095, end=1116, children=0, hash=c3ef8cf5dd779d467afb905b47e80169, type=hvc1, id=46 ***HIDDEN***)
001116:--infe(size=21, start=1116, end=1137, children=0, hash=d0587272a260a46510a46aec00be4685, type=hvc1, id=47 ***HIDDEN***)
001137:--infe(size=21, start=1137, end=1158, children=0, hash=4b0ebf8538ce7e2cdb35cfab637febe, type=hvc1, id=48 ***HIDDEN***)
001158:--infe(size=21, start=1158, end=1179, children=0, hash=916e34770d6ab6dad96fbb1ec0ccaed, type=grid, id=49)
001179:--infe(size=21, start=1179, end=1200, children=0, hash=b9c9d9b6d8415b303a2e439dda71fc22, type=hvc1, id=50)
001200:--infe(size=21, start=1200, end=1221, children=0, hash=641148e477c89943086301ee94479d49, type=hvc1, id=51 ***HIDDEN***)
001221:--infe(size=41, start=1221, end=1262, children=0, hash=3a334fec9b6e504cba4e955fb9169aba, type=mime, id=52 ***HIDDEN***)
001262:--infe(size=21, start=1262, end=1283, children=0, hash=1bbd875ef5e821c1a8bfe5b22f5bcc54, type=Exif, id=53 ***HIDDEN***)

```

Figure 64 – Map file showing hidden images.

The map file is probably the most useful for the forensic examiner to get an immediate indication of whether the HEIF file contains any hidden content, however, the forensic investigator may need to see the hidden image items for themselves in order to determine whether they are legitimate or not.

The tool automatically clears the hidden flag prior to image extraction to ensure hidden images are extracted (refer to Test Case E, sections 4.5). Figure 65 shows hidden tile images extracted from a HEIF file created on an iPhone. All images (including the Auxiliary (depth map) image are extracted.

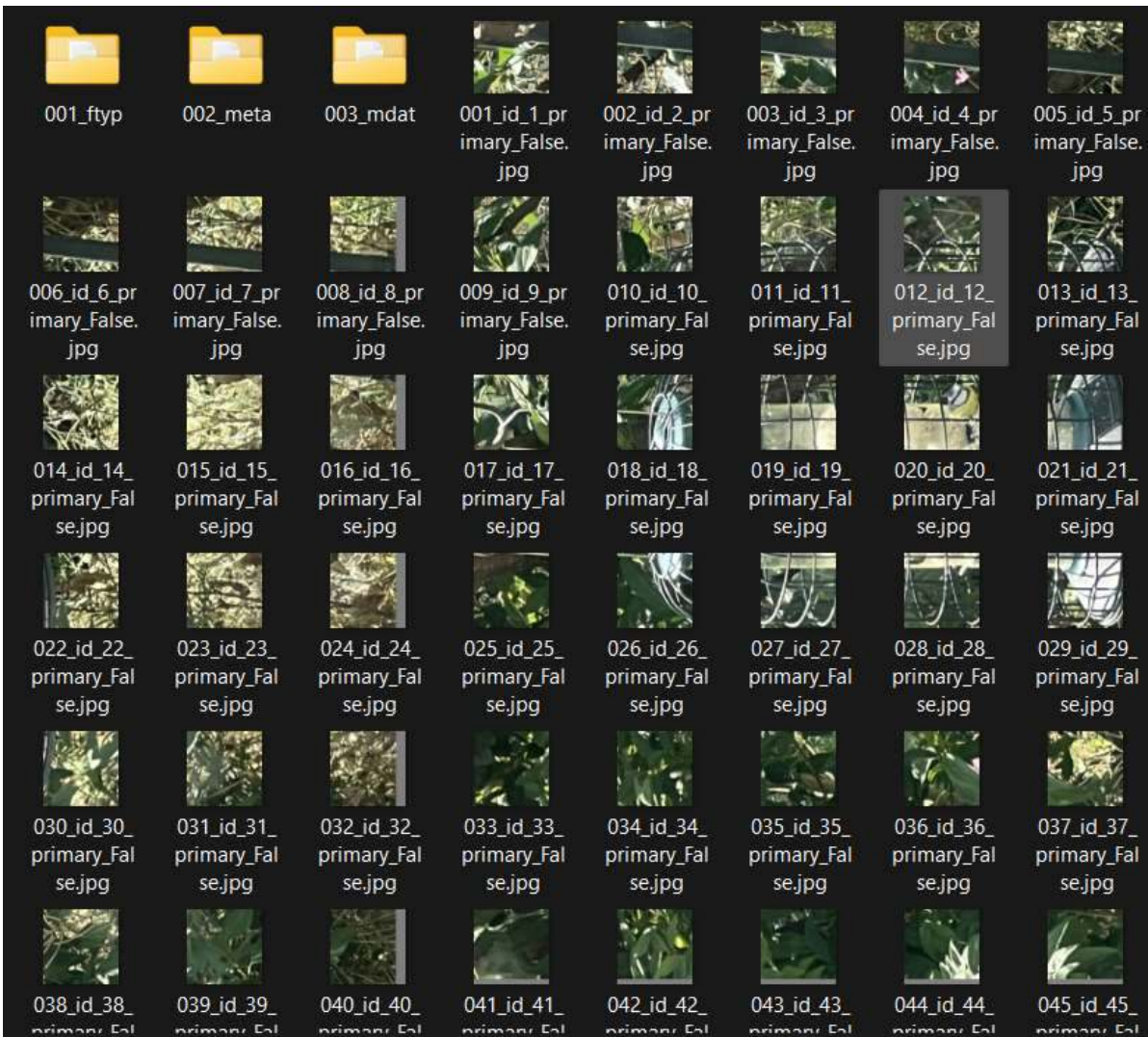


Figure 65 - Hidden images exported from an iPhone HEIF file

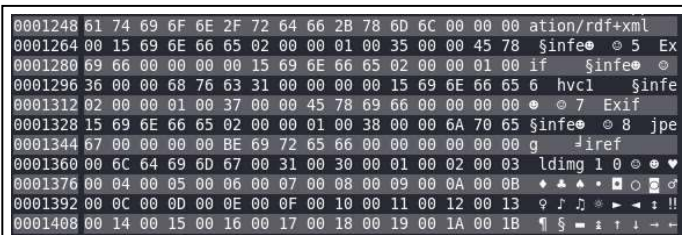
Each image is named with its “infe” box ID so can be cross-referenced with the Map and contents files.

5.2 Additional Hidden / Unrelated Content

Test Case G (section 4.7) demonstrates that it could be possible to use HEIF files to distribute additional, unrelated and perhaps illegal content that would be essentially invisible to a compliant viewer.

Manual Analysis

As with hidden images, the forensic investigator would need to view the HEIF file in a Hex Editor, but would need to understand the format in detail in order to determine whether the file contains any additional content that shouldn't be there. As seen in 5.1 above, HEIF files are difficult to read and interpret using a Hex Editor. Figure 66 shows a segment of a HEIF file in a Hex Editor that contains additional hidden JPEG image content that would not be rendered by a compliant viewer.

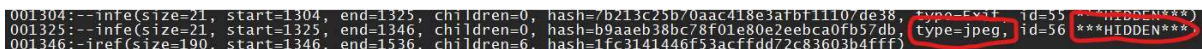


0001248 61 74 69 6F 6E 2F 72 64 66 2B 78 6D 6C 00 00 00 00 ation/rdf+xml
0001264 00 15 69 6E 66 65 02 00 00 01 00 35 00 00 45 78 \$infe 5 Ex
0001280 69 66 00 00 00 00 15 69 6E 66 65 02 00 00 01 00 if \$infe
0001296 36 00 00 68 76 63 31 00 00 00 00 15 69 6E 66 65 6 hvcl \$infe
0001312 02 00 00 01 00 37 00 00 45 78 69 66 00 00 00 00 7 Exif
0001328 15 69 6E 66 65 02 00 00 01 00 38 00 00 6A 70 65 \$infe 8 jpe
0001344 67 00 00 00 00 BE 69 72 65 66 00 00 00 00 00 00 g \$iref
0001360 00 6C 64 69 6D 67 00 31 00 30 00 01 00 02 00 03 ldimg 1 0
0001376 00 04 00 05 00 06 00 07 00 08 00 09 00 0A 00 0B
0001392 00 0C 00 0D 00 0E 00 0F 00 10 00 11 00 12 00 13
0001408 00 14 00 15 00 16 00 17 00 18 00 19 00 1A 00 1B

Figure 66 - Hex editor showing a segment of a HEIF file with additional hidden content

Prototype Tool

Figure 67 shows a segment of the Map file generated by the tool, referencing the same segment of the HEIF file is shown in Figure 67.



001304:--infe(size=21, start=1304, end=1325, children=0, hash=/b213c25b70aac418e3afb1110/de38, type=Exif, id=55, **HIDDEN**)
001325:--infe(size=21, start=1325, end=1346, children=0, hash=b9aaeb38bc78f01e80e2eebca0fb57db, type=jpeg, id=56, **HIDDEN**)
001346:--iref(size=190, start=1346, end=1536, children=6, hash=1fc3141446f53acffdd72c83603b4fff)

Figure 67 - Map file showing additional hidden content

The additional JPEG content is clearly identifiable and is shown along with the status of its hidden flag. Experiments show that compliant viewers such as GIMP do not interpret and display this additional JPEG content whether hidden or not.

Also, the binary data for the additional content (in this case a JPEG image) is automatically exported (see Figure 68) and can be identified using its item ID (shown in the Map file entry). In this case a JPEG image was embedded into the HEIF file, but it could have been any binary content. The HEIF file itself is valid, and can be viewed using GIMP, but the additional JPEG content is not displayed (see Figure 47 on page 51) and is essentially completely invisible to a user or indeed a forensic examiner. The test demonstrates that the tool can be used to identify, extract and ultimately view additional content such as this.

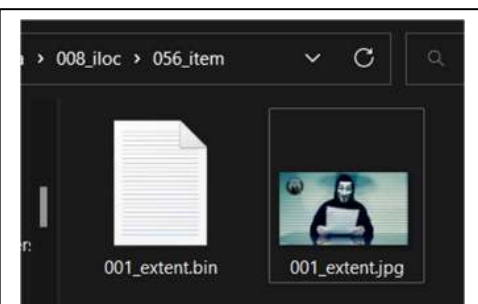


Figure 68 - Exported JPEG image

5.3 Exif Metadata

Metadata associated with images can be useful for forensic examiners. For example, it can be used to establish the date & time, location (GPS coordinates) and even the type of device used to capture an image. This type of information may be required as supporting evidence for a criminal investigation. However, as demonstrated, it is possible for a HEIF file to contain additional hidden and even unrelated content, and it is also possible to attach additional Exif metadata to that hidden / additional content.

Manual analysis

Existing tools such as (*ExifTool by Phil Harvey*, n.d.) can be used to extract Exif data from HEIF files. However, experiments show that tools like this will only extract the Exif data associated with the primary image in the HEIF file, or the first Exif data encountered in the file (see 4.6 Test Case F - Additional Exif data).

It would be possible for a forensic investigator to manually extract additional Exif data, but to do this, they would first need to locate all “infe” boxes associated Exif

```
0001216 68 76 63 31 00 00 00 00 29 69 6E 66 65 02 00 00 hvcl )infe
0001232 01 00 34 00 00 6D 69 6D 65 00 61 70 70 6C 69 63 04 mime applic
0001248 61 74 69 6F 6E 2F 72 64 66 2B 78 6D 6C 00 00 ation/rdf+xml
0001264 00 15 69 6E 66 65 02 00 00 01 35 00 00 45 78 $infe 5 Ex
0001280 69 66 00 00 66 65 00 15 69 6E 66 65 02 00 00 01 00 if $infe
0001296 02 00 00 68 76 63 31 00 00 00 00 15 69 6E 66 65 6 hvcl $infe
0001312 06 00 00 01 00 37 00 00 45 78 69 66 00 00 00 05 07 Exif
0001328 BE 69 72 65 66 00 00 00 00 6C 64 69 6D 4ir f ldim
0001344 67 00 31 00 30 00 01 00 02 00 03 00 04 00 05 00 g l 0 0 0 0 0 0
```

Figure 69 - "infe" boxes associated with Exif data

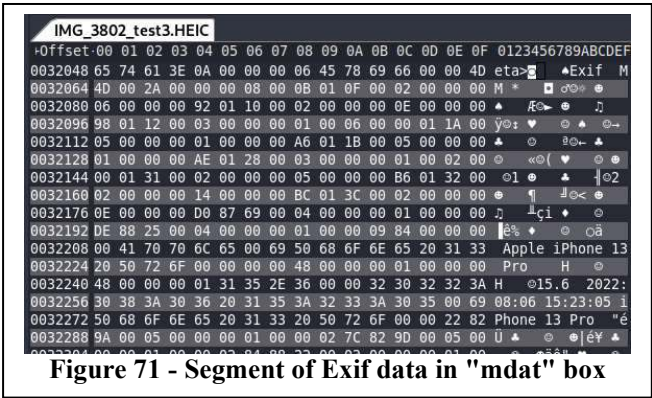
data and extract their ID values. As shown previously, this isn't an easy task, Figure 69 shows a segment of a HEIF file with two "infe" boxes associated with Exif data. The ID values for the two entries in this case are "53" and "55". The investigator would then need to locate "iloc" box, and locate the appropriate entries by ID. From those they would need to locate and extract Item and Extent entries which describe where in the "mdat" box the Exif data is stored. Figure 70 shows "iloc" box item and extent entries for

```
0003776 10 E2 E0 00 00 00 EA 00 31 00 00 00 00 00 01 00 74 1 7
0003776 00 7B CA 00 00 01 6B 00 35 00 00 00 00 01 00 { } 5
0003792 00 7D 35 00 00 0A A0 00 36 00 00 00 00 01 00 } 5 6
0003808 10 82 01 00 00 3A 8F 00 37 00 00 00 00 01 00 } é 5A 7
0003824 11 BB CA 00 00 0A A0 00 00 00 01 6D 64 61 74 00 { } 5A 7 omdat
```

Figure 70 - "iloc" box Item and Extent entries for IDs 53 and 55

IDs 53 and 55. Again, this is big-endian encoded binary data that describes the location and size of the Exif data within the “mdat” box (in this case relative to the beginning of the file). The forensic investigator would need to decode this information to determine start-byte and length of the Exif data then manually carve the binary data from the “mdat” box into separate files, then interpret the data either manually or using an Exif reader tool.

Figure 71 shows a segment of the “mdat” box containing Exif data for one of the data items. The data extends from byte 32053 to byte 34773.



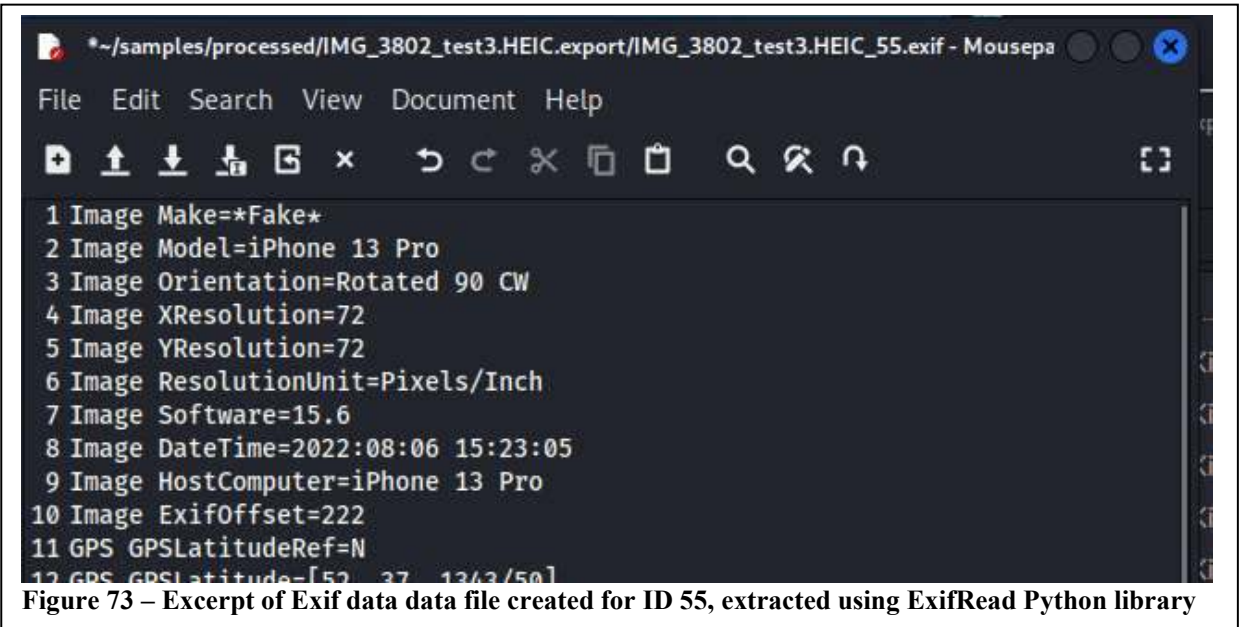
Prototype Tool

The prototype tool addresses these issues by extracting, decoding and recording the “infe” boxes and “iloc” Item and Extent entries to the Map file (see Figure 72).



This gives the forensic investigator an immediate indication of the location of all Exif data within the HEIF file without the need to manually locate and decode the “infe” and “iloc” data using a Hex Editor.

Additionally, the tool automatically extracts the binary data from the “mdat” box for each Exif data item, then uses the “ExifRead” Python library to decode the data, storing it in text files in a human readable form (see Figure 73, Test Case F, section 4.6).



5.4 External References

The HEIF file format supports references to external content, though several HEIF viewers were tested, none appeared to display or reference an external URL. However, a thorough forensic examination would require external references to be extracted for investigation.

Manual Analysis

Compared to other forms of data, manual extraction of URLs referencing external content is relatively simple. The forensic investigator need only locate the “url” or “urn” entries in the “dref” box in the HEIF file using a Hex Editor (see Figure 74).



Figure 74 - Hex Editor showing the "url" entry in the "dref" box

Prototype tool

Test Case H shows that the tool can be used to identify and extract references to external content as the data is automatically extracted and stored in the Contents file (see Figure 75).

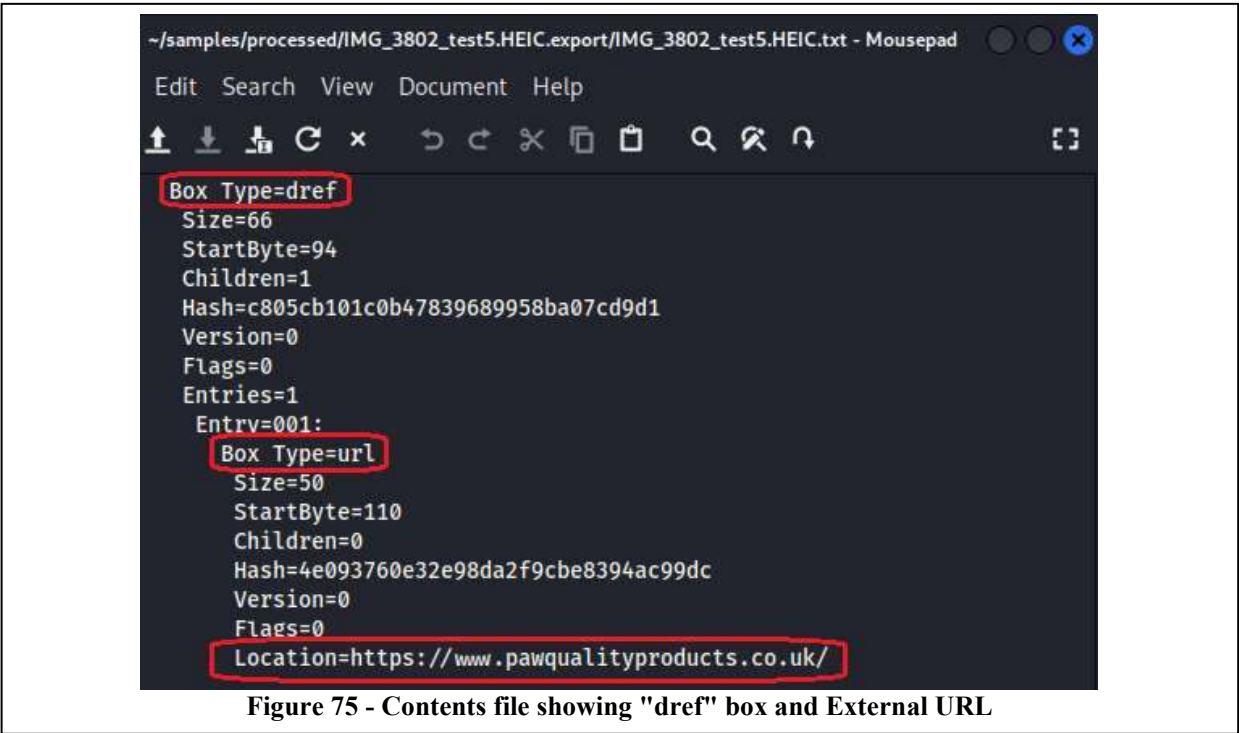


Figure 75 - Contents file showing "dref" box and External URL

5.5 File, Box, Item Hashes

File hashes are used to quickly identify and catalogue images and other file types during a forensic investigation. File hashes taken during an investigation can be matched against databases of known prohibited or illegal content.

Whilst this works well for JPEG and other single-image file formats, it becomes a little more complex for HEIF files. As discussed previously, HEIF files can contain multiple hidden images, and additional unrelated content. A file hash may uniquely identify an individual HEIF file, but this will not help to identify individual images or other content within a HEIF file.

Further, Test Case D (section 4.4) demonstrates that it is possible to subtly change the internal content or even the structure of a HEIF file, without changing its image output in any way. This would mean that a HEIF file previously identified as prohibited or illegal could be missed due to very simple manipulation.

McKeown & Russell, (2020) recommend a hash value be generated for each component within a HEIF file in addition to the overall file hash.

Manual analysis

Manually generating a file hash can be done easily using the “md5sum” command in Linux. However, manually deconstructing a HEIF file into its component parts using a Hex Editor would be an extremely complex, time consuming and error-prone task.

Prototype tool

The prototype tool addresses this issue by parsing the HEIF file, building an in-memory model, extracting and individually hashing the binary representations of each Box, Item and any associated binary data from the “mdat” box.

Test Case A (section 4.1) shows that component and data hashes are stored in the Map and contents files. Further, the map file could be parsed and its contents uploaded to a database to allow cross-referencing of HEIF file contents to identify common components across different HEIF files.

The tool does not however perform perceptual hashing on extracted images. This is one area where the tool could be enhanced.

5.6 HEIF File Structure & Content

HEIF files are highly structured and contain some human readable content, however much of the data held within them is encoded. Figure 76 shows a HEIF file viewed in a text editor.

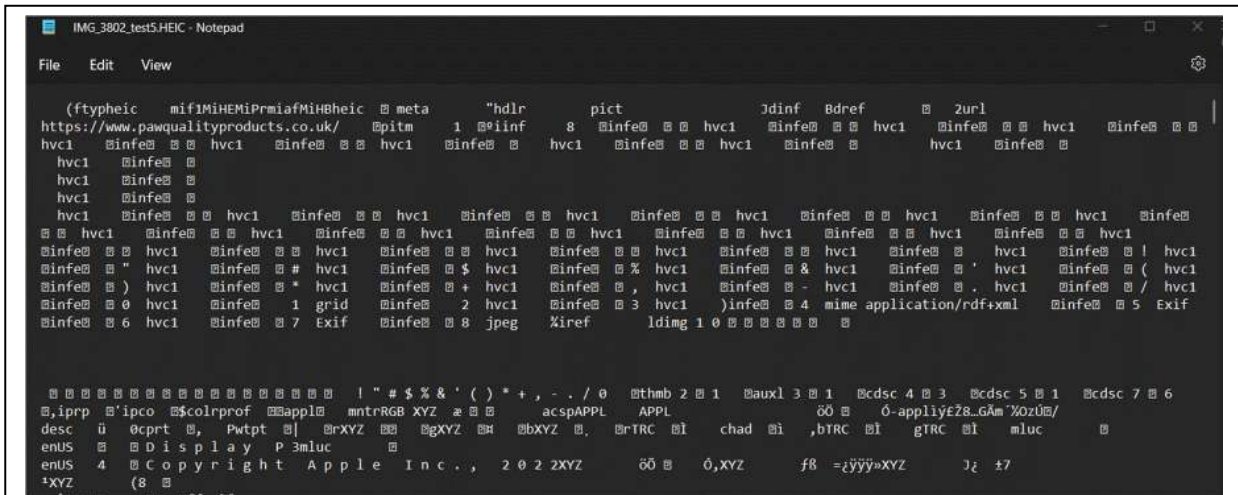


Figure 76 - Heif file as viewed using a simple text editor

Manual analysis

Using a text editor, it would be possible for a forensic investigator to identify the box types and a few human readable words, but most of the content is encoded and requires reference to the ISOBMFF and HEIF file standards to help decode it. A Hex Editor helps a little, however extrapolation of the file structure and extraction of useful information would still be a complicated, time consuming and error-prone task.

Prototype Tool

The prototype tool aims to address these issues by parsing a HEIF file and building an in-memory model representing its structure. This is then used to create a map file which outlines the file structure (one box / item per line), a contents file showing all human readable details of each box and item within the file, and a directory structure with all binary contents of each file component extracted for analysis using other tools and techniques. A HEIF file can be processed with its structure, details, hashes, images and Exif data extracted in around one second or less (depending on the complexity of the file).

5.7 Auxiliary File + Steganography

Test Case J – Auxiliary file / Steganography experiment demonstrates that using nothing more than Python and a little effort, it is possible to embed data within the Auxiliary image of a HEIF file. Apple uses Auxiliary images in HEIF files to store Depth Map information for Photographs captured using iPhones and other Apple devices. However, the Auxiliary image itself is marked as hidden and will therefore never be displayed using compliant viewers. The experiment demonstrates that the Auxiliary image can have data embedded within it and even be completely replaced with no visible affect to the rendered image whatsoever.

Further, since the test retained the original Auxiliary image data, and tagged new modified image data to the end of the “mdat” box, there appears to be no validation between the “iloc” entries and “mdat” box for complaint viewers (at least, neither GIMP nor an iPhone appeared to detect it or flag the file as invalid).

Recently, malware was found to have been distributed using images of Microsoft Logos (Burt, 2022). Unlike scripts and executable binaries, image files tend not to be blocked or scrutinized by firewalls so the ability to embed code with an Auxiliary image, or even simply add code to the “mdat” box of a HEIF file which is unreferenced by “iloc” entries could pose a significant security risk.

Further investigation into detecting steganographic data stored within Auxiliary images, and adding functionality to report on unreferenced data held within the “mdat” box is recommended.

5.8 Extensibility / Extend box definitions

Since the base ISOBMFF file format upon which the HEIF file format is based, the tool may encounter files which it cannot fully interpret. Fortunately, the log file created during processing explicitly identifies parsing errors (see Test Case I, Table 3). Additionally, the base ISOBMFF library that was used as the foundation for the prototype tool can be easily extended.

Addition of a new box structure to the process would be done creating a new class file for the box, being sure to implement key methods such as `__init__` (used to create an instance of a box object based on the box header), `get_box_size()` & `get_box_size_with_header()`, `read` (used to read the rest of the box contents from the file based on the box type), `writeText` (outputs text based information to a file), `writeData` (writes binary data to a file), `write` (recursively writes all box details to a file), and `getBinaryDataFromFile` (which retrieves binary box information from a HEIF file).

Once the new class file is created, it's integrated into the process by adding it to the import list in the "`__init__.py`" file.

5.9 Batch Processing

A forensic investigator may be required to analyse, catalogue and cross-reference a large number of HEIF files during an investigation. As discussed previously, this would be extremely difficult and very time consuming using manual techniques.

The prototype tool aims to address this issue by automatically extracting content and salient information. Additionally, command line parameters can be passed to the tool allowing the process to be scripted and automated. This is demonstrated using Test Cases A and I.

5.10 Facilitating further analysis using other tools

Though the prototype tool helps to resolve many of the issues a forensic examiner faces when analysing HEIF files, it will not do everything required.

For example, the tool does not currently implement perceptual hashing meaning it is not possible to compare images contained within a HEIF file with image databases based on image features using this tool alone. Also, the tool does not decode image sequences or video.

However, deconstruction of the file and automatic extraction of binary data from the “mdat” box could help to facilitate further analysis using other tools.

6 Conclusion and Recommendations

Conclusion

For a forensic investigator, detailed analysis of HEIF files is currently a relatively difficult, complex, time consuming and error prone process due the complexities of the file format and the lack of adequate tooling appropriate for the task. Detailed analysis of a large number of HEIF files within a reasonable amount of time using manual techniques would be impractical and could jeopardise a forensic investigation.

The prototype tool developed as part of this project aims to address many of the issues faced by forensic investigators in relation to the analysis of HEIF files.

The prototype tool processes HEIF files quickly and automatically extracts all images, metadata, and human-readable information. Additionally, the file structure is mapped, individual file components are hashed, and binary contents are extracted ready for cross-referencing and analysis using other tools.

Finally, it is possible to script and automate the processing of a large number of HEIF files, significantly reducing the time and effort required.

Recommendations and further work

The prototype tool could be enhanced to detect potential file-types when extracting binary content, either by using the “type” field of the “infe” boxes or by detecting “Magic Numbers” within the binary content. Binary extracts could then be created with an appropriate file extension to ensure it can be opened and viewed correctly. The prototype tool could also be further enhanced to include Perceptual Hashing of extracted images to allow searches of image databases to be made based on similarity of rendered image content rather than identical binary content.

Some errors were encountered during analysis of the some of the Nokia Conformance test files (see section 4.9, Table 3). The prototype should be enhanced to implement the missing boxes, and the errors encountered during image extraction should be investigated to determine whether they are due to limitations of the PyHeif library used. If so, alternative libraries could be used instead of or in addition to the PyHeif library.

Since the tool has been developed using Python, it should be possible to create a plug-in for Autopsy or other forensic tools.

The potential for Auxiliary images to be used in conjunction with Steganographic techniques to covertly distribute data should be further investigated, however perhaps the most urgent area for further study is the possibility of using unreferenced data contained within the “mdat” box (ie. data not referenced by “iloc” entries) to distribute illegal content or even malware potentially circumventing current firewall and file security checks (refer to section 4.10).

Study Limitations

Due to time-constraints certain desirable features were deemed to be out-of-scope for the prototype tool. This includes extraction of image sequences and video. Further work is recommended to add features deemed out-of-scope to the tool.

Additionally, due to the complexity and flexibility of the HEIF file format the test cases created only cover a sub-set of the entire HEIF feature set. Further work should be conducted to expand the number of test cases to allow for greater coverage of the HEIF feature set.

7 References

- Ali, R. R., Mohamad, K. M., Jamel, S., & Khalid, S. K. A. (2005). A REVIEW OF DIGITAL FORENSICS METHODS FOR JPEG FILE CARVING. . . Vol., 17, 16.
- Ansari, A. S., Mohammadi, M. S., & Ahmed, S. S. (2020). Digital Colour Image Steganography for PNG Format and Secured Based on Encoding and Clustering. *International Journal of Engineering Research and Technology*, 13(2), 345. <https://doi.org/10.37624/IJERT/13.2.2020.345-354>
- Apple Support. (2021, November 4). *Using HEIF or HEVC media on Apple devices*. Apple Support. <https://support.apple.com/en-gb/HT207022>
- Arshad, H., Jantan, A. B., & Abiodun, O. I. (2018). Digital Forensics: Review of Issues in Scientific Validation of Digital Evidence. *Journal of Information Processing Systems*, 14(2), 346–376. <https://doi.org/10.3745/JIPS.03.0095>
- Bellard, F. (2016). *BPG Image format*. <https://bellard.org/bpg/>
- Burt, J. (2022, October 2). *Spyware found hidden in Microsoft logo using stenography*. https://www.theregister.com/2022/10/02/witchetty_windows_logo_spyware/
- Cecco, L. (2019, September 25). Canada's Green Party alters photo of leader using single-use cup. *The Guardian*. <https://www.theguardian.com/world/2019/sep/25/canadas-green-party-alters-photo-of-leader-elizabeth-may-using-single-use-cup>
- Cheeky4n6monkey. (2017, October 14). Cheeky4n6Monkey - Learning About Digital Forensics: Monkey takes a .heic. *Cheeky4n6Monkey - Learning About Digital*

- Forensics*. <https://cheeky4n6monkey.blogspot.com/2017/10/monkey-takes-heic.html>
- Concion, D., & Apple Inc. (2017a). •*High Efficiency Image File Format (Presentation)*. https://devstreaming-cdn.apple.com/videos/wwdc/2017/513fzgbviu23l/513/513_high_efficiency_image_file_format.pdf
- Concion, D., & Apple Inc. (2017b). *High Efficiency Image File Format—WWDC17—Videos*. Apple Developer. <https://developer.apple.com/videos/play/wwdc2017/513>
- Europe's CSAM scanning plan unpicked. (n.d.). *TechCrunch*. Retrieved 3 August 2022, from <https://social.techcrunch.com/2022/05/11/eu-csam-detection-plan/>
- ExifTool by Phil Harvey*. (n.d.). Retrieved 18 November 2022, from <https://exiftool.org/>
- Faulkner, T. (2018, March 30). *Digital Forensics of HEIC files from iOS*. Paraben Corporation. <https://paraben.com/digital-forensics-of-heic-files-from-ios/>
- Ferreira, W. D., Ferreira, C. B. R., da Cruz Júnior, G., & Soares, F. (2020). A review of digital image forensics. *Computers & Electrical Engineering*, 85, 106685. <https://doi.org/10.1016/j.compeleceng.2020.106685>
- Francisco, T. C. in S. (n.d.). *Apple quietly wipes away details of its derided CSAM plan*. Retrieved 6 August 2022, from https://www.theregister.com/2021/12/16/apple_deletes_csam_scanning_plan/
- Garrad, J. (n.d.). *Computer hacking archive at TNMOC*. The National Museum of Computing. Retrieved 31 July 2022, from <https://www.tnmoc.org/news-releases/2016/5/18/computer-hacking-archive-at-tnmoc>

Get HEIC Image Viewer from the Microsoft Store. (n.d.). Retrieved 18 November 2022, from <https://apps.microsoft.com/store/detail/heic-image-viewer/9MXHSBJRS7SH?hl=en-us&gl=us>

GIF89a Specification. (1990, July 31). <https://www.w3.org/Graphics/GIF/spec-gif89a.txt>

GIMP. (n.d.). GIMP. Retrieved 18 November 2022, from <https://www.gimp.org/>

Graphics Interchange Format. (1987, June 15).

<https://www.w3.org/Graphics/GIF/spec-gif87.txt>

Hannuksela, M. M., Lainema, J., & Malamal Vadakital, V. K. (2015). The High Efficiency Image File Format Standard [Standards in a Nutshell]. *IEEE Signal Processing Magazine*, 32(4), 150–156.

<https://doi.org/10.1109/MSP.2015.2419292>

HEIC File Viewer Online. (n.d.). CompressJPG.Net | Heic Images Viewer. Retrieved 18 November 2022, from <https://compressjpg.net/open-heic-file/>

Heikkilä, L. (2016). *High Efficiency Image File Format implementation.* 61.

Hiki, M. (2021). *Isobmff* [Python]. <https://github.com/m-hiki/isobmff> (Original work published 2017)

How to use Magic Eraser on the Google Pixel. (n.d.). ZDNET. Retrieved 5 November 2022, from <https://www.zdnet.com/article/how-to-use-magic-eraser-on-the-google-pixel/>

IACIS - History. (n.d.). IACIS. Retrieved 31 July 2022, from <https://www.iacis.com/about/history/>

Indecent and Prohibited Images of Children | The Crown Prosecution Service. (n.d.). Retrieved 6 August 2022, from <https://www.cps.gov.uk/legal-guidance/indecent-and-prohibited-images-children>

- ISO Base Media File Format | MPEG*. (n.d.). Retrieved 21 July 2022, from <https://mpeg.chiariglione.org/standards/mpeg-4/iso-base-media-file-format>
- ISO/IEC 23008-12:2017 Information technology—High efficiency coding and media delivery in heterogeneous environments*. (2017). ISO/IEC.
- ISO/IEC JTC 1SC 29—ISO/IEC FDIS 14496-12.pdf*. (n.d.).
- Jerian, M. (2017, September 29). *HEIF Image Files Forensics: Authentication Apocalypse?* Amped Blog. <https://blog.ampedsoftware.com/2017/09/29/heif-image-files-forensics-authentication-apocalypse/>
- Kabachinski, J. (2007). TIFF, GIF, and PNG: Get the Picture? *Biomedical Instrumentation & Technology*, 41(4), 297–300. [https://doi.org/10.2345/0899-8205\(2007\)41\[297:TGAPGT\]2.0.CO;2](https://doi.org/10.2345/0899-8205(2007)41[297:TGAPGT]2.0.CO;2)
- Kee, E., Johnson, M. K., & Farid, H. (2011). Digital Image Authentication From JPEG Headers. *IEEE Transactions on Information Forensics and Security*, 6(3), 1066–1075. <https://doi.org/10.1109/TIFS.2011.2128309>
- Kirsch, R. A. (1998). SEAC and the start of image processing at the National Bureau of Standards. *IEEE Annals of the History of Computing*, 20(2), 7–13. <https://doi.org/10.1109/85.667290>
- Kirsch, R. A., Cahn, L., Ray, C., & Urban, G. H. (1958). Experiments in processing pictorial information with a digital computer. *Papers and Discussions Presented at the December 9-13, 1957, Eastern Joint Computer Conference: Computers with Deadlines to Meet on XX - IRE-ACM-AIEE '57 (Eastern)*, 221–229. <https://doi.org/10.1145/1457720.1457763>
- Krawetz, N. (2019, February). *HEIC Yeah—The Hacker Factor Blog*. <https://www.hackerfactor.com/blog/index.php?/archive/833-HEIC-Yeah.html>
- Li, M., Lal, C., Conti, M., & Hu, D. (2021). LEChain: A blockchain-based lawful evidence management scheme for digital forensics. *Future Generation*

Computer Systems, 115, 406–420.

<https://doi.org/10.1016/j.future.2020.09.038>

Mandeel, T. H., Imran Ahmad, M., Khalid, N. A. A., & Md Isa, Mohd. N. (2021). A

Comparative Study on Lossless compression mode in WebP, Better Portable Graphics (BPG), and JPEG XL Image Compression Algorithms. *2021 8th International Conference on Computer and Communication Engineering (ICCCE)*, 17–22. <https://doi.org/10.1109/ICCCE50029.2021.9467224>

Mani, R. G., Parthasarathy, R., Eswaran, S., & Honnavalli, P. (2022). A Survey on

Digital Image Forensics: Metadata and Image forgeries. *Workshop on Applied Computing*, 22–55.

McKeown, S., & Russell, G. (2020). Forensic Considerations for the High Efficiency

Image File Format (HEIF). *2020 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, 1–8.

<https://doi.org/10.1109/CyberSecurity49315.2020.9138890>

Milani, S., Fontani, M., Bestagini, P., Barni, M., Piva, A., Tagliasacchi, M., & Tubaro,

S. (2012). An overview on video forensics. *APSIPA Transactions on Signal and Information Processing*, 1(1). <https://doi.org/10.1017/ATSIP.2012.2>

Mobile Operating System Market Share Worldwide. (n.d.). StatCounter Global Stats.

Retrieved 5 November 2022, from <https://gs.statcounter.com/os-market-share/mobile/worldwide>

Nokia. (2021). *HEIF Example Images—High Efficiency Image File Format*.

<https://nokiatech.github.io/heif/examples.html>

Online Safety Bill, As Amended. (2022). House of Commons.

https://publications.parliament.uk/pa/bills/cbill/58-03/0121/amend/onlinesafety_rm_rep_0706.pdf

- Paes, A., & pypi.org. (n.d.). *pyheif: Python 3.6+ interface to libheif library* (0.7.0) [Python; MacOS :: MacOS X, POSIX :: Linux]. Retrieved 30 October 2022, from <https://github.com/carsales/pyheif>
- PawQualityProducts/HeifER: Heif ER, Prototype forensic analysis tool for HEIF files.* (n.d.). GitHub. Retrieved 5 November 2022, from <https://github.com/PawQualityProducts/HeifER>
- Petersilge, C. A. (2019). Fundamentals of Enterprise Photodocumentation: Connecting the Clinical and Technical—a Review of Key Concepts. *Journal of Digital Imaging*, 32(6), 1052–1061. <https://doi.org/10.1007/s10278-019-00212-4>
- Piva, A. (2013). An Overview on Image Forensics. *ISRN Signal Processing*, 2013, 1–22. <https://doi.org/10.1155/2013/496701>
- Pollitt, M. (2010). A History of Digital Forensics. In K.-P. Chow & S. Shenoi (Eds.), *Advances in Digital Forensics VI* (Vol. 337, pp. 3–15). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-15506-2_1
- Ristenbatt, R. R., Hietpas, J., De Forest, P. R., & Margot, P. A. (2022). Traceology, criminalistics, and forensic science. *Journal of Forensic Sciences*, 67(1), 28–32. <https://doi.org/10.1111/1556-4029.14860>
- Roy, A., Dixit, R., Naskar, R., & Chakraborty, R. S. (2020). Digital Image Forensics. In A. Roy, R. Dixit, R. Naskar, & R. S. Chakraborty, *Digital Image Forensics* (Vol. 755, pp. 1–10). Springer Singapore. https://doi.org/10.1007/978-981-10-7644-2_1
- Samanta, P., & Jain, S. (2021). Analysis of Perceptual Hashing Algorithms in Image Manipulation Detection. *Procedia Computer Science*, 185, 203–212. <https://doi.org/10.1016/j.procs.2021.05.021>

- Sévi, I. (n.d.). *ExifRead: Read Exif metadata from tiff and jpeg files*. (3.0.0) [Python; OS Independent]. Retrieved 30 October 2022, from <https://github.com/ianare/exif-py>
- Sindhu, K. K., & Meshram, B. B. (2012). Digital Forensic Investigation Tools and Procedures. *International Journal of Computer Network and Information Security*, 4(4), 39–48. <https://doi.org/10.5815/ijcnis.2012.04.05>
- Singh, A. (2020). *Photo DNA*. Indraprastha Institute of Information Technology.
- Singh, N. M. (2018). Features and Capabilities of Current Image Formats. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.3201632>
- Technologies, N. (2022). *High Efficiency Image File Format (HEIF) Conformance File Candidates*. https://github.com/nokiatech/heif_conformance (Original work published 2016)
- TIFF Revision 6.0 Specification*. (1992). Adobe.
- Tung Nguyen, & Marpe, D. (2012). Performance analysis of HEVC-based intra coding for still image compression. *2012 Picture Coding Symposium*, 233–236. <https://doi.org/10.1109/PCS.2012.6213335>
- UK wants to force encrypted platforms to do CSAM-scanning. (n.d.). *TechCrunch*. Retrieved 3 August 2022, from <https://social.techcrunch.com/2022/07/06/uk-osb-csam-scanning/>
- Wiggins, R. H., Davidson, H. C., Harnsberger, H. R., Lauman, J. R., & Goede, P. A. (2001). Image File Formats: Past, Present, and Future. *RadioGraphics*, 21(3), 789–798. <https://doi.org/10.1148/radiographics.21.3.g01ma25789>

8 Appendices

8.1 Appendix A – Research Proposal

EDINBURGH NAPIER UNIVERSITY SCHOOL OF COMPUTING MSc RESEARCH PROPOSAL

The process of completing and reviewing the contents of this form is intended ensure that the proposed project is viable. It is also intended to increase the chances of a good pass. Much of the material produced while completing this form may be reused in the dissertation itself.

1. Student details

First name	Paul
Last (family) name	Haines
Edinburgh Napier matriculation number	40440891

2. Details of your programme of study

MSc Programme title	MSc Advanced Security and Digital Forensics
Year that you started your diploma modules	2019

3. Project outline details

Please suggest a title for your proposed project. If you have worked with a supervisor on this proposal, please provide the name. You are strongly advised to work with a member of staff when putting your proposal together.

Title of the proposed project	?
Is your project appropriate to your programme of study?	Yes
Name of supervisor	Sean McKeown

4. Brief description of the research area - background

Please do not describe your project in this section. Instead, provide background information in the box below on the broad research area in which your project sits. You should write in narrative (not bullet points). The academic/theoretical basis of your description of the research area should be evident through the use of citations and references. Your description should be between half and one page in length.

Digital cameras are ubiquitous nowadays and are found in various devices from Smartphones to digital surveillance equipment. Digital images are abundant and can be used by forensics experts to help solve crime and prosecute offenders (Ferreira et al., 2020). However, powerful, easy to use editing software along with software tools capable of hiding data within images (steganography) is also freely available. It is important for digital forensics experts to have appropriate tools, techniques, and processes to allow them to analyse digital images and extract salient information.

A number of different image file formats are widely supported by devices and image editing software, with JPEG being the de facto standard (McKeown & Russell, 2020).

Consequently, forensic analysis software widely supports the most common file formats, but most common image file formats date back to the 1980s and 1990s and lack modern features (Bellard, 2016; Heikkilä, 2016).

In 2017 Apple adopted HEIF as the file format of choice for its devices due to its support for modern features such as more efficient compression algorithms and image sequences (Concion & Apple Inc, 2017b). Apple currently have approximately 25% global market with iOS based mobile phones, meaning as many as 1 in 4 forensic investigations involving a mobile phone could involve HEIF files (McKeown & Russell, 2020).

The HEIF file format was created in 2013 and is a complex "Media Container" file format, however it was not widely supported by devices and image editing software until after it was adopted by Apple as the standard in 2017. Since then, whilst general application support for HEIF files has improved, the forensic community appear to have paid it little attention and as a result forensic analysis tool support for the HEIF file format remains limited (McKeown & Russell, 2020).

Forensics can be defined as the "preservation, identification, extraction and documentation of evidence that can be produced in a court of law" (Mani et al., 2022). Digital Forensics is split into multiple domains but involves the forensic analysis of data on computers, networks, mobile phones and other electronic devices (Mani et al., 2022). Digital Image Forensics is a specific domain within the field of Digital Forensics that targets the validation and authenticity of digital images (Mani et al., 2022).

Existing Digital Image Forensics techniques focus on Metadata tampering (eg. modification of data related to GPS location, date / time, device fingerprint etc) held in metadata fields within the image file, and image forgery analysis (eg. alteration of the images themselves, data-hiding using steganographic techniques)

In contrast to most common image file formats, HEIF files are complex container type data-structures (similar in many respects to the TIFF file format created in the early 90s).

Existing Digital Image Forensics techniques are still applicable to the metadata items and individual images held within an HEIF file, however, the complex structure of an HEIF container offers new opportunities for image and metadata forgery, and for data hiding that do not apply to common single-image files. For example, an HEIF file may contain multiple images (eg. image collections and sequences), some of which may derived from preceding images, maybe marked as Auxiliary (eg. depth maps), or may be marked as hidden and therefore not rendered at all by compliant viewers.

Existing Image Forensics tools should be updated or new ones created to allow for these additional components to be mapped, index and extracted for further analysis.

5. Project outline for the work that you propose to complete

Please complete the project outline in the box below. You should use the emboldened text as a framework. Your project outline should be between half and one page in length.

The idea for this research arose from:

I had an initial conversation that with my dissertation supervisor in relation to research around Open Source Intelligence. During this conversation however we discussed other possible research areas. One area of particular interest was around the forensic analysis of HEIF files and the current lack of suitable forensic tools.

The aims of the project are as follows:

Aims:

1. Identify a set of key features that a forensic analysis tool should provide in relation to the analysis of HEIF files.
2. Evaluate existing digital image forensics tools to establish the current level of support in relation to the analysis of HEIF files.
3. Establish a method for indexing, mapping and extracting components of an HEIF file to aid analysis.
4. Create a tool to facilitate the mapping and extraction of components of an HEIF file, perhaps as a stand-alone command-line tool, GUI based application or a plug-in to existing forensics frameworks such as Autopsy.

Objectives:

1. Evaluate capabilities of existing tools (eg. Autopsy), in relation to the forensic analysis of HEIF files.
2. Identify code libraries that may be useful for the forensic analysis of HEIF files.
3. Identify the key features and requirements for Digital Image Forensics Tools in relation to the analysis of HEIF files.
4. Define how the required features map to the structure of HEIF files, and how they can be applied in relation to the deconstruction and analysis of HEIF files.
5. Create a command line tool which includes a set (or subset) of required features capable of deconstructing HEIF files into their component parts.

The main research questions that this work will address include:

- What key features would a tool require for the forensic analysis of HEIF files?
- How might the deconstruction of HEIF files contribute to forensic analysis and answer questions raised during a forensic investigation?
- What are the capabilities of current digital image forensics tools in relation to HEIF files?
- Can the capabilities of existing tools be improved upon?

The software development/design work/other deliverable of the project will be:

- A process for the indexing, mapping and extraction of components of an HEIF file.
- A software tool capable of indexing, mapping and extracting components of an HEIF file.

- The software tool will be validated against the indexing, mapping and extraction process, and in relation to the set of required features.

The project deliverable will be **evaluated as follows:**

- Has a suitable approach / process been identified for the indexing, mapping and extraction of HEIF file components?
- Does the software tool meet the key objectives for the forensic analysis of HEIF files? (eg. does the tool map and extract components of an HEIF file that would be of interest for forensic analysis?)
- Evaluation of the tools performance using existing samples of HEIF images (Nokia, 2021)
- Evaluation of the tools performance using derived images (eg. with hidden data or partial inconsistencies with respect to the HEIF file specification)
- Does the tool meaningfully contribute to Digital Image Forensics with respect to the forensic Analysis of HEIF image files?

The project will involve the following research/field work/experimentation/evaluation:

- Research existing Digital Image Forensic techniques for common image file formats
- Research and Evaluation of existing tools in relation to the forensic analysis of HEIF files
- Research the HEIF file structure to identify components that may be of significance to a forensic investigation (ie. components that may not be covered by existing tools)
- Develop a process to map and extract the contents of an HEIF file
- Creation of a software tool that extracts the components of an HEIF file for forensic analysis.
- Evaluation of the tools performance using existing and derived sample images evaluation

This work will require the use of specialist software:

- Existing forensic analysis tools for evaluation (eg. Autopsy)
- Microsoft C# .NET 6 Visual Studio 2022
- C++ / Python 3

This work will require the use of specialist hardware:

Not applicable

The project is being undertaken in collaboration with:

Not applicable

6. References

Please supply details of all the material that you have referenced in sections 4 and 5 above. You should include at least three references, and these should be to high quality sources such as refereed journal and conference papers, standards or white papers. Please ensure that you use a standardised referencing style for the presentation of your references, e.g. APA, as on myNapier > Your Studies > Improve your Academic & Study Skills > Referencing and Academic Integrity > Referencing Guidelines > SoC Referencing Guidelines.

Bellard, F., 2016. BPG Image format [online]. Available at: <https://bellard.org/bpg/> [Accessed 11 July 2022].

Concion, D., Apple Inc, 2017. High Efficiency Image File Format - WWDC17 - Videos [online]. Apple Developer. Available at: <https://developer.apple.com/videos/play/wwdc2017/513> [Accessed 11 July 2022].

Ferreira, W.D. et al., 2020. A review of digital image forensics. Computers & Electrical Engineering, 85, p.106685. 10.1016/j.compeleceng.2020.106685.

Heikkilä, L., 2016. High Efficiency Image File Format implementation. , p.61.

Mani, R.G. et al., 2022. A Survey on Digital Image Forensics: Metadata and Image forgeries. In: Workshop on Applied Computing. Workshop on Applied Computing. pp. 22–55.

McKeown, S., Russell, G., 2020. Forensic Considerations for the High Efficiency Image File Format (HEIF) [online]. In: 2020 International Conference on Cyber Security and Protection of Digital Services (Cyber Security). 2020 International Conference on Cyber Security and Protection of Digital Services (Cyber Security). Dublin, Ireland: IEEE, pp. 1–8. Available at: <https://ieeexplore.ieee.org/document/9138890/> [Accessed 3 June 2022].

Nokia, 2021. HEIF Example Images - High Efficiency Image File Format [online]. Available at: <https://nokiotech.github.io/heif/examples.html> [Accessed 12 July 2022].

7. Ethics

If your research involves other people, privacy or controversial research there may be ethical issues to consider (please see the information on the module website). If the answer below is YES then you need to complete a research Ethics and Governance Approval form, available on the website:

<https://livenapierac.sharepoint.com/sites/schools/SOC/research/ri/Ethics/SitePages/UG%20&%20MSc%20Students.aspx> .

Does this project have any ethical or governance issues related to working with, studying or observing other people? (YES/NO)	No
---	----

8. Confidentiality

If your research is being done in conjunction with an outside firm or organisation, there may be issues of confidentiality or intellectual property.

Does this project have any issues of confidentiality or intellectual property? (YES/NO)	No
--	----

8.2 Appendix B – Project Management

Project Plan – Weeks 1 to 16

Week Beginning	09/05/2022	16/05/2022	23/05/2022	30/05/2022	06/06/2022	13/06/2022	20/06/2022	27/06/2022	04/07/2022	11/07/2022	18/07/2022	25/07/2022	01/08/2022	08/08/2022	15/08/2022	22/08/2022
Supervisor meetings				c		c		c		c			p			Holiday
Target Milestones																
Registration																
Proposal Target																
Initial Report (full lit review)																
Dissertation Submission																
Viva																
Project Initiation																
Subject Area	c															
Outline Project Plan			c													
Read Proposal Target Examples				c												
Initial Literature Review			c	c	c											
Source Code API Review			c	c	c								p	p		
Examine Heif File structure													p	p		
Tool Investigation & Evaluation				c									p	p		
Reports																
Write Proposal						c	c	c	c							
Literature (Research)									c	c	c	c				
Literature Review												c				
Write Initial Report													p	p		
Dissertation Writeup													p	p		
Tool Development																
Required Tool Features														p	p	
Outline Design														p	p	
Development & Testing																

Project Plan – Weeks 16 to 32

Week Beginning	22/08/2022	29/08/2022	05/09/2022	12/09/2022	19/09/2022	26/09/2022	03/10/2022	10/10/2022	17/10/2022	24/10/2022	31/10/2022	07/11/2022	14/11/2022	21/11/2022	28/11/2022	05/12/2022	12/12/2022
Supervisor meetings	Holiday		p		p		p		p		p		p		p		p
Target Milestones																	
Registration																	
Proposal Target																	
Initial Report (full lit review)																	
Dissertation Submission																	
Viva																	
Project Initiation																	
Subject Area																	
Outline Project Plan																	
Read Proposal Target Examples																	
Initial Literature Review																	
Source Code API Review																	
Examine Heif File structure																	
Tool Investigation & Evaluation																	
Reports																	
Write Proposal																	
Literature (Research)																	
Literature Review																	
Write Initial Report																	
Dissertation Writeup																	
Tool Development																	
Required Tool Features																	
Outline Design																	
Development & Testing																	

8.3 Appendix C – Extract from process_heif.py

```
134 #Entry -----
154 ...
155     log.writeln("Parsing file -----")
156     try:
157         media_file.read(infile,outdir)
158     except Exception as x:
159         log.writeln(str(x))
160         print("ERROR:{0}".format(str(x)))
161
162     log.writeln("Parse Complete -----")
163
164
165     log.writeln("")
166     log.writeln("Processing Binary Data and Hashes -----")
167     try:
168         media_file.ProcessBinaryDataAndHashes()
169     except Exception as x:
170         log.writeln(str(x))
171         print("ERROR:{0}".format(str(x)))
172
173     log.writeln("Binary Data and Hash processing complete -----")
174
175     log.writeln("")
176     log.writeln("Writing map file-----")
177     try:
178         media_file.mapFile()
179     except Exception as x:
180         log.writeln(str(x))
181         print("ERROR:{0}".format(str(x)))
182     log.writeln("map file complete -----")
183
184     log.writeln("")
185     log.writeln("Writing contents file-----")
186     try:
187         media_file.writeall(infile)
188     except Exception as x:
189         log.writeln(str(x))
190         print("ERROR:{0}".format(str(x)))
191     log.writeln("Contents file complete -----")
192
193     log.writeln("Extracting components-----")
194     try:
195         media_file.exportAll()
196     except Exception as x:
197         log.writeln(str(x))
198         print("ERROR:{0}".format(str(x)))
199     log.writeln("Component Extraction complete-----")
200
201     log.writeln("Extracting metadata-----")
202     try:
203         log.writeln("Extracting metadata-----")
204         exportExif(infile,outdir)
205         log.writeln("  Extracting Exif metadata by iloc Item Record -----")
206         exportExifByLocItem(media_file, outdir)
207     except Exception as x:
208         log.writeln(str(x))
209         print("ERROR:{0}".format(str(x)))
210     log.writeln("Metadata extraction complete-----")
211
212     log.writeln("Extracting Images -----")
213     try:
214         exportAllImages(infile)
215     except Exception as x:
216         log.writeln(str(x))
217         print("ERROR:{0}".format(str(x)))
218     log.writeln("Image extraction complete-----")
219
220     log.close()
221
```

8.4 Appendix D – Map file extract

```
File=IMG_3802.HEIC, hash=af17a02ecac189e492d5b5d8263c2a18
000000:ftyp(size=40, start=0, end=40, children=0, hash=31ae6409fc464c9bd19d36eefa1346d5)
000040:meta(size=3697, start=40, end=3737, children=8, hash=63056bbe6e097b27210edd1d5c7a04e6)
000052:-hdlr(size=34, start=52, end=86, children=0, hash=40a701e65ee1aca1288a85aff8a02c6c)
000086:-dinf(size=36, start=86, end=122, children=1, hash=5ca8ac3580f6779b6362b7b1e7cfeb52)
000094:--dref(size=28, start=94, end=122, children=1, hash=ac0eab12278326c08f7cb3e861005e40)
000110:---url (size=12, start=110, end=122, children=0, hash=470b5e0ae161f2b8581d42f6056f0e59)
000122:-pitm(size=14, start=122, end=136, children=0, hash=daaac087c1539cc3d1f333b67549c085)
000136:-iinf(size=1147, start=136, end=1283, children=53, hash=8c67aac8ec693c775b3e4a045887968b)
000150:--infe(size=21, start=150, end=171, children=0, hash=62e16b72e69c095493e87e873534fb42, type=hvc1, id=1 ***HIDDEN***)
000171:--infe(size=21, start=171, end=192, children=0, hash=15b1939cdd5ed6ed5252c9d63a881aa5, type=hvc1, id=2 ***HIDDEN***)
x-----x
001158:--infe(size=21, start=1158, end=1179, children=0, hash=916e34770d6ab6dad6b96fb1ec0ccaed, type=grid, id=49)
001179:--infe(size=21, start=1179, end=1200, children=0, hash=b9c9d9b6d8415b303a2e439dda71fc22, type=hvc1, id=50)
001221:--infe(size=41, start=1221, end=1262, children=0, hash=3a334fec9b6e504cba4e955fb9169aba, type=mime, id=52
***HIDDEN***)
001262:--infe(size=21, start=1262, end=1283, children=0, hash=1bbd875ef5e821c1a8bfe5b22f5bcc54, type=Exif, id=53 ***HIDDEN***)
001283:--iref(size=176, start=1283, end=1459, children=5, hash=36bf6676272f559920f089c6b2cd5733)
001295:--dimg(size=108, start=1295, end=1403, hash=decf9d0730a4fe6fc7fc5c2f5c87c19a, from_id=49, to_ids=[1, 2, 3, 4, 5, 6, 7, 8,
9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44,
45, 46, 47, 48])
001403:--thmb(size=14, start=1403, end=1417, hash=8a467bf1894aced2e919e690063d764f, from_id=50, to_ids=[49])
001417:--auxl(size=14, start=1417, end=1431, hash=751c42311f1a8c027fd0296611472bcf, from_id=51, to_ids=[49])
001431:--cdsc(size=14, start=1431, end=1445, hash=589e15c969067399c7a478b9ee5930a3, from_id=52, to_ids=[51])
001445:--cdsc(size=14, start=1445, end=1459, hash=bf0cab9149f4a7cbaa809b5a35b41362, from_id=53, to_ids=[49])
001459:--iprp(size=1398, start=1459, end=2857, children=2, hash=cfc08cbb60ac39334fdb8cdcf2757453)
001467:--ipco(size=1063, start=1467, end=2530, children=12, hash=1cbd871d2cd5a216ebdf1e6f93bc6206)
001475:---colr(size=548, start=1475, end=2023, children=0, hash=bf957082a9928941076ea1e697b4789e)
002023:--hvcC(size=112, start=2023, end=2135, children=0, hash=9a6215cbc477045d3c81ead83de7a6a3)
002135:---ispe(size=20, start=2135, end=2155, children=0, hash=079b509faf51816c056bde8bb5c68521)
002155:---ispe(size=20, start=2155, end=2175, children=0, hash=58e2450f1210018508b97da60b61d0f1)
002175:---irof(size=9, start=2175, end=2184, children=0, hash=c3f5ac503ef23b393906783a7f098850)
002184:---pidx(size=16, start=2184, end=2200, children=0, hash=19d227947c9a98cf865960bf9be7cdd4)
002200:---hvcC(size=111, start=2200, end=2311, children=0, hash=10698b6aaa2f6ad2f5f4a05c6a0a70ec)
002311:---ispe(size=20, start=2311, end=2331, children=0, hash=2fc06a2c5c8f2779c902ddd24030ba70)
002331:---hvcC(size=113, start=2331, end=2444, children=0, hash=876f7feadda8db19a5eec60eeaf272bf)
002444:---ispe(size=20, start=2444, end=2464, children=0, hash=f22a7a120531113f610395e50a696539)
002464:---pidx(size=14, start=2464, end=2478, children=0, hash=833776b11b67cfac49fe2f9d440b5b41)
002478:---auxC(size=52, start=2478, end=2530, children=0, hash=f02cb8dcdb6e5cd34a1d63881220ef9b)
002530:--ipma(size=327, start=2530, end=2857, children=0, hash=7a272e18347cb085cd743d28afa5e150)
002857:--idat(size=16, start=2857, end=2873, children=0, hash=75148a54bcca9356340adbcac7859c94)
002873:--iloc(size=864, start=2873, end=3737, items=53, hash=4b4fc1c14cb62f7d65a21f5be424f32c)
002889:- Item=1, (size=16, start=2889, end=2905, extents=1, id=1)
002897:- Extent=1 (size=8, start=2897, end=2905, data=(type=mdat, length=25942, start=34679, end=60621,
hash=f4444657f8e81e3781048768c1e4d85c)
002905:- Item=2, (size=16, start=2905, end=2921, extents=1, id=2)
002913:- Extent=1 (size=8, start=2913, end=2921, data=(type=mdat, length=23369, start=60621, end=83990,
hash=7d71032590e4df713e97a99faae03993)
x-----x
003657:- Item=49, (size=16, start=3657, end=3673, extents=1, id=49)
003665:- Extent=1 (size=8, start=3665, end=3673, data=(type=idat, length=8, start=2865, end=2873,
hash=ca1cdf640a864a2a852b4007f92b2f50)
003673:- Item=50, (size=16, start=3673, end=3689, extents=1, id=50)
003681:- Extent=1 (size=8, start=3681, end=3689, data=(type=mdat, length=27843, start=3753, end=31596,
hash=38b31152e9f297dbb41a3ad08fd54072)
003705:- Item=52, (size=16, start=3705, end=3721, extents=1, id=52)
003713:- Extent=1 (size=8, start=3713, end=3721, data=(type=mdat, length=363, start=31596, end=31959,
hash=be06b05e83091735fb2ca4b355133192)
003721:- Item=53, (size=16, start=3721, end=3737, extents=1, id=53)
003729:- Extent=1 (size=8, start=3729, end=3737, data=(type=mdat, length=2720, start=31959, end=34679,
hash=d4285ce80e394f1c5149ee25213aa296)
003737:mdat(size=1158355, start=3737, end=1162092, children=0, hash=d3c804e8a531f37d1840e915f9fd3a0)
```

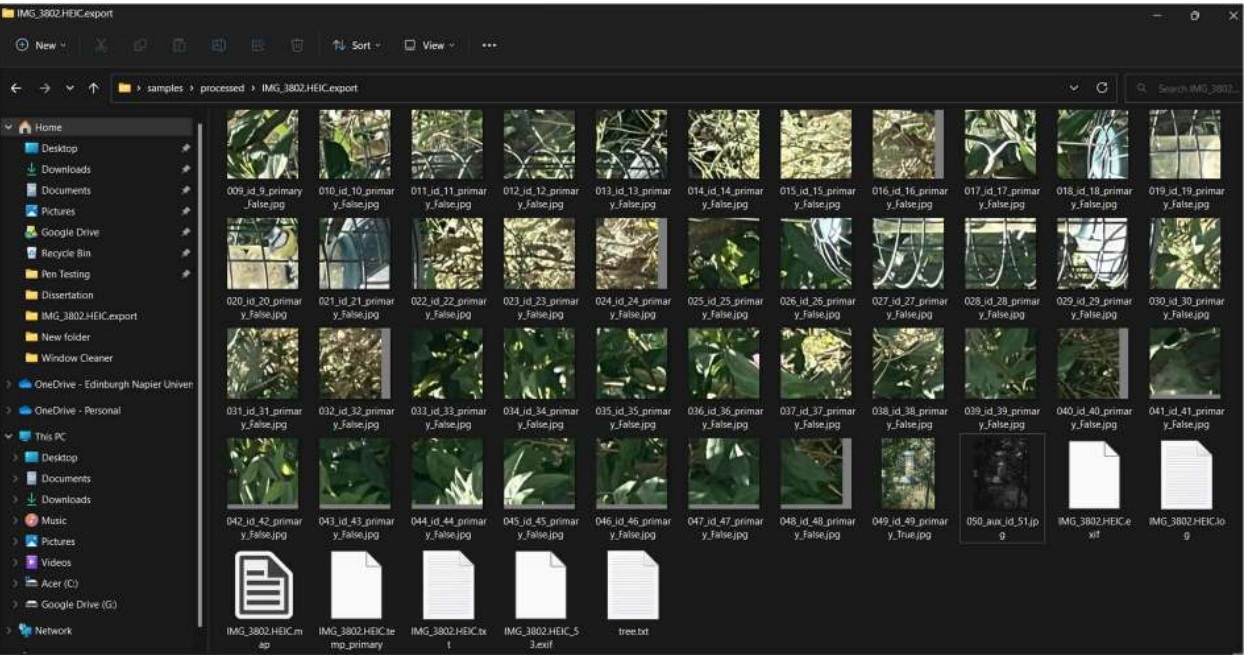
8.5 Appendix E – Contents file extract

<pre> Box Type=ftyp Size=40 StartByte=0 Children=0 Hash=31ae6409fc464c9bd19d36eefa1346d5 major_brand=heic minor_version=0 Compatible Brands(6): compatible_brand=mif1 compatible_brand=MiHE compatible_brand=MiPr compatible_brand=miaf compatible_brand=MiHB compatible_brand=heic Box Type=meta Size=3697 StartByte=40 Children=8 Hash=63056bbe6e097b27210edd1d5c7a04e6 Version=0 Flags=0 Box Type=hd1r Size=34 StartByte=52 Children=0 Hash=40a701e65ee1aca1288a85aff8a02c6c Version=0 Flags=0 pre_defined=0 handler_type=pict reserved=[0, 0, 0] name= Box Type=dinf Size=36 StartByte=86 Children=1 Hash=5ca8ac3580f6779b6362b7b1e7cfeb52 Box Type=dref Size=28 StartByte=94 Children=1 Hash=ac0eab12278326c08f7cb3e861005e40 Version=0 Flags=0 Entries=1 Entry=001: Box Type=url Size=12 StartByte=110 Children=0 Hash=470b5e0ae161f2b8581d42f6056f0e59 Version=0 Flags=1 Location=None Box Type=url Size=12 StartByte=110 Children=0 Hash=470b5e0ae161f2b8581d42f6056f0e59 Version=0 Flags=1 Location=None Box Type=pitm Size=14 StartByte=122 Children=0 Hash=daaac087c1539cc3d1f333b67549c085 Version=0 Flags=0 item_id=49 Box Type=iinf Size=1147 StartByte=136 Children=53 Hash=8c67aac8ec693c775b3e4a045887968b Version=0 Flags=0 Item Info Entries=53 Box Type=infe Size=21 StartByte=150 Children=0 item_name= item_extension=None item_type=mime content_type=application/rdf+xml content_encoding=None uri_type=None Box Type=infe </pre>	<pre> Hash=62e16b72e69c095493e87e873534fb42 Version=2 Flags=1 *** HIDDEN *** item_id=1 item_protection_index=0 item_name= item_extension=None item_type=hvc1 content_type=None content_encoding=None uri_type=None Box Type=infe Size=21 StartByte=171 Children=0 Hash=15b1939cdd5ed6ed5252c9d63a881aa5 Version=2 Flags=1 *** HIDDEN *** item_id=2 item_protection_index=0 item_name= item_extension=None item_type=hvc1 content_type=None content_encoding=None uri_type=None x-----x Box Type=infe Size=21 StartByte=1158 Children=0 Hash=916e34770d6ab6dadb96fbb1ec0ccaed Version=2 Flags=0 item_id=49 item_protection_index=0 item_name= item_extension=None item_type=grid content_type=None content_encoding=None uri_type=None Box Type=infe Size=21 StartByte=1179 Children=0 Hash=b9c9d9b6d8415b303a2e439dda71fc22 Version=2 Flags=0 item_id=50 item_protection_index=0 item_name= item_extension=None item_type=hvc1 content_type=None content_encoding=None uri_type=None Box Type=infe Size=21 StartByte=1200 Children=0 Hash=641148e477c89943086301ee94479d49 Version=2 Flags=1 *** HIDDEN *** item_id=51 item_protection_index=0 item_name= item_extension=None item_type=hvc1 content_type=None content_encoding=None uri_type=None Box Type=infe Size=41 StartByte=1221 Children=0 Hash=3a334fec9b6e504cba4e955fb9169aba Version=2 Flags=1 *** HIDDEN *** item_id=52 item_protection_index=0 to_item_ids=[49] Box Type=cdsc Size=14 StartByte=1431 Children=0 Hash=589e15c969067399c7a478b9ee5930a3 from_item_id=52 </pre>
---	--

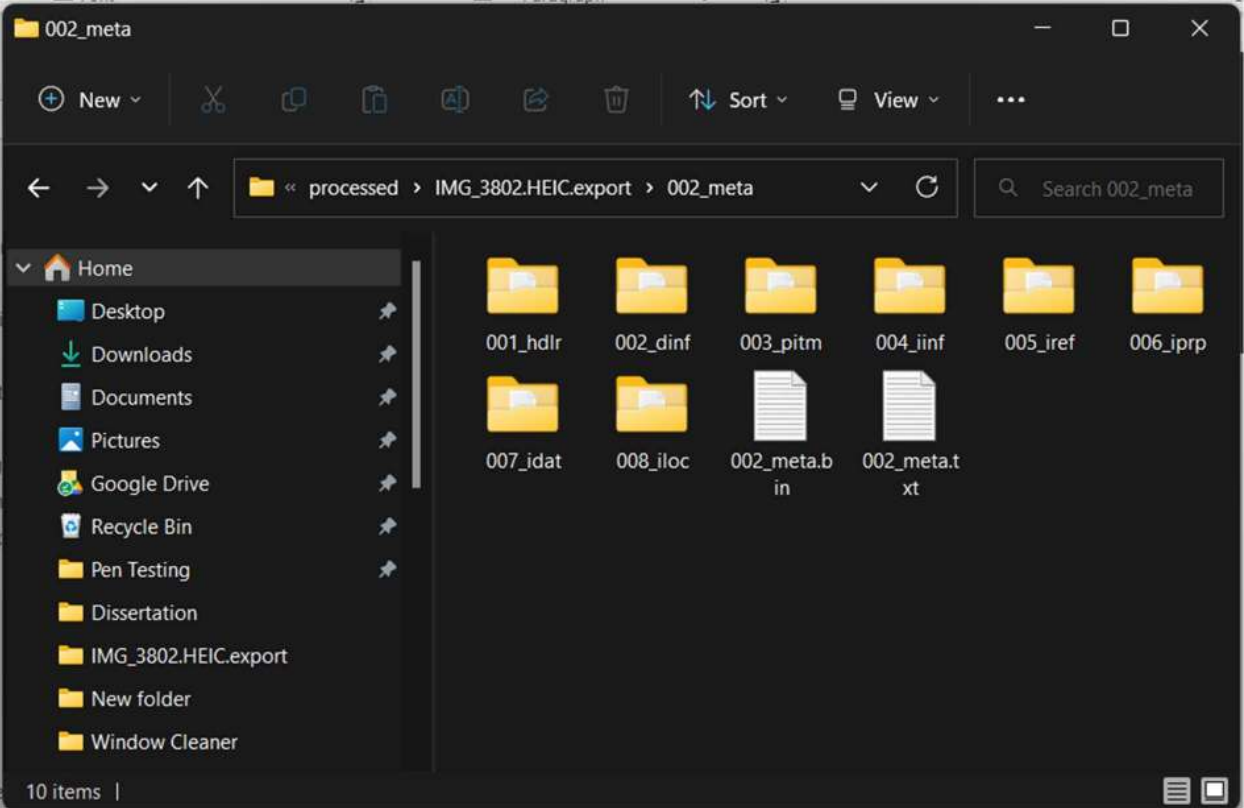
<pre> Size=21 StartByte=1262 Children=0 Hash=1bbd875ef5e821c1a8bfe5b22f5bcc54 Version=2 Flags=1 *** HIDDEN *** item_id=53 item_protection_index=0 item_name= item_extension=None item_type=Exif content_type=None content_encoding=None uri_type=None Box Type=iref Size=176 StartByte=1283 Children=5 Hash=36bf6676272f559920f089c6b2cd5733 Version=0 Flags=0 references=5 reference=001 Box Type=dimg Size=108 StartByte=1295 Children=0 Hash=decf9d0730a4fe6fc7fc5c2f5c87c19a from_item_id=49 to_item_ids=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48] reference=002 Box Type=thmb Size=14 StartByte=1403 Children=0 Hash=8a467bf1894aced2e919e690063d764f from_item_id=50 to_item_ids=[49] reference=003 Box Type=aux1 Size=14 StartByte=1417 Children=0 Hash=751c42311f1a8c027fd0296611472bcf from_item_id=51 to_item_ids=[49] reference=004 Box Type=cpsc Size=14 StartByte=1431 Children=0 Hash=589e15c969067399c7a478b9ee5930a3 from_item_id=52 to_item_ids=[51] reference=005 Box Type=cpsc Size=14 StartByte=1445 Children=0 Hash=bf0cab9149f4a7cbaa809b5a35b41362 from_item_id=53 to_item_ids=[49] Box Type=aux1 Size=14 StartByte=1417 Children=0 Hash=751c42311f1a8c027fd0296611472bcf from_item_id=51 general_tier_flag=0 general_profile_idc=3 general_profile_compat_flags=1879048192 general_const_indicator_flags=193514046488576 reserved1=15 min_spatial_segmentation_idc=0 reserved2=63 parallelism_type=0 reserved3=63 chroma_format=1 bit_depth_luma_minus_8=0 bit_depth_chroma_minus_8=0 avg_frame_rate=0 constant_frame_rate=0 num_temporal_layers=1 temporal_id_nested=0 length_size_minus_1=3 num_of_arrays=None Box Type=ispe Size=20 StartByte=2311 Children=0 Hash=2fc06a2c5c8f2779c902ddd24030ba70 Version=0 Flags=0 width=320 </pre>	<pre> to_item_ids=[51] Box Type=cpsc Size=14 StartByte=1445 Children=0 Hash=bf0cab9149f4a7cbaa809b5a35b41362 from_item_id=53 to_item_ids=[49] Box Type=iprp Size=1398 StartByte=1459 Children=2 Hash=cfc08cbb60ac39334fdb8cdc72757453 Box Type=ipco Size=1063 StartByte=1467 Children=12 Hash=1cbd871d2cd5a216ebdf1e6f93bc6206 Box Type=colr Size=548 StartByte=1475 Children=0 Hash=bf957082a9928941076ea1e6f9b4789e Box Type=hvcC Size=112 StartByte=2023 Children=0 Hash=9a6215cbc477045d3c81ead83de7a6a3 configuration_version=1 general_profile_space=0 general_tier_flag=0 general_profile_idc=3 general_profile_compat_flags=1879048192 general_const_indicator_flags=193514046488576 reserved1=15 min_spatial_segmentation_idc=0 reserved2=63 parallelism_type=0 reserved3=63 chroma_format=1 bit_depth_luma_minus_8=0 bit_depth_chroma_minus_8=0 avg_frame_rate=0 constant_frame_rate=0 num_temporal_layers=1 temporal_id_nested=0 length_size_minus_1=3 num_of_arrays=None Box Type=ispe Size=20 StartByte=2135 Children=0 Hash=079b509faf51816c056bde8bb5c68521 Version=0 Flags=0 width=512 height=512 Box Type=ispe Size=20 StartByte=2155 Children=0 Hash=58e2450f1210018508b97da60b61d0f1 Version=0 Flags=0 width=4032 height=3024 Box Type=irotp Size=9 StartByte=2175 Children=0 Hash=c3f5ac503ef23b393906783a7f098850 Box Type=pixi Size=16 StartByte=2184 Children=0 Hash=19d227947c9a98cf865960bf9be7cdd4 Box Type=hvcC Size=111 StartByte=2200 Children=0 Hash=10698b6aaa2f6ad2f5f4a05c6a0a70ec configuration_version=1 general_profile_space=0 x-----x item=51, id=51 association=1 essential=1 property_index=9 association=2 essential=0 property_index=10 </pre>
--	--

8.6 Appendix F – HEIF File Output Directory (root)

Root directory of the exported HEIF file structure, showing each image extracted, the map file, contents file (IMG_3802_HEIC.txt), Exif data, and log files. A directory is shown for each root-level box:

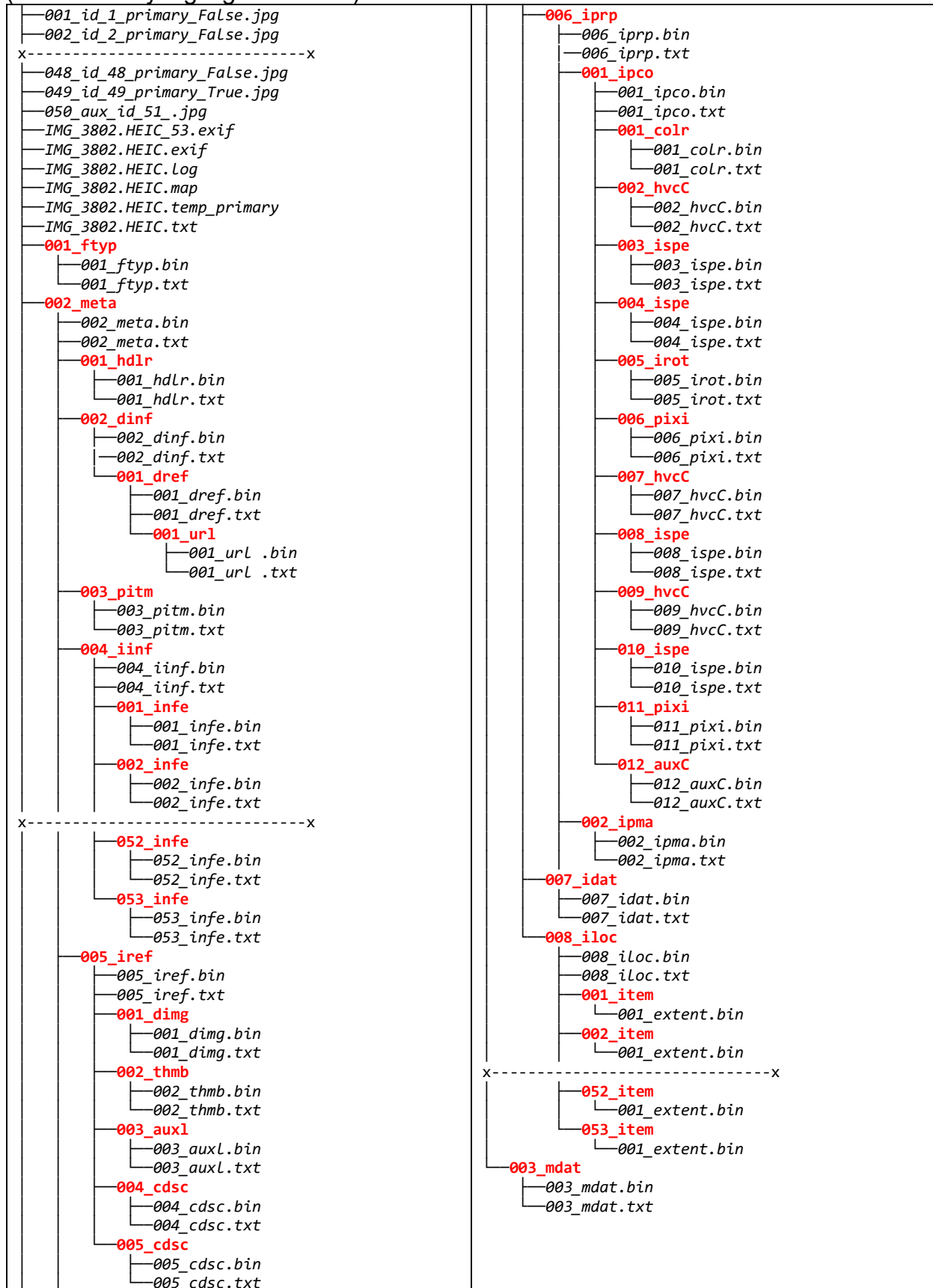


Contents of the Meta box directory showing each child-box directory along with the 002_meta.txt details file and 002_meta.bin binary box contents file:



8.7 Appendix G – HEIF File Directory Structure

Files and folder structure output after processing IMG_3802.HEIC
(Box directory highlighted in red)



8.8 Appendix G – Exif Data Extract

Image Make=Apple	MakerNote Tag 0x0001=14
Image Model=iPhone 13 Pro	MakerNote Tag 0x0002=[173, 0, 203, 0, 152, 1, 34, 1, 198, 0, 206, 0, 96, 0, 204, 0, 16, 1, 253, 0, ...]
Image Orientation=Rotated 90 CW	MakerNote Tag 0x0003=[6, 7, 8, 85, 102, 108, 97, 103, 115, 85, 118, 97, 108, 117, 101, 89, 116, 105, 109, 101, ...]
Image XResolution=72	MakerNote Tag 0x0004=1
Image YResolution=72	MakerNote Tag 0x0005=169
Image ResolutionUnit=Pixels/Inch	MakerNote Tag 0x0006=181
Image Software=15.6	MakerNote Tag 0x0007=1
Image DateTime=2022:08:06 15:23:05	MakerNote Tag 0x0008=[-1691615231/1973223423, -27835/63, 128/257]
Image HostComputer=iPhone 13 Pro	MakerNote Tag 0x000C=[16793912/960837685, 1144597814/1160987437]
Image ExifOffset=222	MakerNote Tag 0x000D=18
GPS GPSLatitudeRef=N	MakerNote Tag 0x000E=0
GPS GPSLatitude=[5x, 3x, 134x/50]	MakerNote Tag 0x0010=1
GPS GPSLongitudeRef=W	MakerNote Tag 0x0011=4DEF-8377-DF798D2626C8
GPS GPSLongitude=[1x, 4x, 168x/50]	MakerNote Tag 0x0014=12
GPS GPSPAltitudeRef=0	MakerNote Tag 0x0017=-1034936320
GPS GPSPAltitude=1832485/17444	MakerNote Tag 0x0019=139264
GPS GPSSpeedRef=K	MakerNote Tag 0x001A=-796D-
GPS GPSSpeed=0	MakerNote Tag 0x001F=0
GPS GPSImgDirectionRef=T	MakerNote Tag 0x0020=4C5B-95F7-7CE21E53CAE3
GPS GPSImgDirection=394721/1467	MakerNote Tag 0x0021=2621441/1849884672
GPS GPSPDestBearingRef=T	MakerNote Tag 0x0023=[150011904, 1724318051]
GPS GPSPDestBearing=394721/1467	MakerNote Tag 0x0025=4238
GPS GPSDate=2022:08:06	MakerNote Tag 0x0026=3
GPS Tag 0x001F=28720/5699	MakerNote Tag 0x0027=-53508044/151570847
Image GPSInfo=2436	MakerNote Tag 0x002D=5271
EXIF ExposureTime=1/131	MakerNote Tag 0x002E=1
EXIF FNumber=14/5	MakerNote Tag 0x002F=73
EXIF ExposureProgram=Program Normal	MakerNote Tag 0x0030=808506369/33752069
EXIF ISOSpeedRatings=32	MakerNote Tag 0x0033=12288
EXIF ExifVersion=0232	MakerNote Tag 0x0034=4
EXIF DateTimeOriginal=2022:08:06 15:23:05	MakerNote Tag 0x0035=3
EXIF DateTimeDigitized=2022:08:06 15:23:05	MakerNote Tag 0x0036=11648
EXIF OffsetTime=+01:00	MakerNote Tag 0x0037=4
EXIF OffsetTimeOriginal=+01:00	MakerNote Tag 0x0038=1494
EXIF OffsetTimeDigitized=+01:00	MakerNote Tag 0x0039=0
EXIF ShutterSpeedValue=32793/4661	MakerNote Tag 0x003A=0
EXIF ApertureValue=77364/26041	MakerNote Tag 0x003B=0
EXIF BrightnessValue=39893/6001	MakerNote Tag 0x003C=4
EXIF ExposureBiasValue=0	MakerNote Tag 0x003D=0
EXIF MeteringMode=Pattern	MakerNote Tag 0x0040=[6, 6, 7, 81, 51, 81, 49, 81, 50, 81, 48, 16, 0, 34, 0, 0, 0, 0, 16, 1, ...]
EXIF Flash=Flash did not fire, compulsory flash mode	MakerNote Tag 0x0041=[0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 9, 98, 112, 108, 105, 115, 116, 48, 48, 8, 8, 0, 0, 0, 0]
EXIF FocalLength=9	MakerNote Tag 0x0042=[0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 9, 0, 0, 0, 21, 0, 0, 0, 4, 0, 4, 144, 245, 0, 2]
EXIF SubjectArea=[5530, 3987, 1130, 675]	MakerNote Tag 0x0043=215637
EXIF MakerNotes=[65, 112, 112, 108, 101, 32, 105, 79, 83, 0, 0, 1, 77, 77, 0, 49, 0, 1, 0, 9, ...]	MakerNote Tag 0x0044=0
EXIF SubSecTimeOriginal=645	MakerNote Tag 0x0045=1
EXIF SubSecTimeDigitized=645	MakerNote Tag 0x0046=0
EXIF ColorSpace=Uncalibrated	MakerNote Tag 0x0048=7
EXIF ExifImageWidth=4032	MakerNote Tag 0x0049=57
EXIF ExifImageLength=3024	MakerNote Tag 0x004A=3
EXIF SensingMethod=One-chip color area	
EXIF SceneType=Directly Photographed	
EXIF ExposureMode=Auto Exposure	
EXIF WhiteBalance=Auto	
EXIF DigitalZoomRatio=21/4	
EXIF FocalLengthIn35mmFilm=406	
EXIF LensSpecification=[299253/190607, 9, 3/2, 14/5]	
EXIF LensMake=Apple	
EXIF LensModel=iPhone 13 Pro back triple camera	
9mm f/2.8	
EXIF Tag 0xA460=2	

8.9 Appendix H – Batch Process

The following “run_process_heif.sh” script was used to test the batch processing capabilities of the tool:

```
python3 HeifER/process_heif.py ~/samples/IMG_3802.HEIC -outdir ~/samples/processed/IMG_3802.HEIC.export
python3 HeifER/process_heif.py ~/samples/IMG_3802_test.HEIC -outdir ~/samples/processed/IMG_3802_test.HEIC.export
python3 HeifER/process_heif.py ~/samples/IMG_3802_test1.HEIC -outdir ~/samples/processed/IMG_3802_test1.HEIC.export
python3 HeifER/process_heif.py ~/samples/IMG_3802_test2.HEIC -outdir ~/samples/processed/IMG_3802_test2.HEIC.export
python3 HeifER/process_heif.py ~/samples/IMG_3802_test3.HEIC -outdir ~/samples/processed/IMG_3802_test3.HEIC.export
python3 HeifER/process_heif.py ~/samples/IMG_3802_test4.HEIC -outdir ~/samples/processed/IMG_3802_test4.HEIC.export
python3 HeifER/process_heif.py ~/samples/IMG_3802_test5.HEIC -outdir ~/samples/processed/IMG_3802_test5.HEIC.export
python3 HeifER/process_heif.py ~/samples/alpha_1440x960.heic -outdir ~/samples/processed/alpha_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/autumn_1440x960.heic -outdir ~/samples/processed/autumn_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/bird_burst.heic -outdir ~/samples/processed/bird_burst.heic.export
python3 HeifER/process_heif.py ~/samples/bothie_1440x960.heic -outdir ~/samples/processed/bothie_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/C001.heic -outdir ~/samples/processed/C001.heic.export
python3 HeifER/process_heif.py ~/samples/cheers_1440x960.heic -outdir ~/samples/processed/cheers_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/crowd_1440x960.heic -outdir ~/samples/processed/crowd_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/grid_960x640.heic -outdir ~/samples/processed/grid_960x640.heic.export
python3 HeifER/process_heif.py ~/samples/lights_1440x960.heic -outdir ~/samples/processed/lights_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/old_bridge_1440x960.heic -outdir ~/samples/processed/old_bridge_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/overlay_1000x680.heic -outdir ~/samples/processed/overlay_1000x680.heic.export
python3 HeifER/process_heif.py ~/samples/rally_burst.heic -outdir ~/samples/processed/rally_burst.heic.export
python3 HeifER/process_heif.py ~/samples/random_collection_1440x960.heic -outdir
~/samples/processed/random_collection_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/sea1_animation.heic -outdir ~/samples/processed/sea1_animation.heic.export
python3 HeifER/process_heif.py ~/samples/season_collection_1440x960.heic -outdir
~/samples/processed/season_collection_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/ski_jump_1440x960.heic -outdir ~/samples/processed/ski_jump_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/spring_1440x960.heic -outdir ~/samples/processed/spring_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/starfield_animation.heic -outdir ~/samples/processed/starfield_animation.heic.export
python3 HeifER/process_heif.py ~/samples/stereo_1200x800.heic -outdir ~/samples/processed/stereo_1200x800.heic.export
python3 HeifER/process_heif.py ~/samples/summer_1440x960.heic -outdir ~/samples/processed/summer_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/surfer_1440x960.heic -outdir ~/samples/processed/surfer_1440x960.heic.export
python3 HeifER/process_heif.py ~/samples/winter_1440x960.heic -outdir ~/samples/processed/winter_1440x960.heic.export
```

8.10 Appendix I – ItemInfoEntry Box Hidden Flag

First bit of Byte 12 in the ItemInfoEntry Box Header is the hidden flag .

Box Size	Box Type	Version	Flags	Rest of Box
4 Bytes	4 Bytes	1 Byte	3 Bytes	data...

```
aligned(8) class Box (unsigned int(32) boxtype, optional unsigned int(8)[16] extended_type)
{
    unsigned int(32) size;
    unsigned int(32) type = boxtype;
    if (size==1)
    {
        unsigned int(64) largesize;
    } else if (size==0)
    {
        // box extends to end of file
    }
    if (boxtype=='uuid')
    {
        unsigned int(8)[16] usertype = extended_type;
    }
}

aligned(8) class FullBox(unsigned int(32) boxtype, unsigned int(8) v, bit(24) f) extends
Box(boxtype)
{
    unsigned int(8) version = v;
    bit(24) flags = f;
}

aligned(8) class ItemInfoEntry extends FullBox('infe', version, 0)
{
    if ((version == 0) || (version == 1))
    {
        unsigned int(16) item_ID;
        unsigned int(16) item_protection_index;
        string item_name; string content_type;
        string content_encoding; //optional
    }
    if (version == 1)
    {
        unsigned int(32) extension_type; //optional
        ItemInfoExtension(extension_type); //optional
    }
    if (version >= 2)
    {
        if (version == 2)
        {
            unsigned int(16) item_ID;
        } else if (version == 3)
        {
            unsigned int(32) item_ID;
        }
        unsigned int(16) item_protection_index;
        unsigned int(32) item_type;
        string item_name;
        if (item_type=='mime')
        {
            string content_type;
            string content_encoding; //optional
        } else if (item_type == 'uri ')
        {
            string item_uri_type;
        }
    }
}
```

8.11 Appendix J – HeifEditor Script Exceptrts

Test Case D Swap infe box entries	<pre>#Alter order of infe records iinfboxes = file1.find_boxes("iinf") temp = iinfboxes[0].children[0] iinfboxes[0].children[0] = iinfboxes[0].children[1] iinfboxes[0].children[1] = temp file1.save("/home/kali/samples/IMG_3802_test1.HEIC")</pre>	
Test Case E Additional Hidden Image	<pre>#add another tile object #copy image tile 47 and add as image item 54 ----- metaBox = file1.find_meta_box() iinfBox = file1.find_iinf_box() ilocBox = file1.find_iloc_box() infeBox47 = file1.find_infe_box(id=47) ilocEntry47 = file1.find_iloc_item(id=47) ipmaEntry47 = file1.find_ipma_item(id=47) newinfebox = copy.deepcopy(infeBox47) newilocitem = copy.deepcopy(ilocEntry47) newipmaitem = copy.deepcopy(ipmaEntry47) file1.set_infe_box_id(newinfebox, 54) file1.set_iloc_item_id(newilocitem, 54) file1.set_ipma_item_id(newipmaitem, 54) file1.add_infe_box(newinfebox) file1.add_iloc_item(newilocitem) file1.add_ipma_item(newipmaitem) endbyte = file1.rebase() file1.save("/home/kali/samples/IMG_3802_test2.HEIC")</pre>	
Test Case F Additional Exif Data	<pre>#add another exif record ----- infeBox53 = file1.find_infe_box(id=53) ilocEntry53 = file1.find_iloc_item(id=53) itemrefbox53 = file1.find_iref_item_box(4) newinfebox = copy.deepcopy(infeBox53) newilocitem = copy.deepcopy(ilocEntry53) itemrefbox = copy.deepcopy(itemrefbox53) file1.set_infe_box_id(newinfebox, 55) file1.set_iloc_item_id(newilocitem, 55) itemrefbox.from_item_ID = 55 itemrefbox.references[0] = 54 file1.add_iref_item_box(itemrefbox) file1.add_infe_box(newinfebox) file1.add_iloc_item(newilocitem) newdataoffset = file1.rebase() mdatBox = file1.find_mdat_box(0) mdatoffset = mdatBox.startByte mdatdataoffset = mdatoffset + 12 dataoffset = newilocitem['extents'][0]['extent_offset'] datalength = newilocitem['extents'][0]['extent_length'] copystart = dataoffset - mdatdataoffset copyend = copystart + datalength copydata = mdatBox.binarydata[copystart:copyend] x = re.compile(b'.*Apple\x00') pat = x.match(copydata) end = pat.regs[0][1] print(copydata[end-6:end]) #alter exif data newdata = copydata[:end-6] + b"*Fake*" + copydata[end:] mdatBox.binarydata += newdata file1.rebase() newilocitem['extents'][0]['extent_offset'] = newdataoffset file1.rebase() file1.save("/home/kali/samples/IMG_3802_test3.HEIC")</pre>	

Test Case G Additional unrelated content	<pre># insert a jpeg ----- #copy image tile 47 and add as image item 56 and repurpose for jpeg infeBox47 = file1.find_infe_box(id=47) ilocEntry47 = file1.find_iloc_item(id=47) ipmaEntry47 = file1.find_ipma_item(id=47) newinfebox2 = copy.deepcopy(infeBox47) newilocitem2 = copy.deepcopy(ilocEntry47) newipmaitem2 = copy.deepcopy(ipmaEntry47) file1.set_infe_box_id(newinfebox2,56) file1.set_iloc_item_id(newilocitem2,56) file1.set_ipma_item_id(newipmaitem2, 56) newinfebox2.item_type = 'jpeg' file1.add_infe_box(newinfebox2) file1.add_iloc_item(newilocitem2) file1.add_ipma_item(newipmaitem2) newdataoffset2 = file1.rebase() #load jpeg data jpegfile = open("/home/kali/samples/Anonymous.jpg","rb") jpegfiledata = jpegfile.read() print("mdat start={0}, length={1}".format(mdatBox.startByte,len(mdatBox.binarydata))) #append jpeg data to end of mdat box mdatBox.binarydata += jpegfiledata file1.rebase() print("jpeg start={0}, length={1}".format(newdataoffset2, len(jpegfiledata))) print("mdat start={0}, length={1}".format(mdatBox.startByte, len(mdatBox.binarydata))) #set the data offset for the jpeg location record newilocitem2['extents'][0]['extent_offset'] = newdataoffset2 newilocitem2['extents'][0]['extent_length'] = len(jpegfiledata) file1.save("/home/kali/samples/IMG_3802_test4.HEIC")</pre>
Test Case H External content reference	<pre>#Add external URL reference ----- urlboxes = file1.find_boxes('url ') #clear internal content flag : NB: pyheif fails to load the file if this flag is not clear (0=external, 1=internal) urlboxes[0].flags=0 urlboxes[0].binarydata = b'https://www.pawqualityproducts.co.uk/\x00' #rebase endbyte = file1.rebase() file1.save("/home/kali/samples/IMG_3802_test5.HEIC")</pre>

8.12 Appendix K – ExifTool output comparison

ExifTool Version Number	: 12.50	ExifTool Version Number	: 12.50
File Name	: IMG_3802.HEIC	File Name	: IMG_3802_test3.HEIC
Directory	: .	Directory	: .
File Size	: 1162 kB	File Size	: 1165 kB
File Modification Date/Time	: 2022:08:06 22:36:09+01:00	File Modification Date/Time	: 2022:10:23 16:47:04+01:00
File Access Date/Time	: 2022:11:18 18:10:48+00:00	File Access Date/Time	: 2022:11:18 18:12:17+00:00
File Creation Date/Time	: 2022:08:06 15:34:22+01:00	File Creation Date/Time	: 2022:10:23 17:12:10+01:00
File Permissions	: -rw-rw-rw-	File Permissions	: -rw-rw-rw-
File Type	: HEIC	File Type	: HEIC
File Type Extension	: heic	File Type Extension	: heic
MIME Type	: image/heic	MIME Type	: image/heic
Major Brand	: High Efficiency	Major Brand	: High Efficiency
Image Format HEVC still image (.HEIC)		Image Format HEVC still image (.HEIC)	
Minor Version	: 0.0.0	Minor Version	: 0.0.0
Compatible Brands	: mif1, MiHE, MiPr, mifaf, MiHB, heic	Compatible Brands	: mif1, MiHE, MiPr, mifaf, MiHB, heic
Handler Type	: Picture	Handler Type	: Picture
Primary Item Reference	: 49	Primary Item Reference	: 49
Meta Image Size	: 4032x3024	Meta Image Size	: 4032x3024
XMP Toolkit	: XMP Core 6.0.0	XMP Toolkit	: XMP Core 6.0.0
HDR Gain Map Version	: 65536	HDR Gain Map Version	: 65536
Exif Byte Order	: Big-endian (Motorola, MM)	Exif Byte Order	: Big-endian (Motorola, MM)
Make	: Apple	Make	: Apple
Camera Model Name	: iPhone 13 Pro	Camera Model Name	: iPhone 13 Pro
Orientation	: Rotate 90 CW	Orientation	: Rotate 90 CW
X Resolution	: 72	X Resolution	: 72
Y Resolution	: 72	Y Resolution	: 72
Resolution Unit	: inches	Resolution Unit	: inches
Software	: 15.6	Software	: 15.6
Modify Date	: 2022:08:06 15:23:05	Modify Date	: 2022:08:06 15:23:05
Host Computer	: iPhone 13 Pro	Host Computer	: iPhone 13 Pro
Exposure Time	: 1/131	Exposure Time	: 1/131
F Number	: 2.8	F Number	: 2.8
Exposure Program	: Program AE	Exposure Program	: Program AE
ISO	: 32	ISO	: 32
Exif Version	: 0232	Exif Version	: 0232
Date/Time Original	: 2022:08:06 15:23:05	Date/Time Original	: 2022:08:06 15:23:05
Create Date	: 2022:08:06 15:23:05	Create Date	: 2022:08:06 15:23:05
Offset Time	: +01:00	Offset Time	: +01:00
Offset Time Original	: +01:00	Offset Time Original	: +01:00
Offset Time Digitized	: +01:00	Offset Time Digitized	: +01:00
Shutter Speed Value	: 1/131	Shutter Speed Value	: 1/131
Aperture Value	: 2.8	Aperture Value	: 2.8
Brightness Value	: 6.647725379	Brightness Value	: 6.647725379
Exposure Compensation	: 0	Exposure Compensation	: 0
Metering Mode	: Multi-segment	Metering Mode	: Multi-segment
Flash	: Off, Did not fire	Flash	: Off, Did not fire
Focal Length	: 9.0 mm	Focal Length	: 9.0 mm
Subject Area	: 5530 3987 1130 675	Subject Area	: 5530 3987 1130 675
Run Time Flags	: Valid	Run Time Flags	: Valid
Run Time Value	: 450194667493083	Run Time Value	: 450194667493083
Run Time Scale	: 1000000000	Run Time Scale	: 1000000000
Run Time Epoch	: 0	Run Time Epoch	: 0
Acceleration Vector	: -0.008853157982 -0.9546317462 -0.2916493118	Acceleration Vector	: -0.008853157982 -0.9546317462 -0.2916493118
Focus Distance Range	: 0.49 - 1.00 m	Focus Distance Range	: 0.49 - 1.00 m
Media Group UUID	: A89E85D9-6E3C-4DEF-8377-DF798D2626C8	Media Group UUID	: A89E85D9-6E3C-4DEF-8377-DF798D2626C8
Live Photo Video Index	: -1034936320	Live Photo Video Index	: -1034936320
Sub Sec Time Original	: 645	Sub Sec Time Original	: 645
Sub Sec Time Digitized	: 645	Sub Sec Time Digitized	: 645
Color Space	: Uncalibrated	Color Space	: Uncalibrated
Exif Image Width	: 4032	Exif Image Width	: 4032
Exif Image Height	: 3024	Exif Image Height	: 3024

Sensing Method	: One-chip color area	Sensing Method	: One-chip color area
Scene Type	: Directly photographed	Scene Type	: Directly photographed
Exposure Mode	: Auto	Exposure Mode	: Auto
White Balance	: Auto	White Balance	: Auto
Digital Zoom Ratio	: 5.25	Digital Zoom Ratio	: 5.25
Focal Length In 35mm Format	: 406 mm	Focal Length In 35mm Format	: 406 mm
Lens Info	: 1.570000052-9mm f/1.5-2.8	Lens Info	: 1.570000052-9mm f/1.5-2.8
Lens Make	: Apple	Lens Make	: Apple
Lens Model	: iPhone 13 Pro	Lens Model	: iPhone 13 Pro
back triple camera 9mm f/2.8		back triple camera 9mm f/2.8	
Composite Image	: General Composite Image	Composite Image	: General Composite Image
GPS Latitude Ref	: North	GPS Latitude Ref	: North
GPS Longitude Ref	: West	GPS Longitude Ref	: West
GPS Altitude Ref	: Above Sea Level	GPS Altitude Ref	: Above Sea Level
GPS Speed Ref	: km/h	GPS Speed Ref	: km/h
GPS Speed	: 0	GPS Speed	: 0
GPS Img Direction Ref	: True North	GPS Img Direction Ref	: True North
GPS Img Direction	: 269.066803	GPS Img Direction	: 269.066803
GPS Dest Bearing Ref	: True North	GPS Dest Bearing Ref	: True North
GPS Dest Bearing	: 269.066803	GPS Dest Bearing	: 269.066803
GPS Date Stamp	: 2022:08:06	GPS Date Stamp	: 2022:08:06
GPS Horizontal Positioning Error	: 5.039480611 m	GPS Horizontal Positioning Error	: 5.039480611 m
Profile CMM Type	: Apple Computer Inc.	Profile CMM Type	: Apple Computer Inc.
Profile Version	: 4.0.0	Profile Version	: 4.0.0
Profile Class	: Display Device	Profile Class	: Display Device
Profile		Profile	
Color Space Data	: RGB	Color Space Data	: RGB
Profile Connection Space	: XYZ	Profile Connection Space	: XYZ
Profile Date Time	: 2022:01:01 00:00:00	Profile Date Time	: 2022:01:01 00:00:00
Profile File Signature	: acsp	Profile File Signature	: acsp
Primary Platform	: Apple Computer Inc.	Primary Platform	: Apple Computer Inc.
CMM Flags	: Not Embedded, Independent	CMM Flags	: Not Embedded, Independent
Device Manufacturer	: Apple Computer Inc.	Device Manufacturer	: Apple Computer Inc.
Device Model	:	Device Model	:
Device Attributes	: Reflective, Glossy, Positive, Color	Device Attributes	: Reflective, Glossy, Positive, Color
Rendering Intent	: Perceptual	Rendering Intent	: Perceptual
Connection Space Illuminant	: 0.9642 1 0.82491	Connection Space Illuminant	: 0.9642 1 0.82491
Profile Creator	: Apple Computer Inc.	Profile Creator	: Apple Computer Inc.
Profile ID	: ecfda38e388547c36db4bd4f7ada182f	Profile ID	: ecfda38e388547c36db4bd4f7ada182f
Profile Description	: Display P3	Profile Description	: Display P3
Profile Copyright	: Copyright Apple Inc., 2022	Profile Copyright	: Copyright Apple Inc., 2022
Media White Point	: 0.96419 1 0.82489	Media White Point	: 0.96419 1 0.82489
Red Matrix Column	: 0.51512 0.2412 -0.00105	Red Matrix Column	: 0.51512 0.2412 -0.00105
Green Matrix Column	: 0.29198 0.69225 0.04189	Green Matrix Column	: 0.29198 0.69225 0.04189
Blue Matrix Column	: 0.1571 0.06657 0.78407	Blue Matrix Column	: 0.1571 0.06657 0.78407
Red Tone Reproduction Curve	: (Binary data 32 bytes, use -b option to extract)	Red Tone Reproduction Curve	: (Binary data 32 bytes, use -b option to extract)
Chromatic Adaptation	: 1.04788 0.02292 -0.0502 0.02959 0.99048 -0.01706 -0.00923 0.01508 0.75168	Chromatic Adaptation	: 1.04788 0.02292 -0.0502 0.02959 0.99048 -0.01706 -0.00923 0.01508 0.75168
Blue Tone Reproduction Curve	: (Binary data 32 bytes, use -b option to extract)	Blue Tone Reproduction Curve	: (Binary data 32 bytes, use -b option to extract)
Green Tone Reproduction Curve	: (Binary data 32 bytes, use -b option to extract)	Green Tone Reproduction Curve	: (Binary data 32 bytes, use -b option to extract)
HEVC Configuration Version	: 1	HEVC Configuration Version	: 1
General Profile Space	: Conforming	General Profile Space	: Conforming
General Tier Flag	: Main Tier	General Tier Flag	: Main Tier
General Profile IDC	: Main Still Picture	General Profile IDC	: Main Still Picture

Gen Profile Compatibility Flags : Main Still Picture, Main 10, Main	Gen Profile Compatibility Flags : Main Still Picture, Main 10, Main
Constraint Indicator Flags : 176 0 0 0 0 0	Constraint Indicator Flags : 176 0 0 0 0 0
General Level IDC : 90 (level 3.0)	General Level IDC : 90 (level 3.0)
Min Spatial Segmentation IDC : 0	Min Spatial Segmentation IDC : 0
Parallelism Type : 0	Parallelism Type : 0
Chroma Format : 4:2:0	Chroma Format : 4:2:0
Bit Depth Luma : 8	Bit Depth Luma : 8
Bit Depth Chroma : 8	Bit Depth Chroma : 8
Average Frame Rate : 0	Average Frame Rate : 0
Constant Frame Rate : Unknown	Constant Frame Rate : Unknown
Num Temporal Layers : 1	Num Temporal Layers : 1
Temporal ID Nested : No	Temporal ID Nested : No
Image Width : 4032	Image Width : 4032
Image Height : 3024	Image Height : 3024
Image Spatial Extent : 4032x3024	Image Spatial Extent : 4032x3024
Rotation : 270	Rotation : 270
Image Pixel Depth : 8	Image Pixel Depth : 8
Auxiliary Image Type : urn:com:apple:photo:2020:aux:hdr:gainmap	Auxiliary Image Type : urn:com:apple:photo:2020:aux:hdr:gainmap
Media Data Size : 1158339	Media Data Size : 1158339
Media Data Offset : 3753	Media Data Offset : 3847
Run Time Since Power Up : 5 days 5:03:15	Warning : Invalid atom size
Aperture : 2.8	Run Time Since Power Up : 5 days 5:03:15
Image Size : 4032x3024	Aperture : 2.8
Megapixels : 12.2	Image Size : 4032x3024
Scale Factor To 35 mm Equivalent: 45.1	Megapixels : 12.2
Shutter Speed : 1/131	Scale Factor To 35 mm Equivalent: 45.1
Create Date : 2022:08:06 15:23:05.645+01:00	Shutter Speed : 1/131
Date/Time Original : 2022:08:06 15:23:05.645+01:00	Create Date : 2022:08:06 15:23:05.645+01:00
Modify Date : 2022:08:06 15:23:05+01:00	Date/Time Original : 2022:08:06 15:23:05.645+01:00
GPS Altitude : 105 m Above Sea Level	Modify Date : 2022:08:06 15:23:05+01:00
GPS Latitude : 52 deg 37' 26.86" N	GPS Altitude : 105 m Above Sea Level
GPS Longitude : 1 deg 4' 33.66" W	GPS Latitude : 52 deg 37' 26.86" N
Circle Of Confusion : 0.001 mm	GPS Longitude : 1 deg 4' 33.66" W
Field Of View : 5.1 deg	Circle Of Confusion : 0.001 mm
Focal Length : 9.0 mm (35 mm equivalent: 406.0 mm)	Field Of View : 5.1 deg
GPS Position : 52 deg 37' 26.86" N, 1 deg 4' 33.66" W	Focal Length : 9.0 mm (35 mm equivalent: 406.0 mm)
Hyperfocal Distance : 43.43 m	GPS Position : 52 deg 37' 26.86" N, 1 deg 4' 33.66" W
Light Value : 11.6	Hyperfocal Distance : 43.43 m
Lens ID : iPhone 13 Pro back triple camera 9mm f/2.8	Light Value : 11.6
	Lens ID : iPhone 13 Pro back triple camera 9mm f/2.8